



(19) 대한민국특허청(KR)

(12) 등록특허공보(B1)

(45) 공고일자 2020년06월08일

(11) 등록번호 10-2114941

(24) 등록일자 2020년05월19일

(51) 국제특허분류(Int. Cl.)
G06F 12/10 (2016.01) G06F 12/12 (2016.01)
G06F 13/24 (2006.01)

(52) CPC특허분류
G06F 12/1009 (2013.01)
G06F 12/1045 (2013.01)

(21) 출원번호 10-2016-7013936

(22) 출원일자(국제) 2014년10월24일
심사청구일자 2019년10월24일

(85) 번역문제출일자 2016년05월25일

(65) 공개번호 10-2016-0076532

(43) 공개일자 2016년06월30일

(86) 국제출원번호 PCT/US2014/062249

(87) 국제공개번호 WO 2015/061731

국제공개일자 2015년04월30일

(30) 우선권주장
61/896,092 2013년10월27일 미국(US)

(56) 선행기술조사문헌

US20130227248 A1

(뒷면에 계속)

전체 청구항 수 : 총 20 항

(73) 특허권자

어드밴스드 마이크로 디바이시즈, 인코포레이티드
미국 캘리포니아 95054 산타 클라라 어거스틴 드
라이브 2485

에이티아이 테크놀로지스 유엘씨

캐나다 온타리오 엘3티 7엑스6 마크햄 커머스 벨
리 드라이브 이스트 1

(72) 발명자

카리아나순드하람 비드하나탄

미국 캘리포니아 95138 산 조세 봉힐 코트 351

엔지 필립

캐나다 온타리오 엠5엔 1엘4 토론토 캐슬필드 애
버뉴 459

(뒷면에 계속)

(74) 대리인

박장원

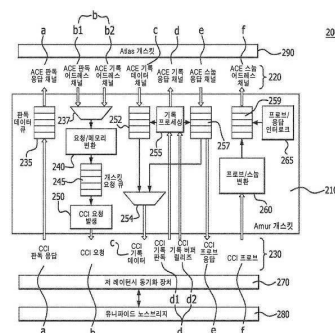
심사관 : 남윤권

(54) 발명의 명칭 입력/출력 메모리 맵 유닛 및 노스브리지

(57) 요약

본 발명은 새로운 원자 테스트[0] 및 OR 및 마스크를 통한 하드웨어에서의 페이지 테이블 액세스 및 더티 비트(dirty bit) 관리에 대해 제공한다. 본 발명은 또한 ACE 대 CCI 변환들을 가능하게 하는 개스킷에 대해 제공한다. 이러한 개스킷은 ACE 및 AND 간 변환 요청, 빅탐(victim) 및 프로브 충돌에 대한 교착 상태 회피, ARM 배리어 핸들링, 및 전력 관리 상호작용들을 더 제공한다. 본 발명은 또한 유니파이드 노스브리지를 교착 상태로 만드는 ARM 빅탐/ 프로브 충돌에 대한 솔루션을 제공한다. 이들 솔루션은 전용 라이트백 가상 채널, 4-홉 프로토콜을 사용하는 IO 요청들에 대한 프로브들, 및 빅탐들이 이들이 요청들을 전달함에 따라 데이터를 갖는 보다 오래된 요청들을 업데이트하는 MCT에서의 라이트백 재순서화 능력을 포함한다.

대표도



- | | |
|---|--|
| <p>(52) CPC특허분류
 G06F 12/12 (2013.01)
 G06F 13/24 (2013.01)
 G06F 2212/684 (2013.01)</p> <p>(72) 발명자
 찬 매기
 캐나다 온타리오 엘6씨 2와이6 마크햄 우드러프 로드 11
 쿠에바 빈센트
 캐나다 온타리오 엠4취 0에이2 토론토 스위트 421 버크브룩 피엘. 21
 아사로 앤쏘니
 캐나다 온타리오 엠1씨 4와이9 토론토 폰리취 트레일 10
 미르자 짐셰드
 캐나다 온타리오 엠6제이 0에이5 토론토 스위트 1505 퀸 에스티 웨스트 1171
 돈리 그리고리 더글라스
 미국 캘리포니아 95117 산 조세 씨니레이크 씨티. 3584
 브로우싸드 브라이언
 미국 텍사스 78739 오스틴 체리씨 드라이브 11616
 치엔 벤자민
 미국 캘리포니아 94555 프레몬트 나이스 티이알. 6056
 아디리 야니브
 이스라엘 리손 레지온 2엔디 에벤 헨 에스티.</p> | <p>(56) 선행기술조사문헌
 US20120317571 A1
 US20090006805 A1
 US20100223447 A1
 JP소화63298652 A
 JP평성02023458 A
 JP평성06236353 A</p> |
|---|--|
-

명세서

청구범위

청구항 1

더티 비트(dirty bit)들의 핸들링(handling)을 포함하는, 컴퓨터 시스템에서의 페이지 테이블(page table)들을 관리하기 위한 방법으로서, 상기 방법은,

적어도 하나의 가상 메모리 어드레스(virtual memory address)의 대응하는 물리적 어드레스(physical address)로의 매핑(mapping)을 제공하는 상기 페이지 테이블들 내의 하나 이상의 테이블 엔트리(table entry)들에 액세스(accessing)하는 단계와, 여기서

상기 액세스는 액세스 플래그(access flag)에 따라 이루어지고,

상기 액세스는 하나의 원자 업데이트(atomic update)에서 수행되도록 하기 위해 하드웨어(hardware)를 통해 일어나고,

상기 액세스는 상기 액세스 플래그 내의 적어도 하나의 액세스 비트(access bit)가 수정되도록 하고,

상기 액세스 플래그는 어드레스 변환(address translation)들을 위해 안전 및 비-안전 테이블 엔트리들을 나타내기 위해 사용되고,

상기 원자 업데이트는 단일 원자 동작(single atomic operation)이며;

상기 하나 이상의 액세스된 페이지 테이블들에서 식별된 메모리 어드레스를 수정하되 상기 메모리 어드레스와 그리고 상기 메모리 어드레스에 대해 수행된 하나 이상의 메모리 동작들에 기초하여 수정하는 단계와, 여기서

상기 수정은 상기 하나 이상의 메모리 동작들을 사용하여 상기 메모리 어드레스를 설정(setting) 및 클리어(clearing)하는 것을 포함하며; 그리고

상기 원자 업데이트에서의 상기 수정된 메모리 어드레스에 응답하여 상기 페이지가 클린(clean)인지 또는 더티(dirty)인지를 나타내는 상기 페이지 테이블 내의 더티 비트를 수정하는 단계를 포함하고,

상기 액세스 플래그 내의 상기 적어도 하나의 액세스 비트, 그리고 상기 더티 비트는 상기 원자 업데이트 동안 상기 액세스된 하나 이상의 테이블 엔트리들에 대해 업데이트되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 2

제1항에 있어서,

상기 수정은 단일 테이블 엔트리에 대해 수행되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 3

제1항에 있어서,

상기 수정은 하드웨어로 관리되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 4

제1항에 있어서,

상기 액세스 플래그가 0으로 설정된 변환 테이블 엔트리가 변환 색인 버퍼(Translation Lookaside Buffer, TLB)로 관독됨에 응답하여, 메모리 내의 상기 변환 테이블 엔트리의 상기 액세스 플래그 비트가 1로 설정되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 5

제1항에 있어서,

상기 액세스 플래그는 상기 안전 및 비-안전 변환들을 나타내기 위해 사용되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 6

제5항에 있어서,

액세스 플래그 필드(access flag field)는 ID_MMFR2[31:28]인 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 7

제1항에 있어서,

상기 액세스 플래그의 하드웨어 관리가 어드레스 변환의 스테이지에 대해 활성화되는 것에 응답하여, 대응하는 변환들에 대해 액세스 플래그 오류들이 발생되지 않는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 8

제1항에 있어서,

상기 방법은 또한, 상기 액세스에 기초하여 액세스 비트를 업데이트하는 것을 포함하는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 9

제8항에 있어서,

상기 액세스 비트 및 상기 더티 비트는 상기 단일 원자 동작 동안 블록 혹은 페이지 변환 테이블 엔트리로부터 업데이트되는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 10

제1항에 있어서,

상기 더티 비트가 더티를 나타낸 경우, 마지막 페이지 테이블 기술자(last page table descriptor)를 점검하고 클린을 나타내기 위해 상기 더티 비트를 변경하는 것을 특징으로 하는 컴퓨터 시스템에서의 페이지 테이블들을 관리하기 위한 방법.

청구항 11

컴퓨터 시스템으로서, 상기 컴퓨터 시스템은,

프로세서와; 그리고

하나 이상의 페이지 테이블들을 포함하는 메모리를 포함하고,

상기 프로세서는,

액세스 플래그에 따라, 적어도 하나의 가상 메모리 어드레스의 대응하는 물리적 어드레스로의 매핑을 제공하는 상기 페이지 테이블들 내의 하나 이상의 테이블 엔트리들에 액세스하는 것과, 여기서

하드웨어는 하나의 원자 업데이트에서 상기 하나 이상의 테이블 엔트리들에 액세스하고,

상기 하드웨어가 액세스하는 것은 상기 액세스 플래그 내의 적어도 하나의 액세스 비트가 수정 되도록 하고,

상기 액세스 플래그는 어드레스 변환들을 위해 안전 및 비-안전 테이블 엔트리들을 나타내기

위해 사용되고,

상기 원자 업데이트는 단일 원자 동작이며;

상기 하나 이상의 액세스된 페이지 테이블들에서 식별된 메모리 어드레스를 수정하되 상기 메모리 어드레스와 그리고 상기 메모리 어드레스에 대해 수행된 하나 이상의 메모리 동작들에 기초하여 수정하는 것과, 여기서

상기 하드웨어가 수정하는 것은 상기 하나 이상의 메모리 동작들을 사용하여 상기 메모리 어드레스를 설정 및 클리어하는 것을 포함하며; 그리고

상기 원자 업데이트에서의 상기 수정된 메모리 어드레스에 응답하여 상기 페이지가 클린인지 또는 더티인지를 나타내는 상기 페이지 테이블 내의 더티 비트를 수정하는 것을

행하도록 되어 있고,

상기 액세스 플래그 내의 상기 적어도 하나의 액세스 비트, 그리고 상기 더티 비트는 상기 원자 업데이트 동안 상기 액세스된 하나 이상의 테이블 엔트리들에 대해 업데이트되는 것을 특징으로 하는 컴퓨터 시스템.

청구항 12

제11항에 있어서,

상기 프로세서는 또한, 단일 테이블 엔트리에 대해 상기 수정을 수행하도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 13

제11항에 있어서,

상기 컴퓨터 시스템은 하드웨어로 상기 수정을 관리하는 것을 특징으로 하는 컴퓨터 시스템.

청구항 14

제11항에 있어서,

상기 프로세서는 또한, 상기 액세스 플래그가 0으로 설정된 변환 테이블 엔트리가 변환 색인 버퍼(TLB)로 관독됨에 응답하여, 메모리 내의 상기 변환 테이블 엔트리의 상기 액세스 플래그 비트를 1로 설정하도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 15

제11항에 있어서,

상기 액세스 플래그는 상기 안전 및 비-안전 변환들을 나타내기 위해 사용되는 것을 특징으로 하는 컴퓨터 시스템.

청구항 16

제15항에 있어서,

액세스 플래그 필드는 ID_MMFR2[31:28]인 것을 특징으로 하는 컴퓨터 시스템.

청구항 17

제11항에 있어서,

상기 프로세서는, 상기 액세스 플래그의 하드웨어 관리가 어드레스 변환의 스테이지에 대해 활성화되는 것에 응답하여, 대응하는 변환들에 대해 액세스 플래그 오류들을 발생시키지 않도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 18

제11항에 있어서,

상기 프로세서는 또한, 상기 액세스에 기초하여 액세스 비트를 업데이트하도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 19

제18항에 있어서,

상기 프로세서는 또한, 상기 액세스 비트 및 상기 더티 비트를 상기 단일 원자 동작 동안 블록 혹은 페이지 변환 테이블 엔트리로부터 업데이트하도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 20

제11항에 있어서,

상기 더티 비트가 더티를 나타낸 경우, 마지막 페이지 테이블 기술자를 점검하고 클린을 나타내기 위해 상기 더티 비트를 변경하도록 되어 있는 것을 특징으로 하는 컴퓨터 시스템.

청구항 21

삭제

청구항 22

삭제

발명의 설명

기술 분야

[0001] 관련 출원들에 대한 상호 참조

[0002] 본 출원은 입력/출력 메모리 맵 유닛 및 노스브리지라는 명칭의, 2013년 10월 27일에 출원된, 미국 가 특허 출원 제61/896,092호의 혜택을 주장하며, 이의 내용이 이에 의해 본 출원에서 참조로 인용된다.

[0003] 개시된 실시예들은 일반적으로 메모리 맵 및 변환기에, 그리고 특히, 입력/출력 메모리 맵 유닛 및 노스브리지에 관한 것이다.

배경 기술

[0004] 페이지 테이블은 가상 어드레스 및 물리적 어드레스 간 매핑을 저장하기 위한 컴퓨터 운영 체제에서 가상 메모리 시스템에 의해 사용되는 데이터 구조이다. 가상 어드레스들은 액세스 프로세스에 의해 사용되는 한편, 물리적 어드레스들은 하드웨어에 의해 또는 보다 구체적으로 RAM을 위해 사용된다.

[0005] 가상 메모리를 사용하는 운영 체제들에서, 모든 프로세스는 그것이 메모리의 큰, 연결 섹션들과 작동하고 있다는 느낌을 받는다. 실제로는, 각 프로세스의 메모리는 물리적 메모리의 상이한 영역들에 걸쳐 분산될 수 있거나, 또는 다른 저장장치, 전형적으로 하드 디스크로 이동(페이지 옮김)되었을 수도 있었다.

[0006] 프로세스가 이의 메모리에서의 데이터에 대한 액세스를 요청할 때, 프로세스에 의해 제공되는 가상 어드레스를 데이터가 저장되는 실제 메모리의 물리적 어드레스로 매핑하는 것은 운영 체제의 책임이다. 페이지 테이블은 운영 체제가 가상 어드레스들의 물리적 어드레스들로의 이의 매핑들을 저장하는 곳이며, 각 매핑은 또한 페이지 테이블 엔트리(PTE; page table entry)로서 알려져 있다.

[0007] 가상을 물리적 어드레스로 변환 시 액션들이 취해질 수 있다. 각 변환은 TLB 손실이 발생하는 경우 재시작되어, 루업이 하드웨어를 통해 정확하게 발생할 수 있다.

[0008] CPU의 메모리 관리 유닛(MMU)은 운영 시스템의 페이지 테이블로부터 최근에 사용된 매핑들의 캐시를 저장한다. 이는 연관 캐시인, 변환 색인 버퍼(TLB; translation lookaside buffer)이라 칭해진다.

[0009] 가상 어드레스가 물리적 어드레스로 변환될 필요가 있을 때, TLB가 먼저 탐색된다. 매치가 발견되는 경우(TLB 히트), 물리적 어드레스가 리턴되고 메모리 액세스가 계속될 수 있다. 그러나, 아무런 매치도 없는 경우(TLB 누락이라 칭해짐), 핸들러는 전형적으로 매핑이 존재하는지 여부를 살피기 위해 페이지 테이블에서의 어드레스 매

핑을 검색할 것이다(페이지 워크). 매핑이 존재하는 경우, 그것은 TLB로 기록되고(하드웨어 액세스 메모리가 가상 메모리 시스템에서의 TLB를 통합에 따라, 이는 반드시 수행되어야 한다), 폴팅 명령(faulting instruction)이 재시작된다(이는 또한 병렬적으로 일어날 수 있다). 이러한 후속 변환은 TLB 히트를 발견할 것이고, 메모리 액세스가 계속될 것이다.

[0010] 페이지 테이블 색인은 두가지 이유로 실패할 수 있다. 첫번째는 가상 어드레스에 이용가능한 변환이 존재하지 않는 경우이며, 이는 가상 어드레스가 무효하다는 것을 의미한다. 이는 전형적으로 프로그래밍에 의해 발생할 것이고, 운영 체제는 문제를 처리할 어떤 액션을 취해야 한다. 최신 운영 체제들 상에서, 그것은 문제가 되는 프로그램에 세그멘테이션 오류를 송신할 것이다.

[0011] 페이지 테이블 색인은 또한 페이지가 물리적 메모리에 상주하지 않는 경우 실패할 수 있다. 이는 요청된 페이지가 다른 페이지를 위한 공간을 만들기 위해 물리적 메모리 밖으로 이동된 경우 발생할 것이다. 이러한 경우 페이지가 매체 이룰때면 하드 디스크 드라이브 상에 위치되는 보조 기억장치(이러한 보조 기억장치, 또는 "백킹 기억장치"는 보통 그것이 디스크 파티션인 경우 "교체 파티션"이라 칭해지거나, 또는 그것이 파일인 경우 교체 파일, "교체파일" 또는 "페이지 파일"이라 칭해진다)로 페이지 옮김된다. 이것이 일어날 경우 페이지는 디스크로부터 취해지고 물리적 메모리에 다시 놓여질 필요가 있다. 유사한 메커니즘이 요구에 따라 가상 메모리에 매칭되고 물리적 메모리로 로딩되는, 메모리 매핑 파일들을 위해 사용된다.

[0012] 물리적 메모리가 가득 차지 않을 때 이는 간단한 작동이다; 페이지가 물리적 메모리로 라이트 백되고, 페이지 테이블 및 TLB가 업데이트되며, 명령이 재시작된다. 그러나, 물리적 메모리가 가득 찰 때, 물리적 메모리에서의 하나 이상의 페이지가 요청된 페이지를 위한 공간을 만들기 위해 페이지 옮김될 필요가 있을 것이다. 페이지 테이블은 이전에 물리적 메모리에 있었던 페이지들이 더 이상 거기에 있지 않다는 것을 표시하기 위해, 그리고 디스크 상에 있었던 페이지가 이제 물리적 메모리에 있다는 것을 표시하기 위해 업데이트될 필요가 있다. TLB는 또한 그것에서 페이지 옮김 페이지의 제거를 포함하여, 업데이트될 필요가 있고, 명령이 재시작된다. 어떤 페이지를 페이지 옮김할지 여부는 페이지 교체 알고리즘들의 문제이다. 그러나, 이들 알고리즘은 하드웨어 관리의 사용을 통해 제공될 수 있는 필요한 능력들을 제공하는데 실패한다.

[0013] ARM 프로세서들은 메모리 서브시스템들과 상호작용하기 위해 ACE 프로토콜을 사용한다. AMD x86 메모리 서브시스템(노스브리지)는 CCI 프로토콜을 사용한다. ARM ACE 프로토콜 및 AMD CCI 프로토콜은 호환가능하지 않다. 실제로, 함께 브리지될 필요가 있는 프로토콜들 간 다수의 차이가 존재한다. 몇몇 예가 다음을 포함한다: 요청, 응답, 및 프로브/스noop 인코딩들; ACE 프로토콜은 메모리 서브시스템으로의 기록 푸시 데이터를 갖고 CCI 프로토콜은 그것이 준비가 되었을 때 메모리 서브시스템 요청 기록 데이터를 갖는다; - 프로브/빅타임 교착 상태; ACE 프로토콜은 프로세서들이 수정된 데이터 카피를 프로브 응답에 관해 메모리 서브시스템에 다시 전달하게 하고 CCI는 이를 가능하게 하지 않는다; ACE 및 CCI 전력 관리 제어 신호들 및 핸드셰이크들(handshakes)은 실질적으로 상이하다; 그리고 양 프로토콜은 프로세서 및 메모리 서브시스템이 상이한 클록 주파수들로 작동하게 하며 CCI는 보다 빠른 클록 도메인에서 클록 배율 조절을 핸들링하기 위해 클록 이네이블 기법을 사용하고 ACE는 준비/유효 핸드셰이크를 사용한다. 이들 두 프로토콜을 효과적으로 브리지할 수 있는 시스템이 요구된다.

[0014] ARM 기반 시스템은 cHT기반 시스템과 상이하게 CPU 라이트백들을 갖고 프로브/스noop 경합 조건들을 핸들링한다. cHT 기반 시스템에서, CPU는 어떤 종속성 없이 프로브 응답을 항상 제공하기를 요청했다. 프로브가 CPU 빅타임을 히팅하는 경우, CPU는 빅타임이 이미 송신되었는지(그리고 이후에 취소될지)를 나타내고 프로브 응답으로 라이트백 데이터를 제공한다. 빅타임이 아직 세팅되지 않은 경우, 이의 상태는 프로브 유형에 따라 저하된다. ARM 시스템에서, CPU는 라이트백이 완료될 때까지 프로브 응답을 블로킹하고 전달하지 않을 것이다. 이는 CPU/클러스터 내에 큐잉되고 아직 문제되지 않은 프로브들에 대해서도 발생할 수 있다. 따라서 유니파이드 노스브리지가 ARM 빅타임/프로브 충돌 핸들링에 기초하여 교착 상태로 될 수 있는 문제가 종료된다.

발명의 내용

[0015] 본 발명은 새로운 원자 테스트[0] 및 OR 및 마스크를 통한 하드웨어에서의 페이지 테이블 액세스 및 더티 비트 관리에 대해 제공한다.

[0016] 본 발명은 또한 ACE 대 CCI 변환들을 가능하게 하는 개스킷에 대해 제공한다. 이러한 개스킷은 ACE 및 AND 간 변환 요청, 빅타임(victim) 및 프로브 충돌에 대한 교착 상태 회피, ARM 배리어 핸들링, 및 전력 관리 상호작용들을 더 제공한다.

[0017] 본 발명은 또한 유니파이드 노스브리지를 교착 상태로 만드는 ARM 빅타임/ 프로브 충돌에 대한 솔루션을

제공한다. 이들 솔루션은 전용 라이트백 가상 채널, 4-홉 프로토콜을 사용하는 IO 요청들에 대한 프로브들, 및 빅티들이 이들이 요청들을 전달함에 따라 데이터를 갖는 보다 오래된 요청들을 업데이트하는 MCT에서의 라이트백 재순서화 능력을 포함한다.

도면의 간단한 설명

[0018]

첨부 도면들과 함께 예로서 제공되는, 이하의 설명으로 보다 상세하게 이해될 수 있으며 여기서:

도 1은 하나 이상의 개시된 실시예가 구현될 수 있는 예시적인 디바이스의 블록도이다;

도 2는 유니파이드 노스브리지 및 아틀라스 클러스터 간 트랜잭션들의 변환들을 제공하기 위한 개스킷을 포함하는 대표적인 시스템을 예시한다;

도 3은 인터럽트들이 전체 C6 엔트리를 통해 마스킹되는 흐름을 예시한다;

도 4는 기존 P상태 로직과의 연결성의 예시적인 다이어그램을 예시한다;

도 5는 WFI를 사용하는 P상태 시퀀스의 예시적인 다이어그램을 예시한다;

도 6은 WFE를 사용하는 P-상태 시퀀스의 예시적인 다이어그램을 예시한다;

도 7은 개스킷 작업 거부를 사용하는 P상태 시퀀스의 예시적인 다이어그램을 예시한다;

도 8은 개스킷 및 유니파이드 노스브리지 간 NBSYN에 걸친 모든 인플라이트 트래픽을 드레인하는 것을 보장하기 위해 제공되는 방법(800)을 예시한다; 그리고

도 9는 교착 상태의 흐름을 예시한다.

발명을 실시하기 위한 구체적인 내용

[0019]

본 발명은 새로운 원자 테스트[0] 및 OR 및 마스크를 통한 하드웨어에서의 페이지 테이블 액세스 및 더티 비트 관리에 대해 제공한다. 본 발명은 또한 ACE 대 CCI 변환들을 가능하게 하는 개스킷에 대해 제공한다. 이러한 개스킷은 ACE 및 AND 간 변환 요청, 빅티(victim) 및 프로브 충돌에 대한 교착 상태 회피, ARM 배리어 핸들링, 및 전력 관리 상호작용들을 더 제공한다. 본 발명은 또한 유니파이드 노스브리지를 교착 상태로 만드는 ARM 빅티/프로브 충돌에 대한 솔루션을 제공한다. 이들 솔루션은 전용 라이트백 가상 채널, 4-홉 프로토콜을 사용하는 IO 요청들에 대한 프로브들, 및 빅티들이 이들이 요청들을 전달함에 따라 데이터를 갖는 보다 오래된 요청들을 업데이트하는 MCT에서의 라이트백 재순서화 능력을 포함한다.

[0020]

도 1은 하나 이상의 개시된 실시예가 구현될 수 있는 예시적인 디바이스(100)의 블록도이다. 디바이스(100)는 예를 들어, 컴퓨터, 게이밍 디바이스, 핸드헬드 디바이스, 셋-탑 박스, 텔레비전, 모바일 전화, 또는 태블릿 컴퓨터를 포함할 수 있다. 디바이스(100)는 프로세서(102), 메모리(104), 저장장치(106), 하나 이상의 입력 디바이스(108), 및 하나 이상의 출력 디바이스(110)를 포함한다. 상기(100)는 또한 입력 드라이버(112) 및 출력 드라이버(114)를 선택적으로 포함할 수 있다. 디바이스(100)는 도 1에 도시되지 않은 추가적인 구성요소들을 포함할 수 있다.

[0021]

프로세서(102)는 중앙 처리 장치(CPU 102), 그래픽 처리 장치(GPU), 동일한 다이 상에 위치되는 CPU 및 GPU, 각 프로세서 코어가 CPU 또는 GPU일 수 있는, 하나 이상의 프로세서 코어를 포함할 수 있다. 메모리(104)는 프로세서(102) 상에 위치될 수 있거나, 프로세서(102)와 별도로 위치될 수 있다. 메모리(104)는 휘발성 또는 비-휘발성 메모리(104), 예를 들어, 랜덤 액세스 메모리(104)(RAM), 동적 RAM, 또는 캐시를 포함할 수 있다.

[0022]

저장 장치(106)는 고정식 또는 착탈가능식 저장 장치, 예를 들어, 하드 디스크 드라이브, 고체 상태 드라이브, 광 디스크, 또는 플래시 드라이브를 포함할 수 있다. 입력 디바이스들(108)은 키보드, 키패드, 터치 스크린, 터치 패드, 검출기, 마이크로폰, 가속도계, 자이로스코프, 생체 인식 스캐너, 또는 네트워크 연결부(예를 들어, 무선 I4E 802 신호들의 전송 및/또는 수신을 위한 무선 로컬 영역 네트워크 카드)를 포함할 수 있다. 출력 디바이스(110)은 디스플레이, 스피커, 프린터, 햅틱 피드백 디바이스, 하나 이상의 광 안테나, 또는 네트워크 연결부(예를 들어, 무선 I4E 802 신호들의 전송 및/또는 수신을 위한 무선 로컬 영역 네트워크 카드)를 포함할 수 있다.

[0023]

입력 드라이버(112)는 프로세서(102) 및 입력 디바이스들(108)과 통신하고, 프로세서(102)가 입력 디바이스들(108)로부터 입력을 수신하게 한다. 출력 드라이버(114)는 프로세서(102) 및 출력 디바이스들(110)과 통신하고,

프로세서(102)가 출력 디바이스들(110)로부터 출력을 송신하게 한다. 입력 드라이버(112) 및 출력 드라이버(114)는 선택적 구성요소들이라는 것, 및 디바이스(100)는 입력 드라이버(112) 및 출력 드라이버(114)가 존재하지 않는 경우와 동일한 방식으로 작동할 것이라는 것이 주의된다.

[0024] 페이지 테이블에서의 테이블 엔트리들에 액세스하는 것에 있어서, 하드웨어는 액세스 비트를 1로 셋팅할 수 있다. Test[0]andOR 원자(또는 Op2가 모드 1들인 Test[0]SetandClr)가 사용될 수 있고 레지스터들에 의해 이네이블될 수 있다. 더티 비트 메커니즘이 유익할 수 있다. 예를 들어, 요청이 기록 허가를 필요로 하나 AP[2]가 1'b1일 때, 하드웨어는 DBM 비트(마지막 페이지 테이블 기술자의 비트 51)를 검사하고 더티 비트가 1'b1로 셋팅된 경우 AP[2]를 1'b0으로 셋팅할 수 있다. Test[0]SetandClr atomic.Test[0] Set and Clr: (이러한 원자 op의 양 32b 및 64b 버전이 지지된다) 이러한 단계에서 사용될 수 있다.

[0025] 예를 들어, 이하의 동작이 페이지 테이블들을 관리하기 위해 하드웨어에서의 기능으로서 사용될 수 있다.

[0026] $Mem[Addr] = Mem[Addr][0] ? ((Mem[Addr] \mid Op1) \& Op2): Mem[Addr]; \quad \text{식 } 1$

[0027] 식 1은 메모리 1 (Mem[Addr])을 수정하기 위해, 비트 0 = 테스트 비트인 경우 세트는 작동을 수행하고, 그렇지 않을 경우 어떤 동작도 하지 않는다(Mem[Addr][0]). 동작은 메모리 어드레스 상에서 작동하는 오퍼랜드들 Op1 및 Op2이다. 예를 들어, 메모리 어드레스는 Op1 및 Op2에 기초하여 ADDR로 설정 및 클리어될 수 있다. 이는 페이지 테이블을 관리하기 위해 하드웨어에서의 기능을 제공한다.

[0028] 더티 비트 메커니즘은 액세스 비트 메커니즘이 또한 이네이블되는 경우 이네이블될 수 있다. 준비가 된 더티 비트 메커니즘으로, 식 2는 AP[2]가 다음을 통해 페이지가 더티인지 또는 클린인지를 나타내게 한다:

[0029] $AP[2] = 1 - \text{페이지 클린}; AP[2] = 0 - \text{페이지 더티} \quad \text{식 } 2$

[0030] 양 액세스 비트 및 AP[2] 비트는 단일 Test[0]SetandClr 원자 동작에서 동일한 블록/페이지 변환 테이블 엔트리로부터의 업데이트들일 수 있다.

[0031] 액세스 플래그를 관리하기 위해 하드웨어를 사용하는 것은 유익할 수 있다. 예를 들어, 안전 및 비-안전 PL1&0 스테이지 1 변환들에 대해, 구현은 액세스 플래그의 하드웨어 관리를 제공할 수 있다. 이러한 경우, 0으로 셋팅된 액세스 플래그를 갖는 변환 테이블 엔트리가 변환 색인 버퍼(TLB)로 판독되는 경우, 하드웨어는 메모리에서의 변환 테이블 엔트리의 액세스 플래그 비트에 1을 기록한다. 안전 및 비-안전 PL1&0 스테이지 1 변환들을 위한 액세스 플래그의 하드웨어 관리를 제공하는 구현은: 이러한 구현 선택을 나타내기 위해, 하드웨어 액세스 플래그, ID_MMFR2[31:28]를 사용할 수 있고; 액세스 플래그의 하드웨어 관리를 이네이블하기 위해 비트를 1로 셋팅함으로써 SCTLR.HA 비트를 구현할 수 있다.

[0032] 숏트-기술자 변환 테이블 형식을 사용할 때, 액세스 플래그의 사용을 이네이블하기 위해, SCTLR.AFE가 1로 셋팅되고, 액세스 플래그의 하드웨어 관리를 이네이블하기 위해 SCTLR.HA가 1로 셋팅되는 경우 액세스 플래그의 하드웨어 관리가 수행된다. SCTLR의 बैंकिंग(banking)은 비트들이 안전 및 비-안전 어드레스 변환들과 독립적으로 정의되게 하여 안전/비안전 비트들에 대해 상이한 결정을 가능하게 한다. 액세스 플래그의 하드웨어 관리가 어드레스 변환의 스테이지에 대해 이네이블될 때, 대응하는 변환들에 대해 어떤 액세스 플래그 오류들도 발생되지 않는다. 이는 하나의 원자 업데이트에서의 업데이트를 허용하고 하드웨어에서 관리되지 않는 경우 발생할 수 없다.

[0033] 본 발명은 동기화 장치 및 코어 클러스터 간 중앙 처리 장치(CPU 102) 클록 도메인에 놓여질 "개스킷"의 실시예들에 관한다. 그 점에서 개스킷은 트랜잭션들의 캐시 코히런트 인터커넥트(CCI; cache coherent interconnect) 트랜잭션으로의 그리고 시스템 프로브들의 스눅들로의 변환을 제공할 수 있다. 덧붙여, 개스킷은 코어 클러스터가 유니파이드 노스브리지(UNB)/코히런트 하이퍼트랜스포트(cHT) 환경과 상호작용하는데 필요한 메모리 맵, 전송 크기들, 및 다른 기능들을 변환할 수 있다.

[0034] 도 2를 참조하면, 유니파이드 노스브리지(280) 및 아틀라스 클러스터(290) 간 트랜잭션들의 변환들을 제공하기 위한 개스킷(210)을 포함하는 대표적인 시스템(200)이 도시된다. 도 2에 도시된 바와 같이, 아틀라스 클러스터(290)는 아틀라스 버스 인터페이스에 다수의 채널(220)을 포함할 수 있다. 이들 채널(220)은 ACE 판독 응답 채널(220a), ACE 판독 어드레스 채널(220b1), ACE 기록 어드레스 채널(220b2), ACE 데이터 채널(220c), ACE 기록 응답 채널(220d), ACE 스눅 응답 채널(220e), 및 ACE 스눅 어드레스 채널(220f)을 포함할 수 있다. 개스킷(210)을 사용하여 아틀라스 클러스터(290) 채널들(220)의 각각이 CCI(230)를 통해 유니파이드 노스브리지(280)

로 변환될 수 있다. 예를 들어, 채널(220a)은 CCI 판독 응답(230a)으로 변환될 수 있고, 채널(220b1 및 220b2, 총괄적으로 220b)은 CCI 요청(230b)으로 변환될 수 있고, 채널(220c)은 CCI 기록 데이터(230c)로 변환될 수 있고, 채널(220d)은 CCI 기록 판독(230d1) 및 CCI 기록 버퍼 릴리즈(230d2)(총괄적으로 230d)로 변환될 수 있고, 채널(220e)은 CCI 프로브 응답(230e)으로 변환될 수 있으며, 채널(220f)은 CCI 프로브(230f)로 변환될 수 있다.

[0035] 저 레이턴시 동기화 장치(270)는 본 출원에 더 설명될 바와 같이 CCI 버스(230)에서 유니파이드 노스브리지(280)까지 사용될 수 있다.

[0036] CPU(102)로부터의 요청들은 어드밴스드 익스텐시블 인터페이스(AXI) 코히런스 익스텐션(ACE) 명령들로부터 동등한 CCI 명령들로 변환될 필요가 있다. 일반적으로, 각 ARM 요청 유형은 특정한 CCI 요청 유형으로 매핑된다. ARM 명령 유형은 판독 및 기록 어드레스 채널들의 AxDomain 및 AxSnoop 필드들에서 인코딩된다. 개스킷(210) 내에서 특정한 변환은 다수의 상이한 형태를 취할 수 있다. 예를 들어 CCI 판독 응답(230a)로부터 ACE 판독 응답 채널(220a)로 변환하는 것에 있어서, 판독 데이터 큐(235)가 구현될 수 있다.

[0037] ACE 판독 어드레스(220b1) 및 ACE 기록 어드레스 채널(220b2)로부터 변환하는 것에 있어서 이를테면 멀티플렉서(237)를 사용함으로써 조합되고 그 후 요청/메모리 변환기(240) 내로 개스킷 요청 큐(245)를 통해 CCI 요청 발생기(250)로 CCI 요청(230b)으로 입력할 수 있다.

[0038] CCI 프로브(230f)로부터 변환하는 것에 있어서 개스킷은 ACE 스누프 어드레스 채널(220f) 상에 출력하기 위해, 큐(259)로 출력하는 프로브/스누프 변환기(260)를 포함하며, 큐(259)는 또한 입력으로서 프로브/응답 인터록(265)로부터의 입력을 수신한다.

[0039] CCI 기록 데이터 및 CCI 판독 데이터(230d)로부터 ACE 기록 응답 채널(220d)로 변환하는 것에 있어서, 개스킷(210) 내 기록 프로세서(255)가 사용될 수 있다. ACE 스누프 응답 채널(220e)은 큐(257) 내로 입력하며, 이는 데이터를 멀티플렉서(254)로 제공하고 CCI 프로브 응답(230e)으로 출력하며, 기록 프로세서(255)로부터 입력을 수신한다. ACE 기록 데이터 채널(220c)은 큐(252)로 입력하며, 이는 멀티플렉서(254)로 출력하며, 기록 프로세서(255)로부터 입력을 수신한다. 멀티플렉서(254)는 CCI 기록 데이터(230c)로 출력한다.

[0040] 표 1에 도시된 이하의 트랜잭션들이 사용될 수 있다.

표 1

ACE 트랜잭션	CCI 트랜잭션												
	RdByte	RdDword	WrByte	WrDword	RdBlk	RdBlkS	RdBlkM	ApvWr	Alloc	WrVidBlk	ClvCbk	ClvD	Lock
ReadOnce		X			X	X	X						
ReadClean					X	X							
ReadNotSharedDirty					X	X	X						
ReadShared					X	X	X						
ReadUnique							X						
CleanInvalid													X
MakeInvalid									X				X
CleanShared													X
ReadNoSnoop		X											
CleanUnique											X		X
MakeUnique									X				X
WriteNoSnoop				X									
WriteUnique			X	X									
WriteLineUnique				X									
WriteBack										X			
WriteClean										X			
충족											X		
X -> Preferred Xlate													
X -> Alternate Xlate													

[0041]

[0042] 표 1에 도 2의 경로로 수행될 수 있는 변환들이 도시된다. ACE 변환이 표의 좌변 상에 도시되고 CCI 변환이 상측에 도시된다. 바람직한 변환이 녹색(보다 어두운 음영)으로 도시되고 대안적인 변환이 복숭아색(보다 밝은 음영)으로 도시된다.

[0043] 버스 연결들은 비트 43까지의 어드레스를 지지할 수 있으며, CCI는 비트 47까지의 어드레스들을 지지한다. 덧붙여, 프로세서는 가득 찬 어드레스 공간을 사용하지 않을 수 있고 따라서 상측 4 비트 중 하나 이상이 사용되지 않을 수 있으며 제로인 것으로 추정될 수 있다. 이들 그 외 구동되지 않는 상측 비트들이 제로로 유도되거나,

또는 ACE 속성들을 캐리하기 위해 오버로딩될 수 있다.

[0044] 현재 정의된 오버로드들은 "안전" 또는 "비-안전" 액세스를 나타내는 AxPROT[1]를 캐리할 Bit [44]를 포함한다 (각각, 0 또는 1). 이는 बैं킹된 레지스터 액세스를 위해 제네릭 인터럽트 컨트롤러(GIC)에 의해 사용된다. 이러한 비트는 임의의 다른 목적지들로 포워딩되기 전 SRI 로직에 의해 제로로 될 수 있다.

[0045] 설명된 바와 같이 요청들을 변환하는 것에 추가하여, 개스킷(210)은 ACE 태그들을 CCI 태그들로 변환하도록 요청될 수 있거나 그렇지 않을 수 있다. ACE 태그 공간이 CCI 태그 공간 내에 딱 맞으면, 태그들은 변경되지 않고 개스킷(210)을 통해 전달될 수 있다. ACE 프로토콜이 둘 이상의 트랜잭션이 순서대로 유지됨을 나타내기 위해 동일한 태그의 재사용을 허용하더라도, 유니파이드 노스브리지(280)는 일반적으로 순서를 유지하지 않는다. 결과적으로, 동일한 태그/id를 공유하는 ACE 요청들은 중복 태그들을 갖는 트랜잭션들이 유니파이드 노스브리지(280)에 이르는 것을 방지하는 개스킷(210) 내에서 직렬화될 수 있다.

[0046] 요청/프로브 조합들의 통합된 테이블이 표 2에 제공된다.

표 2

	트랜잭션/스noop	상태 변경	요청된/대안적인 스noop들							
			ReadOnce	ReadClean	ReadNotSharedDirty	ReadShared	ReadUnique	CleanInvalid	MakeInvalid	CleanShared
읽기 요청	ReadOnce	없음	x	x	x	x	x	x		x
	ReadClean	공유/무효		x	x	x	x	x		
	ReadNotSharedDirty	공유/무효		x	x	x	x	x		
	ReadShared	공유/무효		x	x	x	x	x		
	ReadUnique	무효					x	x		
	CleanInvalid	무효					x	x		
	MakeInvalid	무효							x	
	CleanShared	클린/무효					x	x		x
	ReadNoSnoop	없음	Not Snooped							
	CleanUnique	무효					x	x		
쓰기 요청	MakeUnique	무효							x	
	WriteNoSnoop	없음	Not Snooped							
	WriteUnique	무효					x	x		
	WriteNotUnique	무효							x	
	WriteBack	없음	스noop되지 않음							
	WriteClean	없음	스noop되지 않음							
	Evict	없음	스noop되지 않음							
x -> 바람직한 스noop										
x -> 대안적인 스noop										

[0047]

[0048] 표 2에 요청/프로브 조합들의 통합된 테이블이 도시된다. 이들은 ID 스noop들이 발생하는 ARM 트랜잭션을 포함할 수 있다. 녹색 스noop들(보다 어두운 박스들)이 선호되는 한편 복숭아색 스noop들(보다 밝은 박스들)이 대안적인 스noop들이다.

[0049] CCI 인코딩된 프로브들(230f)이 프로브/스noop 변환기(260)를 통해 동등한 ACE 스noop 트랜잭션으로 변환될 수 있다. ACE 프로토콜은 스noop 채널 상에서 통상 판독 명령들 및 전용 캐시 유지보수 명령들의 조합을 사용한다. 표 3의 변환들이 사용될 수 있다.

표 3

CCI 프로브	ReadOnce	ReadClean	ReadNotSharedDirty	ReadShared	ReadUnique	CleanInvalid	MakeInvalid	CleanShared
비변경	X							
공유				X				
무효화						X		
저하								X
이주								
이웃								
예비 할당1								
예비 할당2								
더티 제공	Y	Y	Y	Y	Y	Y	N*	Y
DRAM 업데이트	N	N*	N*	N	N	Y	N*	Y
유지 상태들	S	CS	MES	MOES	ME	—	—	CS

[0050]

[0051] 표 3은 사용될 수 있는 변환들을 제공한다. 변환들을 ACE 스놉에 기초한 CCI 프로브 발생 세부 사항을 정의한다.

[0052] 단일 CPU(102) 클러스터가 사용되는 경우, 개스킷(210)에 의해 수신되는 프로브들은 상기 경우들로 제한하는 IO/그래픽 처리 유닛(GPU) 요청들의 결과일 수 있다.

[0053] 개스킷(210)이 CCI 인터페이스로부터 판독 응답(230a)을 수신할 때, 클러스터(290)로 ACE 버스(220a)에 대한 동등한 응답을 발생시킬 필요가 있다. 특정한 상황들에서, 개스킷(210)은 데이터의 모든 비트들(beats)이 CCI 인터페이스(230a)로부터 수신될 때까지 ACE 응답(220a)을 시작하는 것을 지연시킬 수 있다. 이것을 필요로 함에 따라 몇몇 상황이 확인된다. 버스트 순서들은 상이할 수 있다. RdBkX 트랜잭션들은 ACE 버스(220)에 의해 예상되는 래핑(wrapping) 버스트 순서로 데이터를 제공한다. 에러 검사 정정(ECC) 에러들은 응답의 재생으로 귀결될 수 있다(즉 CCI 판독 데이터가 불완전할 수 있다).

[0054] 시스템 메모리(104) 맵은 목적들 이를테면 인터럽트 컨트롤러 디코드 등을 위해 IO와 대조적으로 DRAM을 확인하기 위해 사용되어야 한다. ARM 환경은 DRAM을 비- DRAM 어드레스들을 구별하기 위해 그리고 비-DRAM 공간을 더 세분하기 위해 ARM 메모리 맵을 사용한다. cHT 환경들은 어드레스 영역 유형들을 나타내기 위해 명백한 I/O 및 다른 신호들을 사용한다. 개스킷(210)은 SRI에 대한 각 요청으로 송신될 예상 보조 어드레스 유형 신호들을 생성하기 위해 메모리 맵을 사용할 수 있다. 메모리 맵은 ARM 트랜잭션들을 cHT로 변환하기 위해 사용될 수 있다. 어드레스 베이스 레지스터는 GIC 베이스 어드레스 레지스터(ReqIo 및 ReqApic를 세팅하기 위한), 확장된 구성 베이스(ReqExtCfg를 세팅하기 위한), 및 DRAM 베이스(ReqNonCache를 설정하기 위한)를 포함한다.

[0055] ARM 기반 시스템은 cHT기반 시스템과 상이하게 CPU(102) 라이트백들을 갖고 프로브/스놉 경합 조건들을 핸들링한다. cHT 기반 시스템에서, CPU(102)는 어떤 종속성 없이 프로브 응답을 항상 제공하기를 요청했다. 프로브가 CPU(102) 빅탐을 히팅하는 경우, CPU(102)는 빅탐이 이미 송신되었는지(그리고 이후에 취소될지)를 나타내고 프로브 응답으로 라이트백 데이터를 제공한다. 빅탐이 아직 세팅되지 않은 경우, 이의 상태는 프로브 유형에 따라 저하된다. ARM 시스템에서, CPU(102)는 라이트백이 완료될 때까지 프로브 응답을 블로킹하고 전달하지 않을 것이다. 이는 CPU(102)/클러스터 내에 큐잉되고 아직 문제되지 않은 프로브들에 대해서도 발생할 수 있다.

[0056] ARM 프로세서에 대해 교착 상태를 회피하기 위해, 라이트백들은 라이트백들이 DRAM으로 드레인하게 하는 독립적인 가상 채널에 액세스할 수 있다. MCT 내에서, 이들 라이트백은 이들이 통상적으로 의존적일 수도 있었을 동일한 어드레스로 보다 오래된 판독들을 전달해야 할 수 있다. 이를 구현하기 위해 요청되는 특정한 변경들은 라이트백들이 개스킷(210)을 횡단할 비-블로킹 경로, 라이트백들이 SRQ를 횡단할 비-블로킹 경로, 및 크로스-바를 통하는 비-블로킹 경로이다. MCT에 도착할 때, 라이트백은 이전 요청들에 대한 의존성을 설정할 필요가 있는 것이 아니라, 전달되어야 한다. 덧붙여, 라이트백이 완료된 후, 라이트백에 의존적인 가장 오래된 판독이 표시되고, 이미 발생된 임의의 DRAM 판독 동작을 재발행하도록 강제될 수 있다.

- [0057] 소스가 수신되었거나, 이의 프로브들이 완료되지 않았을 경우, 이전 판독이 동일한 어드레스에 도착한 라이트백에 의해 "인터럽트"될 수 있다. 판독 응답이 소스로 발행되었고 라이트백이 도착한 경우, 프로브가 응답될지 아닐지를 말하는 것이 가능하지 않을 수 있다. CPU(102)가 프로브 응답을 제공하였고 그 후 라이트백이 발생되었다는 것은 가능하다. 그러나, 이러한 경우, 소스가 데이터를 갖고 프로브 응답을 수신했을 수 있다. 그것이 발생하면, 응답은 재발행될 필요가 없을 수 있다. MCT는 이를 예상할 수 없을 수 있다. MCT가 라이트백을 재순서화하고 판독을 DRAM으로 재발행한 경우, MCT는 새로운 데이터를 갖고 응답을 재발행해야 할 수 있다. 완료된 타겟 이전/과 응답이 재송신되는 한, 새로운 데이터는 임의의 이전 응답을 오버라이트(overwrite)할 것이다.
- [0058] 개스킷(210)뿐만 아니라 메모리(104)로의 경로가 가득 찬 경우 그러한 CPU(102)가 라이트백을 발행할 수 없는 다른 가능성 있는 교착 상태 시나리오가 존재한다. 아직 발행되지 않은 라이트백(내부적으로 버퍼링됨)에 매칭한 프로브가 수신되고 CPU(102)가 프로브 기능을 정지한 경우, 라이트백들이 이들 자체의 가상 채널을 가지더라도 교착 상태가 발생할 수 있다. CPU(102)/클러스터는 라이트백들 및 비-라이트백 기록들을 인터리빙하지 않음으로써 이러한 상황을 회피한다. 따라서, 미발행된 라이트백에 대한 프로브 의존성이 존재하면, 라이트백 유형 기록들은 개스킷(210)에서 나갈 수 있고 이들 라이트백은 드레인하여, 블로킹 라이트백이 발행(및 드레인)되게 할 수 있다. 발행되었던 비라이트백 기록들이 존재하면, 클러스터는 프로브가 의존적일 수 있는 어떤 라이트백도(미발행된 라이트백도) 발생시키지 않을 수 있다.
- [0059] 개스킷(210)은 유효한 기록 데이터에 대한 기록 명령이 유니파이드 노스브리지(280) 도메인에서의 SRI에 의해 예상되는 5-클록 분리를 따른다는 것을 보장하기 위해 예측된 FIFO 이네이블들을 사용하는 것을 포함하여 동기화 장치(270)로부터 신호들을 유도 및 수신하기 위해 동기화 장치(270) 스킵 클록 로직을 관찰할 필요가 있을 수 있다. 착신 응답들이 또한 5 클록 분리를 따를 수 있다. 개스킷(210)은 ACE 타이밍으로 응답들을 CPU(102)에 포워딩할 수 있다. 동기화 장치(270)는 동기화 장치(270) FIFO가 이후 10 클록을 판독/기록하기 위해 이네이블되는지를 나타내는 신호를 제공한다. 판독하는 것 및 기록하는 것을 이네이블하기 위한 별개 신호가 존재한다. 구동 도메인에서의 로직은 신호들이 동기화 장치(270)의 출력에 수신될 때 요청된 타이밍 관계들이 관측되도록 신호들을 동기화 장치(270)로 송신할 수 있다. 기록 데이터의 경우, 이는 개스킷(210)이 적절한 관계를 갖고 유니파이드 노스브리지(280)에 의해 관측될 CPU_NbDatVal 이후 CPU_NbDat 5 이네이블된 클록을 구동해야 함을 의미한다. 유사하게, 개스킷(210)은 NB_SysDatVal 어써션 이후 NB_SysDat 정확히 5 이네이블된 클록 상에 유효한 판독 데이터를 샘플링할 예상을 해야 한다.
- [0060] ARM 기록들은 CCI가 허용하는 것보다 크기 및/또는 바이트 조합들에 있어서 유연성을 가진다. 따라서, 특정한 ACE 기록들이 다수의 CCI 기록으로 분해될 수 있다는 것이 가능하다. 요청 큐(245) 및/또는 기록 버퍼는 전송되고 적절한 명령들을 발생시킬 필요가 있는 특정한 바이트들을 추적하여, 단지 모든 "서브-기록"이 완료되고 릴리즈된 후 엔트리 및 버퍼를 할당 해제할 필요가 있을 수 있다.
- [0061] CCI/CHT는 접두 데이터 바이트로서 데이터 스트림에 따라 바이트 이네이블들을 제공한다. 나머지 데이터 바이트들에 대한 모든 바이트 이네이블 정보는 단일 접두 데이터 바이트로 제공된다. 바이트 이네이블들은 어드레스 및 카운트 필드들의 맥락에서 해석된다.
- [0062] 지지된 크기 설명은 64-바이트 정렬 블록 내 임의의 수의 연결 완전 2배 워드를 전송하는 2배워드(4-바이트) 동작들을 포함한다. 카운트 필드는 명시된 어드레스에서 시작하고, 오름 차순으로 가면서, 전송될 수 있는 2배 워드 단어 요소들의 수를 인코딩한다. 0 내지 15의 카운트 코드들은 각각, 전송될 1 내지 16 데이터 요소를 나타낸다. 64-바이트 경계에 걸친 요청들은 다수의 트랜잭션으로 분해되어, 오름 어드레스 차순으로 발행될 수 있다.
- [0063] 바이트 기록들은 자연적으로 정렬된 32-바이트 어드레스 영역 내 바이트들의 임의의 조합들을 전송할 수 있다. 정렬된 32-바이트 경계에 걸친 전송들은 다수의 하이퍼트랜스포트 트랜잭션으로 분해되어 오름 어드레스 차순으로 발행될 수 있다.
- [0064] CPU_CMD_WRBYTE: 5'b01010 - 정렬된 32 바이트들 블록 내, 가능성 있게 비-연접한 바이트들, 1 내지 32의 비-캐싱가능한 기록. 1 또는 2 데이터 바이트 기록. 액세스되는 최하위 유효 바이트가 바이트들 4 내지 7에 있으면, ReqAddr[2] = 1.
- [0065] CPU_CMD_WRDWORD: 5'b01011 - 64 바이트 블록 내 1 내지 16 연결 2배 워드의 비-캐싱가능한 기록. 1 내지 4 바이트 기록.
- [0066] ACE 프로토콜은 데이터의 각 바이트에 따른 바이트 이네이블들을 제공한다. 바이트 이네이블들은 어드레스 또는

길이 정보에 관계없이 대응하는 데이터 바이트들에 적용된다. 임의의 "크기의" ACE 기록들에 대해, 바이트 이네이블들의 임의의 조합이 허용된다. 결과적으로, 바이트 기록들(비-연접, 비-2배워드 정렬, 또는 비-2배워드 다수 크기)은, 32- 바이트 경계에 걸치고 다수의 전송을 요청할 수 있다. 임의의 ACE 기록이 최대한 2 CCI 유형 바이트 기록으로 변환될 수 있다.

[0067] CPU(102)/클러스터 내에서 조합하는 기록을 16 바이트 전송으로 귀결될 수 있다. 그 결과로 초래된다수의 CPU(102) 16 바이트 기록이 조합되더라도, 그 결과로 초래된 버퍼는 항상 단일 CCI DWORD 기록으로 변환될 수 있다. 기록이 디세이블되는 임의의 바이트 스트로브들을 갖고 수신되는 언제든, 버퍼는 단일 바이트 기록 트랜잭션 시 바로 플러싱될 수 있고, 모든 다른 버퍼는 다수의 기록을 조합하든 아니든, 단일 DWORD 기록으로 기록될 수 있다.

[0068] 본 출원에서 설명되는 프로세서 코어는 어느 정도 기록 조합을 제공하나, 프로세서는 단지 기록 데이터의 16 바이트를 조합할 수 있을 수 있다. 몇몇 애플리케이션에 대해, 가득 찬 캐시 라인까지 기록들을 조합하는 것에 상당한 혜택이 존재할 수 있다. 개스킷(210)에 구현되는 기록 버퍼들의 수는 기록 요청 엔트리들의 수보다 적을 수 있다. 기록 데이터는 단지 그것이 SRI에 의해 버퍼로부터 플러싱될 때까지 유지될 수 있는 한편, 기록 요청은 응답이 CPU(102)로 송신될 수 있을 때까지 유지할 필요가 있다(이는 SRI 버퍼 릴리즈 후 완료된 타겟에 기반하지 않는다). 몇몇, 또는 이들 중 모든, 기록 버퍼는 기록 조합 능력을 가질 수 있다.

[0069] 기록 조합 능력을 추가하는 것을 버퍼를 플러싱하는 것을 필요로 한다. 몇몇 가능한 알고리즘이 존재하고 구성 가능한 모드들로서, 또는 병렬적으로, 이들 중 다수를 구현하는 것이 필요할 수 있다. 가능한 버퍼 플러시 트리거들은 다음을 포함한다: 버퍼가 완전히 유효한 임의의 시간 - 필수적 플러시; 미송신된 버퍼들이 고갈되는 임의의 시간 - 필수적 플러시; CPU(102) 기록이 해당 CPU(102)에 대해 이미 개방된 버퍼를 히트하지 않는 임의의 시간; 개방된 버퍼들의 수에 대해 프로그래밍된 제한에 도달되는 임의의 시간; 및 버퍼가 더 이상 기록되고 있지 않다는 것을 나타내는 다른 휴리스틱들. 버퍼 플러시에 대한 한계들은 단지 기록들을 정확한 캐시 능력 속성들(버퍼링가능)과 조합할 수 있을 때, 배리어들에 대한 모든 버퍼를 플러싱할 필요를 포함할 수 있고 몇몇 실시예에서, DvmSync에 대한 모든 버퍼를 플러싱할 필요가 있을 수 있다.

[0070] 배리어 트랜잭션들은 ARM 프로그래밍/일관성 모델이 그 외 제공하지 않는 특정한 순서화 보장들을 제공하기 위해 사용될 수 있다. ACE 프로토콜은 별개 관독 및 기록 트랜잭션 채널들에 대해 제공하고, 따라서, 배리어 트랜잭션들은 관독/기록 쌍으로 발행될 수 있다. 양 트랜잭션이 완료되면, 배리어 이후 발행되는 트랜잭션들이 배리어 이전 트랜잭션들 이후 순서화될 수 있다고 추정하는 것이 안전하다. ACE 프로토콜은 두 유형의 배리어를 정의한다: 메모리 및 동기화.

[0071] 메모리 배리어는 적절한 도메인에서의 다른 마스터가 배리어 이후 발행된 임의의 트랜잭션을 관측할 수 있는 경우 그것은 배리어 이전 발행된 모든 트랜잭션을 관측할 수 있어야 한다는 것을 보장하기 위해 마스터에 의해 발행될 수 있다. 메모리 배리어는 메모리 기반 통신을 위해 사용될 수 있다. 예를 들어, 데이터의 어레이를 메모리(104)에 기록할 때, 마스터 구성요소는 어레이가 이용가능하다는 것을 나타내도록 메모리 플래그를 설정하기 전 메모리 배리어를 발행할 수 있다. 플래그를 관찰할 수 있는 임의의 다른 마스터 구성요소는 어레이에 기록하는 트랜잭션들을 관찰해야 한다.

[0072] 동기화 배리어는 배리어가 완료될 때 배리어가 적절한 도메인에서 모든 마스터에 의해 관찰가능하기 전 트랜잭션들이 발행했다는 것을 보장하기 위해 마스터에 의해 발행될 수 있다. 시스템 도메인 동기화 배리어들은 배리어가 완료되기 전 이들이 향하게 될 종단점 슬레이브들에 도달했어야 하기 전 트랜잭션들이 발행했다는 추가적인 요건을 가진다. 동기화 배리어는 다양한 형태의 측-파대 시그널링 통신과 사용될 수 있다. 예를 들어, 데이터의 어레이를 메모리(104)에 기록할 때, 마스터 구성요소는 어레이가 이용가능하다는 것을 나타내도록 인터럽트를 발생시키기 전 동기화 배리어를 사용할 수 있다. 동기화 배리어가 완료될 때, 업데이트 어레이는 도메인에서 모든 마스터에 의해 관찰가능한 것으로 보장된다.

[0073] cHT 프로토콜은 이후 기록의 관찰이 이전 기록들의 관찰을 암시하도록 기록 순서에 대해 제공한다. 메모리 및 동기화 배리어 명령들의 핸들링 간 임의의 차이가 있을 필요가 있다. 양자는 모든 이전 요청이 CPU(102)로 완료를 나타내기 전 개스킷(210)에 의해 SRI로 발행되었다는 것을 간단히 보장함으로써 개스킷(210) 내에서 전적으로 핸들링될 수 있다. 보다 오래된 요청들이 발행(또는, 보다 보수적으로, 완료)되었을 때, 개스킷(210)은 CPU(102)로 다시 배리어 트랜잭션에 대한 응답을 송신할 수 있다.

[0074] ACE는 관독 및 기록 요청 채널들에 대한 액세스 허가들을 제공하기 위해 ARPROT[2:0] 및 AWPROT[2:0]를 제공한

다. 이들 비트는 다음과 같이 정의된다:

[0075] AxPROT[0] : 0/1 - 비특권/특권 액세스;

[0076] AxPROT[1] : 0/1 - 안전/비-안전 액세스; 및

[0077] AxPROT[2] : 0/1 - 데이터/명령 액세스.

[0078] 단지 비특권/특권 비트가 SRI를 통해 전달된다. 이것은 SRI가 프로그래밍 인터페이스를 GIC 분배기에 제공하도록 계획되고 몇몇 레지스터가 가상화를 지지할 때 특권 레벨에 기초하여 बैं킹되기 때문에 필요하다. 어드레스 유니파이드 노스브리지(280) 지지 어드레스 공간이 코어의 어드레스 공간보다 크기 때문에, 상측 어드레스 비트들은 포워딩될 필요가 있을 수 있는 임의의 추가적인 액세스 허가를 위해 동일하게 행하기 위해 그리고 특권 표시자로 오버로딩될 수 있다.

[0079] 개스킷(210)은 전력 관리 동작에의 어느 정도 참여를 가질 것이다. 개스킷(210)은 다음에 의해 상호작용할 수 있다. 클러스터에 의해 제공되는 다양한 이벤트를 카운팅하는 요청된 CAC 레지스터들을 구현하고; 전력 관리 상태를 변경하기 위해 "작업 거부(Quiesce)" 기능을 제공함으로써 개스킷(210)은 입력 신호를 통해 전력 관리를 요청함으로써 추가 CCI 요청들을 블로킹하기 위해 상호작용하여, 완료할 미처리 CCI 요청들을 기다리고, 그 후 인터페이스가 유휴 상태라는 것을 나타내기 위한 출력 신호를 구동할 수 있다.

[0080] 전력 관리 목적들을 위해, 사이클 카운트는 주파수에 기초하여 가중될 수 있다. 클러스터는 주파수 정보에의 액세스를 가지지 않는다. 주파수는 원하는 P-상태로 레지스터를 프로그래밍하는 소프트웨어의 결과로서 유니파이드 노스브리지(280)에 의해 셋팅된다. 전력 관리 소프트웨어(PEP)는 주파수 변경이 발생할 때마다 개스킷(210)에서의 가중치들을 업데이트하기 위해 SMU를 시그널링할 수 있다. 대안적으로, P-상태 주파수 테이블은 개스킷(210)에 복제될 수 있고 현재 P-상태의 업데이트 기록이 개스킷(210)에 의해 관찰될 수 있으며 가중치들은 투명하게 업데이트했다.

[0081] 상기 방법들에 의해 제한되지 않기 위해, 몇몇 다수의 카운터는 관심 있는 것으로 여겨지는 PMU 이벤트 신호들 중 임의의 신호를 카운팅하도록 프로그래밍되도록 제공될 수 있다. 각 카운터는 PMU 이벤트들, 버스 판독/기록 표시들, 또는 클록 이벤트들 중 어떤 것이 카운팅될 지를 나타내도록 프로그래밍될 수 있다. 예를 들어, 사이클이 이벤트/인터럽트 대기(WFE/WFI)를 카운팅하는 구성은 활동 사이클들을 카운팅하도록 시그널링한다. POR 및 SMU는 이들 카운터의 각각에 대한 가중치들을 제공할 수 있다.

[0082] 전력 관리는 소프트웨어 및 인터럽트 컨트롤러와 인터페이싱한다. 일 구현예에서, 소프트웨어 구성요소는 마이크로코드이고 인터럽트 컨트롤러는 APIC이다. ARM 아키텍처에서, 마이크로코드를 구현하기 위한 어떤 단서도 없고, 인터럽트 컨트롤러는 GIC이다.

[0083] 전력 관리는 특정한 구현이다. SOC에서의 특정한 하드웨어는 OS에 보이지 않는다. OS는 각 구현이 벤더 특정 소프트웨어의 뒤에 가려지는 구체적으로 정의된 인터페이스들을 통해 전력 관리 하드웨어에 액세스한다. C-상태들 및 P-상태들은 ACPI 호들을 통해 OS로 공고되고, OS는 마이크로코드에 의해 트랩되고 핸들링되는 요청들을 행한다. ARM에서, 이러한 계층의 코드가 윈도우즈 및 안드로이드 양자에 공급될 수 있다. 윈도우즈 RT 상에서, 이러한 코드는 공지되고 플랫폼 엔진 플러그인(PEP; Platform Engine Plugin)이며 안드로이드에서, 전력 관리 드라이버 코드는 커널로 직접 컴파일될 수 있다. 이들은 C-상태들에 대해 CPUidle 그리고 P-상태 변경들에 대해 CPUfreq의 형태가 된다. 전력 관리 하드웨어에 대한 인터페이스는 유코드(ucode) 인터페이스와 유사하게, PEP 또는 커널 드라이버에서 이행될 수 있다. 간단함을 위해, ARM 구현을 설명하면서 안드로이드 커널 드라이버들에 대한 언급이 제공될 것이다. OS 소프트웨어 스택으로 상이하게 조직화되고 명명되더라도, PEP는 마이크로아키텍처를 OS로 추출하는 안드로이드 커널 드라이버 또는 마이크로코드와 같은 역할을 이행한다. 안드로이드 전력 관리 커널 드라이버 아키텍처를 취하고 이를 본 출원에서의 설명에 기초하여 마이크로소프트 플랫폼에 적용할 수 있다.

[0084] 전력 관리 하드웨어에 대한 마이크로코드(유코드) 또는 커널 드라이버 인터페이스와 달리, APIC 또는 GIC는 아키텍처상 OS에 보인다. SOC 하드웨어는 아키텍처에 대응하는 인터럽트 컨트롤러를 구현할 수 있다. 따라서, 공통 전력 관리 하드웨어는 양 인터페이스를 수용할 수 있다.

[0085] GIC v2 사양이 구현될 수 있다. GIC v3와 달리, 레거시 GIC-400은 8개까지의 코어를 지지하기 위해 하드웨어에 내장된 하나의 구성요소에 양 분배 및 전달(CPU(102) 인터페이스)를 구현한다. CPU(102) 인터페이스의 출력은 코어-당 FIQ/IRQ 핀들 및 가상화된 대응물들이다. 이러한 아키텍처는 APIC와 유사하고, 예지들은 C-상태들을 중

료하기 위해 기상 요청을 발생시키는 것과 함께, 양 소프트웨어(CC_INT_MASK) 및 하드웨어를 사용하여 마스킹될 수 있다.

- [0086] 전력 흐름 흐름들은 소프트웨어 지지를 필요로 한다. 인터럽트들은 유코드에 의해 먼저 인터셉트되며, 이는 내부적으로 이들 유아크(uarch) 인터럽트를 제공하고 그 후 OS에 아키텍처상 보이는 그것들을 포워딩한다. 유아크 인터럽트들은 중단 클록(StpClk) 및 시작 클록(StartClk)을 포함한다. ARM 코어들이 유코드를 구현하지 않기 때문에 GIC 인터럽트들이 OS에 보인다. ARM 상 StpClk 및 StartClk을 지지하기 위해, 벤더 특정 인터럽트 핸들러들은 유니파이드 노스브리지(280)에 등록될 수 있으며, 이는 OS에 의해 디바이스로서 취급될 수 있다.
- [0087] APIC 인터럽트 전달은 원격 기록들(외부 인터럽트들에 대한)에 의해 또는 로컬 인터럽트 핀들을 통해 수행될 수 있다. 로컬 인터럽트의 예는 APIC330이며, 이는 과열 이벤트들에 기인하여 이의 요청된 수행 상태가 조절 또는 복원되었다는 것을 OS에 나타내기 위해 사용된다. GIC 인터럽트 전달은 APIC 원격 기록들과 유사한 기능을 하는, 데이터 패브릭으로 이의 AXI 인터페이스를 통하는 것, 및 공유된 주변 인터럽트(SPI; shared peripheral interrupt) 또는 개인 주변 인터럽트(PPI; private peripheral interrupt)의 형태인 로컬 인터럽트 핀들을 통하는 것 양자이다. 각 SPI는 단일 코어를 타겟팅하도록 프로그래밍될 수 있다. SPI들을 사용함으로써, 유니파이드 노스브리지(280)는 인터럽트를 요청하기 위해 GIC로 그리고 패브릭으로 AXI 메시지를 발생시키지 않아야 한다. APIC330의 경우, 단일 SPI id가 취해질 수 있다. StpClk, StartClk, DbReq, 및 DbReq2와 같은, 다수의 코어를 타겟팅하는 인터럽트들에 대해, 유니파이드 노스브리지(280)는 인터럽트 유형마다 코어당 하나의 SPI id를 예비 할당하고 코어(들)를 타겟팅하도록 SW 구성에 요청할 수 있다. 단지 네 개의 코어가 존재하는 경우, StpClk, StartClk, 및 DbReq들을 구현함으로써, 작동하는 16 SPI가 사용될 수 있다.
- [0088] 대안에는 GIC-400에 대해 PPI들로서 이용가능해지는, nLEGACYIRQ[n]/nLEGACYFIQ[n] 핀들로 유니파이드 노스브리지(280) 다중-타겟 인터럽트 유형들을 후킹(hooking)하는 것이다. 상태 레지스터에는 인터럽트 핸들러가 동일한 핀으로 인코딩되는 서브-유형을 결정하도록 공급될 수 있다. 이는 레거시 핀들을 공유하는 인터럽트들이 동일한 우선순위를 공유해야 할 수 있다는 것을 필요로 할 수 있으나, StpClk, StartClk, 및 DbReq들은 그래도 가장 높은 우선순위로 셋팅될 수 있다.
- [0089] 현재, APIC는 인터럽트 에지를 nbcor 440, 예를 들어 NBAPC_Intr로 유도하며, 그 후 레벨 인터럽트 핀을 코어, 예를 들어 NBCOR_IntrEdge로 셋팅한다. 그 후 유코드는 제공할 때 SprClrInt(F3xB4)를 통해 그것을 클리어한다. GIC v2는 레벨 핀들, 예를 들어 IRQ/FIQ를, nbcor 440로 유도할 수 있으나, 이들은 코어가 인터럽트를 확인 응답한 후 GIC 410 그 자체가 대신 클리어될 수 있다. 이는 아키텍처상 정의된 GIC(410) 핀들에 대해 SprClrInt를 구현할 필요를 최소화한다. nbcor(440)에서의 IRQ/FIQ 핀들은 기상을 트리거하기 위한, IntrEdge로서 플롭(flop) 및 사용될 수 있다.
- [0090] 기존 유아크 인터럽트들은 이제 GIC(410)로 유도되고 FIQ 상에 드러난다. StpClkEdge, StartClkEdge, 및 DbReqEdge를 포함하여, 이들 인터럽트의 대부분에 대해 nbcor(440)에서 레벨 인터럽트 핀들을 또한 구현할 필요가 더 이상 없다. 기존 핀들은 레벨로부터 에지로 변환되고 nb->코어로부터 nb->GIC로 재 타겟팅될 수 있다. 인터럽트 핸들러들이 두 별개 레벨 인터럽트 상태를 클리어할 어떤 이유도 없다. 상기 세 개 외 사용되는 유일한 레벨 표시는 StartClkEdge이며, 이는 정규 및 동시 이력 현상 카운터들 간 스와프(swap)를 위해 사용된다. 사용은 StartClk이 취해질 때까지 스와프를 단지 지연시킬 것이고, 일어나는 시나리오는 C1e이며, 이는 우리가 안전하게 무시할 수 있어야 하는 것이다. 유일한 예외는 취소를 발생시키기 위해 이의 레벨 표시를 사용하는, DbReq이고, 따라서 우리는 DbReq2가 취해진다는 것을 나타내기 위해 SprClrInt를 통해 시그널링하기 위해 SW를 필요로 하고 그에 따라 그 지점 이후 취소를 회피한다. 이는 DbReq2에 대해 FIQ 에지를 클리어하는 것에 추가적이다.
- [0091] 현재, 유니파이드 노스브리지(280) C-상태는 InbWake 상에서 종료되며, 이는 외부 인터럽트들을 커버한다. 또한 코어를 기상시킬 필요가 있는 지역적으로 소성된 인터럽트들에 대해, 이들이 APIC IntrEdge 및 DbReq로부터 매핑할 수 있는 유형들이 키잉 오프(keying off)될 수 있다. GIC에 대해 로컬 인터럽트들이 설명될 수 있다. GIC에 대한 로컬 SPI 입력들이 자주 클록(free running clock) 상에 있으면, 로컬 입력들이 그것을 FIQ/IRQ, 또는 에러 또는 가상화 대응물들 상에서 파악할 것이 추정될 수 있고, 따라서 우리는 간단히 유휴 상태 종료로 재 매핑할 수 있다. 입력들이 GIC에 이르기 전 게이팅된 로직을 통해 이동하면, 이들 인터럽트의 소스는 유휴 상태 종료로 갈 수 있다.
- [0092] x86/ACPI 상에서, 유휴 상태인 프로세서를 히팅 시, OS는 정지 명령을 통해 정지를 또는 ACPI에 의해 공고되는 C-상태 오프셋을 셋팅한 후 IO 관독을 요청한다. 정지 또는 IO 관독이 마이크로코드로 트랩되며, 이는 그 후 유

니파이드 노스브리지(280)로 정지 특정 브로드캐스트 사이클(SBC; special broadcast cycle)을 발행하고, 패브릭이 작업 거부하기를 기다리며, 그 후 ConnectMask 사이드밴드를 어썬트 해제한다. C-상태에 진입하는 마지막 코어로부터의 SBC는 모든 코어 상에 C-상태의 시작을 시그널링하기 위해 FCH에 대한 다운스트림 시스템 관리 명령을 야기한다. 이는 또한 보다 깊은 C-상태들에 대한 업그레이드들을 통제하는 모니터들을 구동하기 위해 사용할 수 있다. ConnectMask 사이드밴드는 전력 관리를 개시하기 위해 사용된다.

- [0093] ARM/안드로이드 상에서, OS는 이의 미리 정의된 API를 통해 공급된 CPUidle 커널 드라이버 코드를 호출한다. 그 후 CPUidle은 정지 SBC를 이의 메모리(104) 매핑된 어드레스를 통해 NB로 발행할 수 있다. x86 상에서, 이는 FDF91 베이스로 매핑된다. 이는 유니파이드 노스브리지(280)에서 하드코딩되고, 따라서 ARM 코어는 해당 어드레스로 메모리(104) 요청들을 발생시킬 수 있어야 할 필요가 있을 수 있다. 이는 FCH 통신뿐만 아니라 모니터 트레이닝을 충족시킨다. 그 후 CPUidle은 WFI 명령을 발행하며, 이는 STANDBYWFI 사이드밴드를 어썬트한다. STANDBYWFI는 반전된 후 유니파이드 노스브리지(280) 내부에서 내부적으로 ConnectMask로서 취급된다.
- [0094] 특정한 코어들 상에서, ConnectMask를 보는 것은 정지 중에 어썬트 해제될 수 있거나 HTC SBC(P-상태들에 대한)는 CK에 대한 임박한 클로킹 요청을 위해 데이터 패브릭을 준비하기 위해 하드웨어를 개시할 수 있다. 실제 클로킹 요청 발생으로 진행하기 전 다음이 완료될 수 있다:
- [0095] 1. 모든 코어로의 프로브 발행을 블로킹하고 이들의 프로브 응답들 및/또는 데이터 이동들이 완료하기를 기다림으로써 영향을 받은 코어(들)로부터 모든 계류 중인 프로브를 드레인한다. 이는 CPC가 모두 플러싱되는 경우 적용하지 않는다.
- [0096] 2. 영향을 받은 코어(들)가 이들의 NbBusy를 드롭하기를 기다리며, 이들은 예를 들어 스크리빙 동안, 저 전력 상태에 대한 코어들의 이의들이다.
- [0097] 3. FIFO들로 인플라이트 트래픽에 의해 트리거되는 새로운 이벤트들이 이의를 제기하기에 충분한 시간을 가질 수 있도록 NBSYN FIFO들에 대한 왕복 지연을 커버하기에 충분한 시간을 무조건적으로 대기한다.
- [0098] 4. 블로킹하고 Cac 인터페이스 상의 계류 중인 APM 요청들이 응답 또는 확인 응답되기를 기다린다.
- [0099] 5. 전력 감소를 위한 이력 현상을 충족시킨다. 이력 현상은 프로브 도착 레이트(정규) 및 요청-간 프로브 갭(등시) 양자에 대해 구현된다. 이는 CPC가 모두 플러싱되는 경우 적용하지 않는다.
- [0100] 전력 관리는 또한 ConnectMask 어썬션 해제가 있을 때 디세이بل될 수 있다. 클록 분배기를 EnaCPUDiv1LowPwr?=0을 이용하여 div-1로 셋팅하는 것은 저 전력 엔트리를 스킵하고 CK에 대한 클로킹 요청을 야기하지 않을 수 있다.
- [0101] 유니파이드 노스브리지(280)는 CK에 대한 다음 클로킹 요청 유형들을 발생시킬 수 있다: 이네이블된 프로브로 클록 게이팅되는 코어 (프로브-가능 CC1로서 지칭됨); 디세이بل된 프로브로 클록 게이팅되는 코어(비-프로브-가능 CC1로서 지칭됨); 플러싱된 캐시로 클록 게이팅되는 코어(캐시-플러시-온-정지 CFOH로서 지칭됨); 및 유지 모드(RCC3).
- [0102] 이네이블된 프로브로 클록 게이팅되는 코어(프로브-가능 CC1로서 지칭됨)를 위해, CK은 타겟팅된 코어에 대한 모든 클록 게이트를 어썬트 해제하고, 그 후 프로브들과 연관된 게이트들을 재-이네이블할 수 있다. L2는 프로브들이 있을 때 프로브 게이트 재이네이블을 무시하고, 코어를 제외하며, 이 자체의 클록 램프 업/다운을 수행한다는 것을 주의하자. 다른 코어는 WFI를 송신 시 그 자체를 클록 게이팅할 수 있고 CK에 의해 추가 클록 게이팅할 필요는 없다. CC1은 EnaCPUDiv1LowPwr?=0을 이용하여 CCR을 div-1로 셋팅함으로써 간단히 디세이بل된 전력 관리로서 코어 상에서 구현될 수 있다.
- [0103] 디세이بل된 프로브로 클록 게이팅되는 코어(비-프로브-가능 CC1로서 지칭됨)를 위해, 이는 프로브 게이트들이 재이네이블되지 않는다는 것을 제외하고는, 프로브-가능 경우와 동일하다. 이러한 모드는 몇몇 코어에서 사용되지 않고 단지 치킨 비트(chicken bit)로서 존재한다. 프로브들이 있을 때, 전력 관리는 이력 현상을 보장하는 준비 로직으로, C-상태 종료 및 재 엔트리를 요청하도록 요구된다.
- [0104] 플러싱된 캐시와 클록 게이팅되는 코어(캐시-플러시-온-정지 CFOH로서 지칭됨)를 위해, 이러한 모드는 착신 프로브들을 제공하기 위해 디세이بل된 프로브들로 그리고 램프 업 없이 클록 게이팅된 상태에의 상주를 가능하게 한다. 이러한 모드는 단지 전체 CPC가 플러싱되었을 때 유익하다.
- [0105] 유지 모드(RCC3)에 대해, 이는 코어 전압이 전력 게이팅(PG) 헤더들을 조절하고 피드백 회로를 통해 해당 전압

을 유지함으로써 감소되는 비-프로브-가능 CC1 상태의 보다 깊은 버전이다. 레이턴시 특성들은 보다 많은 전력 저감을 갖는다는 것을 제외하고는 비-프로브-가능 CC1과 매우 유사하다. RCC3는 플러싱으로 또는 플러싱 없이 적용될 수 있으며, 여기서 플러싱은 그것을 보다 양호한 CFOH 모드로 만든다.

[0106] 몇몇 코어는 유지 모드를 가능하게 하기 위해 이의 전력 컨트롤러와의 Q-채널 핸드셰이크를 요청하여, 이러한 모드를 구현할 수 있다. 이러한 핸드셰이크를 수행하지 않고, 클러스터는 패브릭 개입 없이 인터럽트 시 WFI에서 종료하기 위해 그리고 프로브들을 제공하기 위해 코어를 자동적으로 클록 게이팅하고 클록들을 자동적으로 램핑한다. 이러한 핸드셰이크를 수행한 후, 클러스터는 어느 하나의 작업을 완료하기 위해 저 전력 상태에서 종료하게 하기 위해 핸드셰이크를 완료하는 것에 의존할 수 있다. 유지 모드는 핸드셰이크에 의해 가능하게 될 때 패브릭에서 전력 컨트롤러의 제어 하에서 진입/종료된다. 따라서, WFI 시, 클러스터는 먼저 STANDBYWFI를 어썬트하고 이의 클록을 게이팅 오프할 수 있다. 일정 시간 이후, 그것은 유지 핸드셰이크가 허용된다는 것을 나타내기 위해 CPUACTIVE를 디썬트 해제할 것이다. 전력 컨트롤러는 유지를 요청하기 위해 CPUREQ를 어썬트할 수 있다. 그 후 클러스터는 CPUACCEPT 또는 CPUQDENY를 어썬트함으로써 3-방향 핸드셰이크의 썬드 파트를 통해 유지를 승인 또는 거부할 수 있다. 그 전에, 인터럽트는 코어를 자동적으로 재개하고 핸드셰이크에 있는 경우, CPUQDENY를 야기한다. CPUACCEPT를 본 후, 유지 엔트리는 커밋된 것으로 여겨지고 클러스터는 유지 종료를 위해 전력 컨트롤러에 의존적이고 그 후 클록 게이팅 또는 WFI 종료 전 CPUACCEPT를 어썬트 해제할 수 있다.

[0107] 코어 C-상태 커밋 지점은 유니파이드 노스브리지(280)에서 SetCPULowPwr이다. 이러한 피처를 지지하기 위해, 핸드셰이크가 완료될 필요가 있을 수 있다. 핸드셰이크 개시를 트리거하기 위한 최적의 위치는 CPULowPwrReq 어썬션 상이며, 이는 STANDBYWFI 어썬션을 확인하고, 그 후 CPUACCEPT 어썬션까지 VceLowPwrRdy를 블로킹하며, 이는 코어가 커밋했음을 보장한다.

[0108] GIC-400은 CPU(102) 인터페이스로부터 FIQ/IRQ 핀들 상에 직접 착신 인터럽트들을 노출시킨다. 이들 인터럽트는 PwrMgtOrStpGntCC6MaskIntCPU에 의해 코어로부터 마스킹될 수 있으며, 이는 RCC3에서(또는 CC6에서) 코어의 상주를 커버한다. FIQ/IRQ 에지들은 IobcOrHalt를 클리어하기 위해 기존 로직을 사용하여 기상 이벤트(CoreWakeUpInt)를 생성하기 위해 사용될 수 있으며, 이는 그 후 RCC3로부터의 종료를 트리거할 수 있다. 종료 완료되면, PwrMgtOrStpGntCC6MaskIntCPU는 어썬트 해제될 수 있고 모든 인터럽트가 전달될 수 있다. 인터럽트 전달은 어썬트 해제하기 위해 CPUACTIVE를 트리거할 수 있고, 유니파이드 노스브리지(280)는 그 후 CPUREQ를 어썬트 해제함으로써 유지 핸드셰이크를 종료할 수 있다.

[0109] CPUACTIVE는 유지 시 코어로 타게팅되는 프로브들에 대해 어썬트 해제될 수 있으나, RCC3는 캐시 플러시 이후 이네이블될 수 있고, 따라서 프로브들은 기상 상태가 아니다. L2는 l2_snp_active가 ck_CPU_logic을 기상시키는 L1에 대한 스냅 필터를 구현한다. 이는 L1 플러싱된 코어가 L2로부터의 프로브들을 보지 않을 수 있다는 것을 의미하며, 이는 착신 시스템 프로브들에 대한 유지 종료가 없음을 보장해야 한다.

[0110] 코어는 분산적 가상 메모리(104)(DVM) 동작들을 수신 시 CPUACTIVE를 추가적으로 어썬트 해제한다. DVM 동작들은 TLB 썬-다운, BTB 실패, 및 i-캐시 실패들을 포함하고 코어가 일관성을 잃고 취해지지 않는 한, 프로토콜에 의, 즉 CPUECTLR SMP 비트를 클리어함으로써 대칭(SMP)을 비대칭 멀티프로세싱(AMP) 모드로 스위칭함으로써 이의 전력 감소 시퀀스에의 코어의 참여를 요청한다. CC6 흐름에서, 전력 감소는 DVM 타겟들에 대한 실패로서의 역할을 한다. 그러나, 유지를 위해, 종료시 리셋이 플러징되지 않을 수 있고 유지로의 엔트리에 앞서, TLB 및 BTB를 포함하여, 구조들을 무효화하기 위해 또는 DVM 동작들을 제공하기 위해 램프 업이 발생할 수 있다. i-캐시는 캐시 플러시 시퀀스에서 무효화되었을 수 있다. DVM 동작들은 프로브들과 달리, 드문 것으로 추정될 수 있고, 비-인터럽트 소스로부터 유지 종료를 지지하기 위해, DVM 트리거된 종료는 코어들이 비-프로브-가능 C1 상태에 있는 동안 프로브들에 대해 취해지는 종료 경로와 유사할 수 있다. 아래 코드는 프로브 경로를 핸들링할 수 있다.

ClrCPULowPwrPre_m1[i] = (~CPULowPwrReq[i] | SrqCPUPrbReq[i] |
EarlyIntrReq[i]) & CPULowPwrActive[i] & ~WaitClrCPULowPwr[i] &
~ClrCPULowPwrFound; and

SrqCPUPrbReq[i] = ((AnySrqPrbVal_d1 | SrqAnyCPUPrbRsp) &
~SrqMaskPrbWakeUpCPU[i]) | CkExitToPstate_d1.

[0111]

[0112] DVM 검출 시 저 전력에서의 종료가 발생할 수 있고, 그 후 종료가 완료되면, IobcOrHalt를 수정하지 않고, 재 엔트리가 발생할 수 있다. i-캐시가 플러싱되더라도, SMP 모드에서의 코어는 i-캐시 실효 DVM들에 응답할 필요가 여전히 있을 수 있고 DVM 동작 시 CPUACTIVE 어썬트 해제를 볼 수 있다.

[0113] 코어-측 상의 Q-채널 로직은 코어가 유지에 있는 동안 작동 중일 수 있다. 이는 작동 전압이 가동 중이고, 단지 보다 낮은 것이 보장된 보다 낮은 di/dt를 갖는다는 것을 의미한다. 이는 NB AltVid와 같은 제약이다. 백업을 위해 그리고 가능한 더 낮은 유지 전압을 위해, 모드가 소프트웨어로 TLB 및 BTB를 무효화하도록 지지되고 그 후 SMP 비트를 턴 오프할 수 있다. 이러한 모드에서, Q-채널 핸드셰이크의 언와인드(unwind)는 인터럽트들을 종료의 유일한 소스로서 추정함으로써 사소하게 만들어지거나, Q-채널 핸드셰이크를 완전히 디세이블할 수 있다. 후자는 아키텍처상 보다 간단하나 예기치 않은 방식으로 코어에 인터페이스상의 위험을 캐리한다.

[0114] CK에 대한 유니파이드 노스브리지(280)로부터의 RCC3 시그널링에서, XV 컴퓨팅 유닛에서의 코어들은 일체화 RCC3에 진입하고, 예를 들어 코어-당 C-상태가 아니다, 그에 따라 nbsyn 전력 오프를 RCC3 엔트리와 항상 연결한다. 이는 RCC3(프로브-가능 C1에 비해) 및 nbsyn 전력 오프 양자를 시그널링하기 위해 NB_PwrMgtReqPrbEn==0을 가능하게 한다. RCC3의 일 구현 동안, 이들은 분리될 필요가 있을 수 있다. RCC3는 각 코어에 대해서 nbsyn 전력 오프는 각 CPC 또는 컨트롤러에 대한다. nbsyn 전력 오프는 별개 NB->CK 인터페이스 신호로 분리되고, 그것을 NB_PwrMgtReqCciEn으로서 지칭할 수 있으며, 이는 RCC3 또는 CC6에 진입하는 마지막 코어에 대해 단지 클리어된다.

[0115] CC6에 진입하기 위해, 유니파이드 노스브리지(280)는 CK에 대한 클로킹 요청 시 전력 토글을 요청할 수 있다. CC6는 단지 캐시 및 코어 상태들이 플러싱될 때 허용된다. CC6은 RCC3에 비해 추가적인 전력 저감을 제공하나, 종료 시 상태 복원을 필요로 하며, 그에 따라 종료 레이턴시를 증가시킨다. L2로부터 복원할 때 CC6 종료 레이턴시는 ~20us 내지 ~40us이고 L2를 포함하는 전체 CPC가 CC6(XC6라고도 공지됨)에 있는 경우, DRAM으로부터일 때는 ~80us이다. RCC3 종료는 ~1us이다. CC6 종료는 CCI 또는 RCC3와 상이하지 않게 트리거되며, 이는 IobcOrHalt를 클리어하기 위해 계류 중인 APIC 인터럽트 예지들을 사용하기 위함이며, 그에 따라 종료 상태를 야기한다.

[0116] GIC-400은 ProcessorSleep 비트를 구현하지 않는다. 하드웨어 구현들은 GIC-400에서의 분배기와 같은 장소에 위치되는 CPU(102) 인터페이스로부터의 출력 상에서 작동할 것으로 예상된다. GIC-400은 세 개의 모드로 이의 CPU(102) 인터페이스에서의 FIQ/IRQ 거동을 프로그래밍할 수 있다. 이들은 (1) GIC로부터의 nIRQCPU[n]/nFIQCPU[n] 핀들이 모든 상황에서 구동되는 정규 모드, (2) 레거시 FIQ/IRQ 핀들로 GIC 출력을 오버라이드하는 우회 모드, 및 (3) 코어들에 대해 IRQ/FIQ 핀들을 마스킹하고 저 전력 종료를 위해 유형들의 인터럽트들이 있을 때 전력 컨트롤러로 nIRQOUT[n]/nFIQOUT[n] 핀들을 대신 유도하는 우회 디세이블 모드를 포함한다.

[0117] 예로서, 정규 모드가 구현될 수 있고 기존 유니파이드 노스브리지 마스킹 및 인터럽트 기상 메커니즘들이 사용될 수 있다. GIC는 그것이 모든 상황에서 작동하고 전력 관리가 예지들이 코어들에 의해 보여지지 않을 경우들에 전달을 보류할 수 있다는 것을 추정한다. 이는 C6 주위에, RC3 및 C6를 위해 사용되나 CC1을 위해 디세이블되는, 기존 C-상태 인터럽트 마스킹이 하드웨어 마스킹을 형성할 수 있다는 것을 의미한다. C6에서, 마스킹을 클리어하는 것(C6ExitReset)은 소프트웨어에 의해 수행될 수 있다. StpClk 마스킹은 모두 C6 주위에서 수행될 수 있다. C6 마스킹이 GIC-400 흐름에서 소프트웨어 대신 하드웨어에 의해 셋팅되기 때문에, 어떤 별개 유니파이드 노스브리지 마스킹도 StpClk 이벤트들 주위 예지들을 필터링하도록 요구되지 않는다. 유니파이드 노스브리지 하드웨어는 IobcOrHalt 상태 및 STANDBYWFI 어썬선에 기초하여 StpClk 마스킹을 내부적으로 결정할 수 있다.

StpClk 마스킹은 이것이 StpClk을 취하기 위해 코어들을 유지에서 종료하게 할 수 있음에 따라, RC3 코어들에 대해 셋팅되지 않는다.

- [0118] C6 종료 상에서, 타겟팅된 코어는 리셋하고 예외 벡터로부터 코드를 페칭하는 것을 시작할 수 있으며, 이는 리셋에서 나오는 핀-스트랩을 통해, 0 또는 FFFF0000의 고정 어드레스, 또는 별개 어드레스에 위치된다. 이러한 어드레스는 전형적으로 부팅 ROM에 매핑된다. 예외 벡터는 건너뛰 테이블을 구현하며, 이 내부에 리셋 핸들러가 엔트리이다. 그 후 리셋 엔트리를 건너뛰는 것은 리셋 핸들러를 야기한다. 이러한 코드는 리셋이 C6 종료를 향한 폴링이었는지 대 워/콜드 리셋을 향한 폴링이었는지를 결정하기 위해 C6ExitReset SPR을 검사할 수 있다. 그 후 C6 종료는 C6 핸들러로 건너뛸 수 있고 워/콜드 리셋은 부팅 코드로 건너뛸 수 있다. 유니파이드 노스브리지(280)가 예외 벡터가 위치되는 어드레스를 명시하기 위한 선택을 가진다는 것을 고려할 때, 유니파이드 노스브리지(280)는 리셋 핸들러가 유형을 검사할 필요가 없도록 C6 종료 대 워/리셋을 효과적으로 고유화하기 위해 부팅 이후 벡터를 재위치시키도록 선택할 수 있다. 이는 C6 핸들러가 상이한 메모리(104) 영역에 배치되게 할 수 있고, 따라서 그것은 C6 종료 상에서 ROM으로부터의 초기 패치를 회피하기 위해 부팅 ROM 대신 보다 빠른 DRAM에 위치될 수 있다. 이를 가능하게 하기 위해, 개스킷(210)은 예외 벡터 위치의 소프트웨어 프로그램가능성을 구현할 수 있고, 따라서 BIOS가 리셋 흐름을 완료한 후, BIOS는 그것을 미래 C6 종료들을 위해 다른 오프셋으로 셋팅할 수 있다. 예외 벡터의 메모리(104) 매핑이 안전한 부팅 결과들을 가질 수 있으며, 이 경우, PSP가 DRAM을 초기화하고 개스킷(210)으로 크레딧들을 송신하기 전 거기에 안전한 부팅 코드를 두어, 효과적으로 부팅 코드가 DRAM으로 매핑되게 한다는 것을 주의하자. 비-안전 부팅은 퓨즈 구성된 어드레스의 부팅 ROM에 위치될 수 있다. C6 종료 또는 안전 부팅은 예외 벡터를 DRAM에서의 FDF7 범위에 위치되는 C6 영역에서의 어드레스로 핀-스트랩핑할 수 있다. 그 후 개스킷(210)은 퓨즈 또는 C6ExitReset CCI 시그널링에 기초하여 이들 범위 간을 선택한다.
- [0119] 코어는 목시의 XC6를 구현하며, 이는 우리가 CC6에 진입하는 마지막 코어에 대해 XC6에 자동적으로 진입한다는 것을 의미한다. RXC3는 최종 결과로 초래된 클러스터 상태가 모두 RCC3 또는 RCC3 및 CC6 코어들의 혼합인 C-상태에 진입하는 마지막 코어에 대한 L2 유지를 요청함으로써 구현될 수 있다. RXC3는 캐시 플러시 이후 공통적으로 사용될 수 있다.
- [0120] L2는 플러싱되지 않고(L2 휴면 모드) 그 후 프로브들을 제공하기 위해 RXC3 외로 램핑한 채로 남겨질 수 있다. 그것을 해당 상태로 다시 리턴하기 전, 이의 L1이 플러싱되었을 수 있더라도 마지막 코어는 플러싱된 마지막 캐시(목시의 XC6 요건의 부분)로서 표시되지 않을 수 있고 프로브가 RXC3 외로 램핑함으로써 제공될 수 있다. RCC3는 단지 캐시 플러시 이후 상주를 가능하게 하기 위해 사용될 수 있으며, 이는 모니터에 의해 백킹된다.
- [0121] RXC3 모드는 약 80us일 것으로 예상되는 DRAM 저장/복원 페널티에 기인한 보다 높은 XC6 종료 레이턴시의 위험을 완화한다. RXC3는 이를 ~1us로 감소시킨다. 이는 프로브들에 대해 긴 레이턴시이나, 인터럽트들에 대해서는 짧다. 클러스터는 (CC6 및 RCC3와 함께) XC6 및 RXC3 간을 동적으로 재구성하기 위해 SMU에 의존적일 수 있다. RXC3 지지는 RCC3의 확장일 수 있다. RCC3에 진입하기를 요청하는 마지막 코어 상에서, 유니파이드 노스브리지(280)는 코어와 함께 L2와 Q-채널 핸드셰이크를 수행할 수 있다.
- [0122] 일반적으로 XC 상태들을 지지하기 위해, L2 스탠바이 핸드셰이크(ACINACTM/STANDBYWFIL2)가 구현될 수 있다. 이는 NbBusy 요건을 충족시키며, 이는 그것이 아키텍처상 유휴 상태이더라도 전력 관리를 위해 준비되지 않았다는 것을 시그널링하는 CPC의 방식이다. ARM 상에서, ACINACTM 및 STANDBYWFIL2에 대한 커밋 지점은 프로브 드레인 및 NbBusy와 상이하다. 따라서, 핸드셰이크는 저 전력에 커밋하기 전 수행될 수 있는데, 이는 프로토콜에 의해, L2가 유니파이드 노스브리지(280)이 무엇을 원하는지에 관계없이 STANDBYWFIL2를 보는 것을 금지하도록 허용되기 때문이다. 커밋 이후, 그 후 드레인, 및 L2는 장애를 방지하기 위해 NbBusy를 드롭하도록 보장한다. 따라서, XC-상태들에 대해, ACINACTM/STANDBYWFIL2 핸드셰이크가 RCC3 또는 CC6에 진입하는 마지막 코어 상에서 수행될 수 있고, 해당 핸드셰이크가 RXC3에 대한 L2 L2QACTIVE/L2QREQn 핸드셰이크와 병렬적으로 진행될 수 있다. 양 XC 핸드셰이크 플러스 마지막 코어의 RCC3 핸드셰이크가 완료되었을 때, SetCPULowPwr가 진행하도록 허용될 수 있다.
- [0123] RCC3와 유사하게, RXC3가 무효화된 TLB 및 BTB 및 디세이블된 SMP 모드(XC6와 유사한)로, 또는 SMP 모드 하에서 플러싱되지 않은 채로 남겨 허용될 수 있다. 단일 클러스터 구현 시, 해당 클러스터가 RXC3로 가면, 시스템에서 어떤 코어도 DVM들을 발생할 수 없고, 어떤 L2QACTIVE 어썬션 해제도 부여될 수 없다. 캐시들이 플러싱되고 GIC-400에서 인터럽트들이 마스킹되는 것을 고려할 때, L2QACTIVE 어썬션 해제 어느 하나에 대해 어떤 다른 조건도 없어야 하고, L2 Q채널 인터페이스는 CPU(102) 것들과 같이 사소하게 만들어질 수 있다. 클린 업은 먼저

유지에서 종료하고 인터럽트 발생들을 전달할 때 수행될 수 있다. 다중-클러스터 SOC에 대해, 그 후 클러스터가 SMP 모드로부터 디세이블되지 않았는지를 핸들링하는 DVM 브로드캐스트에 대한 처리가 수행될 수 있다. 이러한 상황에서, L2는 L2QACTIVE를 드롭하기 위한 전압 유지 동안 작동 중인 채로 남겨질 수 있고, 우리는 지정된 코어 상에서 종료를 강제하기 위해 L2QACTIVE 어썬션 해제로부터 IobcOrHalt 클리어링 term을 생성할 수 있다.

[0124] 복합 전력 게이팅 모드(XC6)가 채용될 수 있다. XC6 엔트리는 마치 RXC3에서와 같이 ACINACTM/STANDBYWFI L2 핸드셰이크를 사용할 수 있다. XC6 동안, 개스킷(210)은 전력 오프하고, 따라서 개스킷(210)은 모든 크레딧이 유니파이드 노스브리지(280)에 의해 릴리즈될 때까지 유니파이드 노스브리지(280)에 전달하지 않을 수 있다. 따라서, 개스킷(210)은 리셋 어썬션 해제 이후 가능한 빨리인, 크레딧들을 수용할 준비가 되었을 때 ResetUcodeGo를 구동할 책임이 있을 수 있다. 개스킷(210)은 어썬션 해제된 모든 코어에 대해 ResetUcodeGo를 STANDBYWFI 및 STANDBYWFIL2로 결부시킬 수 있을 수 있다. STANDBY 신호들은 전력 게이팅 동안 1로 분리되고 그 후 0으로 리셋팅 된다. 체로(0)는 분리가 CPL에 의해 릴리즈될 때까지 개스킷(210)에 노출되지 않을 수 있다. 그때, 개스킷(210)은 ResetUcodeGo를 어썬트함으로써 크레딧들을 수용할 준비가 될 수 있다. 이러한 시퀀스는 XC6 종료에 대해 부팅에 대해서와 같을 수 있다. 후자에 대해, 유니파이드 노스브리지(280)는 안전한 부팅 이유들로 인해 ResetUcodeGo 어썬션 시 크레딧들을 릴리즈하는 것을 지연시킬 수 있다. 이는 CC6 종료에 대한 경우는 아닐 수 있다.

[0125] 캐시/상태 플러시가 수행될 수 있다. 기존 유니파이드 노스브리지(280) 모니터들은 초기 정지 엔트리 상에서 보다 깊은 C-상태 상주를 예측하려고 시도한다. 이는 즉각적인 플러시 요청을 가능하게 하는, 성공, 또는 쉘로우(shallow) C-상태(CC1)를 이탈하는, 고장 중 어느 하나를 야기한다. 따라서, 단기 C-상태 간격들 상에서, 고장이 예측될 수 있고 인터럽트 기상은 저 레이턴시인, CC1 외로 전이할 필요가 있을 수 있다. 이러한 모니터는 예측된 고장이 예측 불가능한 또는 변경 워크로드들을 커버하기 위해, 타임아웃 이후 보다 깊은 C-상태로 전이하게 하는 보조 대기 메커니즘에 의해 백킹된다. CC6+ 상태들은 이들 모니터에 의해 백킹되고, RCC3는 쉘로우(사전 플러시) 또는 딥(사후-플러시) 중 어느 하나로 고려될 수 있다. 사전-코어(CC) 상태들 및 CPC-당(XC) 상태들 커버하는 모니터들의 두 개의 상이한 레벨이 또한 존재한다.

[0126] 모니터 또는 대기 메커니즘 중 어느 하나에 의해 발생하는 플러시 요청 시, StpClk은 타겟팅된 코어로 송신될 수 있다. 코어는 캐시(및/또는 RCC3 내 CC6에 따른 상태)를 플러싱하고, 적절한 비트들을 표시하여 유니파이드 노스브리지(280)에 플러싱된 상태를 시그널링하며 그 후 사전-플러시와 유사한 정지에 재-진입할 수 있다. 유니파이드 노스브리지(280)는 정지와 함께 플러시 상태를 관찰하고, CK에 대해 클로킹/전력 요청을 발생할 때 보다 깊은 C-상태로 진행할 수 있다.

[0127] 클러스터는 어떤 코어도 플러시를 개시하기에 이용가능하지 않은, 명시적 XC6에서 일반적으로 사용되는, 하드웨어 L2 플러시 위젯을 구현한다. 명시적 XC6가 코어 상에서 수행되기 때문에, 코어는 x86 코어들과 같은 기저 소프트웨어 및 하드웨어 인터페이스를 형성하기 위해 플러시를 개시할 수 있다.

[0128] 코어들은 L2FLUSHREQ가 어썬트될 수 있기 전에 WFI에 있을 수 있다. 코어는 SPR을 통해 플러시 요청을 개시하기 위해 사용되고, 완료를 기다릴 수 있다. 이와 같이, 유니파이드 노스브리지(280) 흐름이 저 전력에 진입하는 마지막 코어에 대한 L2 플러시 요청을 포함하지 않을 수 있기 때문에, 소프트웨어는 캐시 유지보수 동작들을 사용하여 L2 플러시를 수행할 수 있다. 이러한 접근법에 대해, 어떤 특정 로직도 L2를 플러싱하기 위해 유니파이드 노스브리지(280)에서 요청되지 않을 수 있고, 유니파이드 노스브리지(280)는 마지막 코어가 저 전력에 진입하기 위한 요청이 관찰될 때 L2가 플러싱된다는 것을 추정할 수 있다.

[0129] x86 유코드에서, APIC 인터럽트들이 대기 상태에 있지 않을 때 자동적으로 마스킹된다. 긴 지속 기간 서비스 루틴들 이를테면 캐시 및/또는 상태 플러시가, 인터럽트 불가능한 코드의 긴 구간들을 회피하기 위해, 유코드는 인터럽트 검사들(intchks)을 포함시키며, 이는 존재하는 경우 계류 중인 인터럽트들에 대해 코어의 인터럽트 예지들을 샘플링하고, 플러시 외로 중단한다. 클러스터는 동일한 소프트웨어 기능을 지지하지 않는다. 대신, 인터럽트들은 아래에 설명될 도 3의 전체 C6 엔트리 흐름을 통해 마스킹될 것이다. 단계(310)에서, 적절한 시스템 제어 레지스터 C 비트, 데이터 또는 유니파이드 캐시 이네이블이 추가적인 데이터 캐시 할을 방지하기 위해 클리어될 수 있다. 단계(315)에서, L2 프리페치들은 체로들을 CPU 확장 제어 레지스터의 비트들[38, 36:35, 33:32]에 기록함으로써 디세이블될 수 있다. ISB 명령은 CPU 확장 제어 레지스터 기록이 완료되었다는 것을 보장하기 위해 단계(320)에서 실행될 수 있다. DBS 명령은 임의의 이전 프리페치 요청들의 완료를 보장하기 위해 단계 325에서 실행될 수 있다. 단계 330에서, L1 데이터 캐시로부터의 모든 데이터가 클린되고 무효화될 수 있다. 이러한 프로세서를 위한 L2 중복 스냅 태그 RAM이 이제 빈 상태일 수 있다. 이는 멀티프로세서에서의 다른

프로세서들로부터의 임의의 새로운 데이터 캐시 스놉들 또는 데이터 캐시 유지보수 동작들이 이러한 프로세서로 발행되는 것을 방지한다. 단계 335에서, CPUECTLR.SMPEN 비트는 클리어될 수 있다. SMPEN 비트의 이러한 클리어링은 프로세서가 멀티프로세서에서의 다른 프로세서들에 의해 명령 캐시, TLB, 또는 BTB 유지보수 동작 브로드캐스트를 수신하는 것을 방지함으로써 프로세서가 일관성을 잃게 한다. 단계 340에서, 시스템은 어떤 인터럽트도 전력 감소되고 있는 프로세서로 송신되지 않는다는 것을 보장한다. 디버그 인터페이스들이 작업 거부되게 강제하는 단계 345에서 DBGOSDLR.DLK, 이중 로크 제어 비트가 셋팅된다. ISB 명령은 이전 단계들로부터의 모든 시스템 레지스터 변경이 커밋되었다는 것을 보장하기 위해 단계 350에서 실행된다. DBS 명령은 SMPEN 비트가 클리어되기 전 멀티프로세서에서의 임의의 프로세서에 의해 발행되는 모든 명령 캐시 TLB 및 분기 예측기 유지보수 동작이 완료되었다는 것을 보장하기 위해 단계 355에서 실행된다. 프로세서가 유휴 상태이고 WFI 저-전력 상태에 있다는 것을 나타내는 단계 360에서 WFI 명령이 실행되고 STANDBYWFI 출력은 출력에 대해 모니터링했다. 프로세서(102) 출력 클램프들은 단계 365에서 활성화된다. 단계 370에서 전력이 프로세서(102) 전력 도메인에서 제거된다.

[0130] 인터럽트 마스킹이 단계 310 이전에 CPSR.I/F 비트들을 셋팅함으로써 수행된다. 단계 310은 프로세서를 비-일관되게 만들 수 있고 그에 따라 이를 인식하지 않는 핸들러에 진입하기 위해 인터럽트를 필요로 하지 않을 수 있다.

[0131] Intchk는 구현될 필요가 없을 수 있는데 이는 (1) 하드웨어 L2 플러시 위젯을 구현함으로써, 유일한 소프트웨어 캐시 플러시 요건이 L1이며, 이는 단지 C6 엔트리에서의 캐시 플러시 오버헤드가 단일 디지털 마이크로초 범위에 있다는 것을 의미하는 5000 사이클을 단지 취하기 때문이며, (2) 인터럽트들이 CPSR.I/F 에 의해 마스킹되더라도 클러스터가 IRQ/FIQ 핀 어썬들 상에서 WFI를 종료하고 다음 명령을 실행하려고 진행할 수 있기 때문이다. 이는 전력 컨트롤러가 CPU(102)에 의해 요청되는 전력 감소를 이행하는 것 회피하는 것을 선택하는 경우 C6 엔트리에 대한 중단 메커니즘의 역할을 할 수 있다. 이는 단계 310 이전 CPSR.I/F 마스킹으로 시작하는 비-중단가능한 전체 시퀀스를 이네이블하기 위해 단계 340에서의 제시된 흐름(300)의 선택적 측면일 수 있다.

[0132] 캐시 플러시는 C6 진입+종료에 대해 가장 나쁜 경우 C6 인터럽트 레이턴시에 대해 증가하지 않을 수 있으며, 캐시 플러시의 비-인터럽트가능한 간격이 또한 SOC에서 롱 폴(long pole)이 아니다. 흐름은 CC6(XC6 상태로도 공지됨)에 진입하는 마지막 코어가 A15의 그것과 유사한 소프트웨어 흐름을 사용하여 L2를 플러싱할 필요가 있다는 점에서 제시된 흐름에서 벗어난다. 이유는 목시의 XC6 엔트리가 구현되기 때문이다. 소프트웨어 L2 플러시는 훨씬 더 오래 걸릴 수 있다. 적절한 인터럽트 서비스 레이턴시를 보장하기 위해, intchk들은 성능에 대해 그리고 x86와의 균일성에 대해 포함될 수 있다. intchk는 CCI_INT_EDGE 레지스터(F3x2F0)를 통해 FIQ/IRQ 상태를 노출시킴으로써 유니파이드 노스브리지(28) 측 상에서 구현될 수 있다. 따라서, 캐시 플러시 루틴에서, 소프트웨어는 CPSR.I/F 비트들을 셋팅하고, 그 후 캐시 플러시의 체크들과 사전 마스킹된 에지들을 폴링하는 것을 인터리빙할 수 있다. 체크 크기는 효율성, 즉, intchk들을 플러시 대역폭에 영향을 주지 않도록 저빈도로 만드는 것, 및 RCC3에 대해 ~2us 그리고 CC6에 대해 ~25us인, 상태들의 엔트리 및 종료 레이턴시에 비해 무의미하게 될 수 있는, 인터럽트 불가능한 간격 간 트레이드오프에 의해 결정될 수 있다. 단계 350에서 에지가 캐시 플러시 동안 레지스터에서 보여지는 경우, 플러시 루틴은 캐시 할당(단계 310) 및 L2 프리페치(단계 315)를 재 이네이블함으로써 일관성을 재 이네이블하고, 그 후 인터럽트를 취하기 위해 CPSR.I/F 비트들을 클리어할 수 있다.

[0133] 시퀀스가 CPUECTLR.SMPEN을 디세이블하기 위해 단계 335로 진행했다면, 실행이 단계 360까지 일어난다(단계 340을 스킵하여). 이들 단계 동안, 유니파이드 노스브리지(280)가 FIQ/IRQ를 타겟팅하면, WFI를 가리지 않고 다음 명령, 및 재 이네이블 캐시 할당, L2 프리페치, 및 SMP로 스텝. 그렇지 않으면, 시스템은 C6에 커밋한다. 유니파이드 노스브리지(280)는 C- 상태에 커밋하면서 인터럽트들을 적절히 마스킹하는 것을 처리할 수 있고, 따라서 단계 340이 불필요할 수 있다. 단계 340은 전력 컨트롤러가 인터럽트들을 디세이블하는 것 및 C6 엔트리에 커밋하는 것 간 위험 요소들을 회피하기 위해 요구될 수 있고, 단계는 유니파이드 노스브리지(280)가 지지한다. 이러한 구현은 엔트리를 시그널링하는 WFI를 계속 진행하기 전, 전력 컨트롤러에서 소프트웨어를 디세이블로 셋팅하기 위해 단계 340을 필요로 하고 그에 따라 모든 계류 중인 IRQ/FIQ 전달의 마스킹 및 작업 거부를 시그널링하는 ACK 백을 필요로 한다. 구현은 인터럽트들을 디세이블하기 위한 요구를 전달하기 위해 단계 360을 사용할 수 있고 인터럽트 및 커밋 간 하드웨어 인터로크에 의존적이다.

[0134] 단지 하나의 코어가 L2를 플러싱할 수 있고 이는 코어가 CC6을 종료할 때 L2 플러시가 중단되게 함으로써 복잡하게 된다. L2I 제어 비트는 L1 플러시들을 완료하고 L2와 등록 해제하며, 이는 하나의 코어가 마지막으로 되게 하고, 그리고 해당 코어가 L2 플러시를 수행하게 강제하는 다수의 코어가 있을 때, 작동하는 마지막 코어를 나타낸다. 이를 수행하는 하나의 방식은 어떤 코어가 벡터를 셋팅하는 마지막 코어인지 확인하기 위해 메모리

(104)에서의 별개 캐시 플러시 벡터 상에의 원자 액세스를 구현하는 것이다. L2 플러시를 수행하는 해당 코어는 중단을 위해 임의의 PwrStatusLo (F5xF0) CC6 비트 클리어링을 폴링할 수 있다.

[0135] 코어는 SMU 블록 내부 DAP 컨트롤러를 구현할 수 있다. 이는 CoreSight 디버그 구현의 부분이다. 표준 디버깅 도구들은 JTAG 또는 BP 핀 중 어느 하나를 통해 DAP에 인터페이스한다. DAP는 클러스터의 전력 상태를 인식하지 못하고, 클러스터 또는 시스템 요청들을 발생시킬 수 있다. DAP 요청들은 KDSM이라 칭해지는 VDD 도메인에 CoreSight 로직이 상주하게 할 수 있다. KDSM은 ACP 포트를 통해 클러스터 디버그 요청들을 하고, 시스템은 AXI 버스를 통해 개스킷(210)으로 요청한다. DAP가 클러스터 또는 시스템 중 어느 하나에 액세스하기 위해, VDD는 PC6 외로 램핑될 필요가 있다. 추가적으로, 개스킷(210)을 통해 시스템에 요청하기 위해, RXC3 또는 XC6은 종료 되고, nbsyn이 다시 스피닝하게 하며, 후자의 경우, UcodeResetGo 및 크레딧 릴리즈할 필요가 있을 수 있다. 복시의 XC6 구현으로 인해, 하나 이상의 코어는 CC6 외로 나올 수 있다.

[0136] 디버그 요청들을 제공하기 위해 저 전력 상태 외 적절한 로직을 획득하기 위해, DAP는 전력 관리 로직과 CSYSPWRREQ/CSYSPWRACK 인터페이스를 구현한다. 디버그 요청들을 삽입하기 전에, DAP는 먼저 CSYSPWRREQ를 어썬트할 수 있다. DbReq 인터럽트는 CSYSPWRREQ가 어썬트되는 한편 클러스터가 RXC3 또는 XC6 중 어느 하나에 있는 경우 시그널링될 수 있다. DbReq는 모드 코어를 재개할 수 있고, 인터럽트 핸들러는 NB_MISC를 통해 DbRdy를 시그널링할 수 있다. DbRdy는 CSYSPWRACK로 시그널링될 수 있으며, 이는 DAP가 그 후 요청하게 한다. 종료 시, DAP는 CSYSPWRREQ를 어썬트 해제할 수 있다. 코어들이 이제 C-상태가 아니고 저 전력에 재진입하게 하기 위해, CSYSPWRREQ는 DbReq 핸들러가 폴링할 SPR 상에 노출될 수 있다. 어썬트 해제를 볼 때, 핸들러는 DbRdy를 클리어 하고 그에 따라 CSYSPWRACK를 어썬트 해제하며, 그 후 인터럽트로부터 리턴할 수 있다. OS 유희 상태 루틴은 WFI 재엔트리를 인계받아 가능하게 할 수 있으며, 이는 그 후 C-상태 엔트리 프로세스를 재시작한다. 이는 PDM 모드와 유사하다. 그러나, 클러스터는 SBI_IAI에 대한 HDT2 인터페이스를 구현하지 않으며, 이는 종료가 SBI_IAI를 통해 시그널링되는 대신 종료를 위한 폴이 요구될 수 있다는 것을 의미한다.

[0137] x86에서, P-상태들은 ACPI 호들을 통해 OS에 노출될 수 있다. 그 후 OS는 미리 정의된 MSR들을 호출함으로써 P-상태 변경들을 요청하며, 이는 유코드가 유니파이드 노스브리지(280)에서의 CofVidCtl 레지스터에 P-상태 id를 트랩 및 기록하는 것이다. P-상태 id는 P-상태 테이블 엔트리에 매핑하며, 이는 프로그래밍된 주파수(FID/DID) 및 전압(VID) 셋팅을 가진다. 현재 및 요청 주파수 및 전압 셋팅들 간 차이가 존재하는지에 따라, 주파수 및/또는 전압 변경이 요청될 수 있다. 유니파이드 노스브리지(280)는 두 개까지의 요청을 내부적으로 발생시키고, 전압 증가가 전압 감소에 앞서 순서화되는, 주파수 변경에 앞서도록 요청들을 순서화할 수 있다. 코어 상호작용 없이 유니파이드 노스브리지(280) 및 CPL 간 전압 변경들이 내부적으로 충족된다. 주파수 변경은 코어 상호작용 필요로 한다. 유니파이드 노스브리지(280)는 전압 변경, 외부 StpClk들, 또는 캐시 플러시들과 같은 다른 전력 관리 요청들과 함께 유니파이드 노스브리지(280)에 의해 또는 다른 CU들/CPC들에 의해 요청된 것들과 주파수 변경을 순서화한다. 주파수 변경이 선택될 때, 유니파이드 노스브리지(280)는 StpClk를 CU/CPC에서의 모든 이네이블된, 비-CC6 코어로 타겟팅하고 CC6 코어들이 종료하는 것을 블로킹한다. 그 후 코어(들)는 StpClk를 취하고 HTC SBC 시스템 관리 브로드캐스트를 주파수 변경을 확인 응답하고, 모든 미처리 트래픽을 작업 거부하며, 그 후 ConnectMask를 드롭하기 위해 송신한다. StpClk로 타겟팅된 모든 코어가 이들 단계를 완료하고 그것들의 계류 중인 CC6 엔트리가 엔트리를 완료하면, 유니파이드 노스브리지(280)가 그 후 CPL에 대한 주파수 변경 요청을 할 수 있다. 완료 시, 유니파이드 노스브리지(280)는 CU/CPC로 StartClk 에지를 상승시키며, 이는 유코드가 이의 작업 거부를 종료하고 OS로 리턴하도록 촉진한다.

[0138] ARM 구현에서, ACPI 호 대신, OS는 CPUfreq 커널 드라이버를 호출하며, 이는 C-상태들을 위한 CPUidle 루틴의 것과 유사한, 안드로이드에의 벤터-중립 API 인터페이스를 구현한다. 안드로이드에 대해, CPUfreq 인터페이스는 최대 퍼센테이지에 의해 표시되는 성능 레벨의 것이다. 그 후 CPUfreq 루틴은 해당 퍼센테이지 레벨을 우리의 8 개의 이산적 P-상태 레벨 중 하나로 매핑하고 유니파이드 노스브리지(280)에서의 CofVidCtl 레지스터에서의 그것에 요청할 수 있다. 이런 점에서, 주파수 변경을 위한 유사한 흐름이 예상될 수 있으며, 이는 피커(picker) 선택, 작업 거부 주파수 요청 및 비 작업 거부를 필요로 한다. 작업 거부 및 비 작업 거부 부분들은 구현들이 상이한 것이다. 본 출원에서 아래에 설명되는 도 5 및 도 6에 도시된 두 가지 방법은 프로토콜/하드웨어 복잡도, 및 성능의 형태로 제시되는, 위험을 완화하기 위해 일체로 또는 개별적으로 구현될 수 있다. 일반적으로, 이들 방법은 프로세서를 ARM 정의 WFI 또는 WFE 상태들에 배치하기 위해 GIC를 사용하는 CPU를 인터럽트할 수 있다. 어느 하나의 상태는 CPU(420)가 추가 명령들을 실행하는 것 및 클록 도메인 경계에 걸쳐 새로운 트랜잭션들을 개시하는 것을 방지할 수 있다. 도 5 및 도 6은 아래에서 인터페이스/흐름 세부사항들을 제공한다. 작업 거부의 양 유형은 위험완화를 위해 구현되고 구성에 의해 그것을 가능하게 할 수 있다. 인터럽트 기반의 그

것들은 S3에 대해 구현될 필요가 있을 수 있고, 개스킷(210) 기반의 것으로 대체되지 않을 수 있으며, P-상태를 지지하기 위해 증가하는 비용이 제로에 가까울 수 있다. 개스킷(210) 기반의 것은 보다 양호한 성능을 가질 수 있으나 프로토콜 및 크로스-IP 검증 위험들을 캐리한다.

[0139] 기존 P상태 로직과의 연결성의 예시적인 다이어그램(400)이 도 4에 도시된다. 도 4는 기존 P-상태 로직과의 연결성을 도시한다. 도 4에 도시된 바와 같이, 인터럽트를 전달하는 ARM 일반화된 인터럽트 컨트롤러인 GIC(410)가 존재한다. GIC(410)는 유니파이드 노스브리지(280) 내에 위치될 수 있다. CPU(420)가 또한 도시된다. CPU(420)는 아틀라스 클러스터(290)의 부분일 수 있다. 개스킷(430)이 또한 도 4에 도시되고 개스킷(210)의 부분일 수 있다. 또한, NBCOR(440)이 도시된다. NBCOR(440)은 본 출원에서 위에서 논의된 CCI 프로토콜을 포함하여 유니파이드 노스브리지(280)의 부분일 수 있다. 보여질 수 있는 바와 같이 GIC(410), CPU(420), 및 개스킷(430)의 각각은 NBCOR(440)과 통신할 수 있다. 이들 통신 및 신호의 각각은 본 출원에서 아래에 설명될 것이다.

[0140] 인터럽트 기반 작업 거부 및 비 작업 거부는 P-상태들에 대한 최소 하드웨어 비용 접근법이 StpClk를 FIQ로 후킹함으로써 StpClk 기반 작업 거부를 수행하는 것이라는 것을 제공할 수 있다. FIQ는 안전 인터럽트로서 사용될 수 있다. 신뢰 준이 코어 및 GIC(410)에 구현되지 않으면, FIQ는 이용가능할 수 있고 StpClk는 복잡하지 않고 성능에 위험을 거의 제공하지 않을 수 있다. 정규 인터럽트들(IRQ에 연결된)에 대해, 소프트웨어 변환은 IRQ 예외 벡터 상에의 비-재진입이다. 이는 보다 높은 우선순위 IRQ 인터럽트가 도착하는 경우, 보다 높은 우선순위 인터럽트가 코어에 의해 현재 제공되는 보다 낮은 우선순위 IRQ 인터럽트를 선취하는 것을 가능하게 하지 않는다는 것을 의미한다. 성능 이유로 인해, 안드로이드는 인터럽트 서비스 루틴을 상반 및 하반으로 분해한다. 상반은 인터럽트 불가능한 절반이다. 변환은 호를 리턴하고 그에 따라 인터럽트 가능한 코드를 리턴하기 전에, 간단히 인터럽트를 취하고, 수행될 필요가 있는 것을 파악하며, 그 후 실제 작업을 수행할 OS로 하반을 스케줄링하는 것이다. 하반은 그 후 행해지는 실제 작업이다.

[0141] 그러나, 상이한 예외 유형들에 걸쳐, 하드웨어는 스택 포인터, 호 리턴을 위해 사용되는 링크 레지스터, 및 상태 레지스터들과 같은, 중요한 제어 흐름 레지스터들을 뱅킹(banking)함으로써 완전한 선취가능성을 보장한다. 변환에 따라, IRQ들은 FIQ들을 마스킹하지 않을 수 있고, 따라서 후자는 IRQ 핸들러의 상반에 있을 때에도 취해질 수 있다. 비-안전 시스템에서, 코어들이 작업 거부 상태로 들어갈 정규 작업 외에 취해진다는 것을 고려할 때, StpClk들은 P-상태 변경들을 위해 요구되는, 레이턴시가 거의 없이 취해질 수 있다. 그러나, 신뢰 준이 구현되면, FIQ 인터럽트는 안전 모니터의 프레임워크로 들어갈 필요가 있을 수 있다. P-상태 변경들은 등록될 수 있고 다른 StpClk는 보안 서비스들로서 소싱한다. FIQ 핀은 다른 서비스들과 공유될 수 있다. 이를 수행하기 위한 하나의 방식은 전체 시스템 상의 성능에 영향을 미치는 잘못 거동하는 신뢰 서비스를 위태롭게 하지 않으면, 변환에 따라 최소화될 FIQ 루틴들의 상반에 의존하는 것이다. StpClk는 배리어 동기화된 작업 거부로서의 역할을 할 수 있고, 여기서 늦게 도착하는 스레드는 병렬화되지 않은 프로그램으로서 거동한다. 대안에는 FIQ 예외 핸들러를 재진입하게 만드는 것이다. 리턴 어드레스가 스택으로 푸싱되는, 다른 아키텍처들과 같이 진정한 재진입 대신, 링크 레지스터는 각 예외 모드를 위해 뱅킹되고, 따라서 하나의 예외를 위한 재진입을 가능하게 하기 위해, 그 후 리턴 어드레스를 위해 이의 뱅킹된 링크 레지스터를 사용하기 위해 감독 모드(SVD)로 스위칭하며, 그에 따라 예외 벡터의 다른 엔트리를 위해 FIQ 링크 레지스터를 해제한다. 이는 FIQ로의 두 인스턴스의 재엔트리를 가능하게 할 수 있다. 다른 신뢰 서비스들의 StpClk 선점을 지지하기 위해, 그 후 FIQ 예외 핸들러는 StpClk를 제외한 다른 신뢰 서비스 인터럽트들을 위해 마스킹되게 인터럽트들을 남기면서 SVC 모드로 스위칭할 필요가 있다.

[0142] P-상태 StpClk 제공 시, 코드는 유니파이드 노스브리지(280)로 HTC SBC 시스템 관리 브로드캐스트를 송신하고, 데이터 동기화 배리어(DSB) 명령을 수행함으로써 모든 미처리 트래픽을 작업 거부하며, 그 후 인터럽트 대기(WFI/WFE)를 시그널링할 수 있다. 이러한 경우 DBS 명령은 휴지-대기로서의 역할을 한다. 휴지-대기는 미처리 요청들에 대한 모든 응답, 뿐만 아니라 토큰 릴리즈들을 기다리나, DSB는 단지 응답들을 기다린다. 유효한/준비 인터페이스의 ACE 구현을 고려할 때, 응답들을 보는 것은 작업 거부에 충분할 수 있다. 토큰 기반 SDP 인터페이스를 사용하는 구현은 SOC15 상의, 만약에 있다면, 작업 거부 상황들에 대해 진정한 휴지-대기를 필요로 할 수 있다. WFI/WFE는 간단히 유니파이드 노스브리지(280)에 주파수 변경에 대해 준비가 되어 있음을 시그널링하는 것이다.

[0143] 유니파이드 노스브리지(280)이 P-상태 변경에 대한 WFI의 형태로 코어 준비 상태를 보면, 유니파이드 노스브리지(280)는 이들이 선택되는 것을 블로킹함으로써 클러스터에 대한 프로브들을 작업 거부하고, 이미 선택된 프로브들에 관계없이 이의 파이프라인을 드레인하며, 클러스터가 이들을 완료할 때 미처리 프로브들이 이들의 버퍼들을 할당 해제하기를 기다릴 수 있다. 프로브들이 드레인되면, 유니파이드 노스브리지(280)는 CPL에서 P-상태

변경을 스케줄링하기 전 클러스터와의 추가적인 ACINACTM/STANDBYWFI2 핸드셰이크를 수행할 필요가 있다. ACINACTM은 SrqCPUProbesDrained이고 STANDBYWFI2는 SetCPULowPwr 피커에서 준비 상태를 블로킹하는 추가적인 항일 수 있다. 이러한 L2 핸드셰이크는 P-상태의 경우, 그것이 캐시 플러시로 사소하게 만들어지지 않고 프로브들이 능동적으로 드레인된다는 점을 제외하고, RXC3 및 XC6에 대해 수행되는 것과 유사하다. 프로브 드레인의 순서는 드레인 전 커밋이 발생하는 경우, x86에서 수행될 수 있는 것의 역전이다. 클러스터에 대해, 드레인은 커밋 전 먼저일 수 있다. 유니파이드 노스브리지(280)는 EnDrainPrbOnHtcStpClk를 통해 이러한 모드를 지지하며, 이는 커밋 이전에 프로브 드레인을 시작한다. 코어에 대해, EnDrainPrbOnHtcStpClk =1은 인터럽트 기반 작업 거부를 이네이블할 때 요구될 수 있다.

[0144] StartClk의 WFI 변형은 클러스터가 P-상태 변경의 중앙에 있는 한편, 클러스터가 시스템 요청을 하거나 인터럽트를 취할 수 없다는 것을 포함할 수 있다. 인터럽트 예지들은 전력 관리 이벤트들 동안 x86에서 APIC 예지들에 대해 수행됨에 따라 유니파이드 노스브리지(280)에 의해 마스크될 수 있다. 그러나, 소프트웨어는 그것이 유니파이드 노스브리지(280)에 요청들을 필요로 할 수 있기 때문에 이전에 설명된 intchk를 수행하지 않을 수 있다. 따라서, intchk 대신, 유니파이드 노스브리지(280)는 CPL이 P-상태 변경을 완료하면, 코어를 작업 거부에서 기상시킬 수 있다. 이는 StartClk 인터럽트와 수행될 수 있다. 그러나, StartClk 인터럽트는 StartClk가 선택될 필요가 있다는 것을 암시한다. FIQ 재진입이 StpClk 동안 가능하게 되면, 그리고 신뢰 존이 이미 재진입을 필요로 하면, 그것은 세 개까지의 레벨을 추가한다. 다른 안전 서비스들은 비-재진입하게 만들어지거나 StpClk를 통해 비 표준의 인터럽트 재진입을 구현할 수 있다. StpClk에서 인터럽트에 바람직한 유일한 인터럽트가 StartClk라는 것을 고려할 때, 예를 들어 스택에 푸싱하는 대신 범용 레지스터에 링크 레지스터 어드레스를 저장하여, 숏컷들(shortcuts)이 취해질 수 있다. 전자는 가장 높은 FIQ 우선순위를 갖도록 StartClk를 구성하는 것을 필요로 하며, 이는 신뢰 존의 밴더 지지가 가능한 제공되어야 한다.

[0145] WFI를 사용하는 P상태 시퀀스의 예시적인 다이어그램(500)이 도 5에 도시된다. WFI를 사용하는 P상태 시퀀스(500)는 단계 510에서 P상태를 요청하는 드라이버를 포함할 수 있다. 단계 520에서, 모든 코어로의 StpClk 송신이 수행되고 그 후 STANDBYWFI 대기가 대기된다. 단계 520과 거의 동시에, 단계 530은 nFIQ 상에 보여지는 StpClk를 제공하고 소프트웨어는 FIQ 인터럽트 핸들러 루틴에 진입한다. 단계 540에서, FIQ 인터럽트 핸들러가 활성화되고, HTC SBC는 DBS(작업 거부) 및 WFI 명령을 브로드캐스트한다. 단계 550에서, 프로브들이 드레인/작업 거부되고 ACINACTM/STANDBYWFI2 핸드셰이크가 시작된다. 단계 560에서, 코어 FID/DID/VID가 변경되고 StartClk이 SPI를 통해 송신된다. 단계 570에서, nFIQ는 WFI로부터 코어를 기상시키고 인터럽트 핸들러를 건너뛰며 StartClk 인터럽트를 클리어한다.

[0146] StpClk로부터 기상시키기 위한 다른 방식은 작업 거부 이후 WFI 대신 WFE를 사용하는 것이다. WFE 기상은 EVENTI 핀을 통해 인터럽트들 또는 이벤트들 중 어느 하나로부터 트리거할 수 있다. 따라서, 모든 인터럽트, IRQ 및 FIQ는 마스크 오프되고 EVENTI를 여전히 기상시킬 수 있으며, 이는 본질적으로 StartClk의 목적에 기여한다. WFE StartClk을 이네이블하기 위해, STANDBYWFE가 (HtcStpClkActiveCore에 기초하여) 모든 P-상태에 대해 ~ConnectMask로 그리고 모든 다른 상황에 대해 STANDBYWFI로 멀티플렉싱될 수 있다. 그 후 StartClk은 종료를 트리거하기 위해 이러한 모드에서 EVENTI까지 후킹될 수 있다. 코어가 WFE로부터 종료되면, 그것은 여전히 StpClk 핸들러에 있을 수 있으며, 이는 인터럽트들을 언마스크하고 종료될 것이다.

[0147] 다수의 클러스터를 지지하기 위해, StartClk 예지는 P-상태들을 수행할 때 EVENTI로 그리고 모든 다른 상황에서 EVENTO로 멀티플렉싱될 수 있다. 정규 모드에서, 모든 EVENTO는 함께 OR되고 EVENTI로 유도된다. 어떤 EVENTO도 존재하지 않을 수 있기 때문에, 이들은 스와프될 수 있고 내부의 StartClk 예지가 EVENTI로 매핑될 수 있다.

[0148] 코어에 대해, 구성에 의한 어느 하나의 StartClk 변형이 이네이블될 수 있다. 기존 StartClkEdge는 StartClk으로 할당되는 EVENTO 및 GIC(410) 분배기 SPI 핀들 양자로 라우팅될 수 있다. WFE StartClk 모드가 이네이블될 때, ConnectMask 및 EVENTI는 적절한 WFE 시그널링을 선택할 수 있고, StartClk과 연관된 SPI id들이 디세이블될 수 있다. WFI StpClk 모드에 대해, StartClk SPI id들의 GIC(410) 프로그래밍이 추정될 수 있다.

[0149] 코어는 그것이 StpClk 제공 시 SEV 명령들을 실행하는 것일 수 있는 사용자 코드에 있을 수 있다. 상이한 코어들은 상이한 시간들에 인터럽트를 취할 수 있고, 따라서 어떤 소프트웨어도 없이 하나의 코어가 StpClk 핸들러에서 WFE를 이미 실행할 수 있었을 수 있는 한편 다른 코어는 StpClk를 취하기 전에 SEV를 실행한다는 보장이 가능하다. 이러한 흐름을 적절히 제공하기 위해, StpClk 핸들러가 WFE를 실행하기 전에 소프트웨어 동기화를 수행할 수 있다. 유니파이드 노스브리지(280)는 하드웨어 세마포어 및 스캐치 레지스터들을 제공한다. StpClk 핸들러는 WFE로 전진하기 위해 PwrStatusLo 레지스터로부터 CC6 상태를 사용하는 것과 함께, 핸들러에서 각 코

어의 존재를 원자적으로 업데이트하기 위해 하드웨어 메커니즘들을 사용할 수 있다.

- [0150] L2는 STANDBYWFE2 인터페이스를 구현하지 않을 수 있다. WFE 이전에, 코어들이 모든 미처리 요청을 작업 거부하는, DBS 명령을 실행할 수 있다면, L2에의 어떤 계류 중인 요청 또는 할당도 존재하지 않아야 한다. 이는 DRAM에 대한 어떤 L2 프리페치 또는 빅팀도 존재하지 않아야 한다는 것을 보장할 수 있다. 이것이 사실이라는 것을 보장하기 위해, 소프트웨어는 DSB 이후 WFE를 지연시키고, 그렇게 함으로써 P-상태 변경에 커밋하는 것으로부터 유니파이드 노스브리지(280)를 지연시킬 수 있으며, 이는 STANDBYWFIL2를 기다리지 않고 발생할 필요가 있다.
- [0151] WFE를 사용하는 P-상태 시퀀스의 예시적인 다이어그램(600)이 도 6에 도시된다. 시퀀스(600)는 단계 610에서 P-상태를 요청하는 드라이버를 포함한다. 단계 620에서, StpClk가 모든 코어로 송신되고 시스템은 STANDBYWFE를 기다린다. 거의 동시에, 단계 630에서 StpClk는 nFI 상에 보여질 수 있고 소프트웨어는 FIQ 인터럽트 핸들러 루틴에 진입한다. 단계 640에서, FIQ 인터럽트 핸들러가 준비 상태가 되고 HTC SBC가 브로드캐스트이고, 코어 간 소프트웨어 동기화(SEV에 대한)가 DSB(작업 거부)와 함께 발생하며 L2 작업 거부로 인한 소프트웨어 지연이 WFE 명령과 함께 발생한다. 단계 650에서 프로브들은 드레인/작업 거부된다. 단계 660에서, 코어 FID/DID/VID가 변경되고 StartClk이 EVENTI를 통해 송신된다. 단계 670에서, EVENTI는 WFE로부터 코어를 기상시키고 StpClk 핸들러(nFIQ)로 다시 돌아간다.
- [0152] StpClk/StartClk 기반 작업 거부는 기존 유니파이드 노스브리지(280)로부터 최소한 증가하는 로직을 필요로 한다. 그러나, 성능에의 위험이 존재하고 그것은 신뢰 준 인터럽트들이 코어 내부에서 핸들링될 필요가 있는 경우 추가적인 소프트웨어 복잡도를 추가한다. 이들 위험을 완화하기 위해, 작업 거부 대안적인 모드는 개스킷(210) 내에 구현되게 되고 클러스터에 보이지 않게 만들었다. 높은 레벨 상에서, ACE는 계속해서 작동하기 위해 클러스터 및 개스킷(210) 간을 인터페이싱하나 NBSYN의 어느 하나의 단부 상의 CCI 인터페이스를 작업 거부한다. 개스킷(210)은 새로운 요청들을 블로킹하고 코어 대신 미처리 응답들을 기다릴 수 있다. 이는 상기 위험들을 주로 개스킷(210)에서의 추가적인 하드웨어의 것, 및 이러한 구현이 따르는 프로토콜 의존성과 트레이드한다. 주요 프로토콜 의존성은 프로브 및 라이트백들 간이다. ACE 상에서, 라이트백들(빅팀 요청들)은 어떤 다른 의존성도 없이 DRAM으로 포워드 진행되는 정규 요청들로부터 별개 가상 채널 상에 배치된다. 클러스터는 이들 라이트백이 완료할 때까지 라이트백들을 히팅하는 프로브들이 블로킹하게 함으로써 이에 의존적이다. 따라서, 유니파이드 노스브리지(280)가 프로브들을 작업 거부하기 위해, 코어는 라이트백들을 완료하는 것을 가능하게 되어야 한다. 도 8에 도시된 방법(800)은 개스킷(210) 및 유니파이드 노스브리지(280) 간 NBSYN 에 걸쳐 모든 인플라이트 트래픽을 드레인하는 것을 보장하기 위해 제공된다. 단계 810에서, 방법(800)은, P-상태 변경 요청 시, 유니파이드 노스브리지(280)가 StpClk로서의 역할을 하여 작업 거부하기 위해 사이드밴드를 통해 개스킷(210)에 시그널링한다. 단계 820에서, 개스킷(210)은 새로운 비-라이트백 요청들이 유니파이드 노스브리지(280)으로 송신되는 것을 블로킹하고 모든 미처리 요청을 드레인한다. 완료될 때, 개스킷(210)은 유니파이드 노스브리지(280)에 "작업 거부" 응답을 시그널링한다. 이는 HTC SBC 시스템 관리 명령과 같이 동작한다. 단계 830에서, 유니파이드 노스브리지(280)는 새로운 프로브들이 선택되는 것을 블로킹하고 이의 프로브 버퍼 크레딧들을 검사함으로써 미처리 프로브들이 완료하기를 기다린다. 단계 840에서, 개스킷(210)은 단지 어떤 프로브도 유니파이드 노스브리지(280)로부터 미처리되고 있지 않을 때 라이트백을 블로킹한다. 개스킷(210)은 유니파이드 노스브리지(280)에 계류 중인 라이트백들이 존재하는 한 NbBusy를 구동할 수 있다. 단계 850에서, 유니파이드 노스브리지(280)는 단지 모든 프로브 버퍼가 리턴되고 개스킷(210)이 NbBusy를 어썬트하지 않고 있을 때 P-상태 변경 개시하도록 허용될 수 있다.
- [0153] 비 작업 거부 부분은 작업 거부 요청을 비활성화하는 유니파이드 노스브리지(280) 및 "작업 거부" 응답을 제거함으로써 응답하는 개스킷(210)에 의해 수행될 수 있다. 그 후 유니파이드 노스브리지(280)는 ACK(응답)가 새로운 요청을 개시하기 전 이전 핸드셰이크로부터 어썬트 해제되었다는 것을 보장할 필요가 있다. 따라서, 유니파이드 노스브리지(280)에 대한 변경은 P-상태들에 대해 StpClk들의 사용을 비활성화하기 위한 그리고 작업 거부를 위한 개스킷(210)에 대한 사이드밴드 요청/ACK일 수 있다. 우리는 현재 설계에서 이미 양 상태를 구동하고 싱크(sink)하기 때문에, 이는 간단해야 한다. 그러나, 이는 개스킷(210)에 추가적인 작업을 추가할 수 있다. 이는 이들이 캐시 플러시 및 S3에 대해 요구될 수 있음에 따라 함께 StpClk들을 제거하지 않는다. 캐시 플러시에 대해, intchk들이 사용될 수 있고 S3가 가중하게 OS 및 BIOS 수반된다.
- [0154] 개스킷 작업 거부를 사용하는 P상태 시퀀스의 예시적인 다이어그램(700)이 도 7에 도시된다. 이러한 P상태 변경은 두 개의 경계 간에 놓여지는 개스킷(210)에 인터페이스 트랜잭션 진행의 정지를 제공함으로써 발생할 수 있다. 시퀀스(700)에 도시된 바와 같이, 단계 710에서 P상태를 요청하는 드라이버를 포함할 수 있다. 단계 720에

서, NB_QuiesceReq는 개스킷(210)으로 송신된다. 거의 동시에, 단계 730에서, NB_QuiesceReq는 개스킷(210)에 의해 보여진다. 단계 740에서, 유니파이드 노스브리지는 새로운 프로브들 및 응답들이 선택되는 것을 블로킹하고, 프로브들을 드레인하며, NbBusy가 어썬트 해제되기를 기다린다. 단계 750에서, 코어 FID/DID/VID가 변경되고, NB_QuiesceReq가 개스킷(210)에 트래픽 재개를 알리기 위해 어썬트 해제된다. 단계 760에서, 드라이버는 새로운 주파수로 사용자 코드를 지속한다.

- [0155] 유니파이드 노스브리지 P-상태는 CPU P-상태로서 동일한 메커니즘을 통해 수행될 수 있다. 다중-클러스터 시스템에 대해, 모든 클러스터는 CPU P-상태에 의해 타겟팅되는 단지 단일 클러스터와 대조적으로 작업 거부될 필요가 있다. 유니파이드 노스브리지 C-상태는 변경되지 않은 채로 유지될 수 있다.
- [0156] S3 개시는 OS 및 BIOS 간 협력에 의해 수행될 수 있다. StpClk 지지는 모든 디바이스가 D3에 배치된 후 FCH에 대한 IO 관독에 의해 개시될 수 있다. 그 후 FCH는 외부 StpClk 흐름을 개시한다. ARM StpClk 흐름은 SPI를 통해 GIC(410)로 모든 코어로 StpClk를 시그널링하는지 제공하기 위해 사용될 수 있다. 인터럽트 핸들러는 인터럽트들을 마스킹하고 그 후 StpGnt SBC를 유니파이드 노스브리지(280)로 송신할 수 있다. 그때, 코어들은 전력 오프될 때까지 인터럽트를 StpGnt에 마스킹되게 유지할 수 있다. 인터럽트 핸들러는 StartClk을 설명할 필요가 없으나, 그것이 코드를 제공되는 P-상태 변경들과 공유하는 경우 그렇게 할 수 있기는 하더라도 후자는 인터럽트 기반 작업 거부를 지지한다. 소프트웨어 델타 대 P-상태 변경은 단지 특별한 브로드캐스트 사이클의 인코딩된 유형이다. 유니파이드 노스브리지(280)는 DRAM 셀프-복구로의 엔트리를 개시하기 위해 모든 코어 상의 StpGnt 상태를 사용할 수 있다. FCH가 StpGnt를 수신한 후, FCH는 시한 대기를 시작하고, 워م 리셋(warm reset)을 폴링하며 VDDNB를 램핑 다운할 수 있다. 종료 시, S3은 콜드 리셋(cold reset)과 유사하고 부팅 대 S3 종료를 확인하기 위해 소프트웨어에 의존할 수 있다.
- [0157] CS 흐름은 주로 종료 가속 및 보안을 돕기 위한 S3의 상부 상의 다른 소프트웨어 계층이다. 하드웨어는 저장 및 재개 프로세스 동안 이의 상태를 추적하기 위해 소프트웨어에 대한 몇몇 CS 특정 레지스터 필드를 지지한다.
- [0158] CAC는 코어 CAC 관리자(DPM으로도 공지됨)에서 유니파이드 노스브리지(280)들 간을 인터페이스하고 클러스터에 존재하지 않을 수 있다. DPM은 개스킷(210)에서 구현될 수 있다. DPM은 FU 요청/응답 인터페이스를 통해 유니파이드 노스브리지(280)와 통신할 수 있다. DPM은 클러스터 성능 모니터 출력들을 사용하고 CAC 가중치들을 할당할 수 있다. 그것이 구현되는 경우, 양방향 어플리케이션 전력 관리(BAPM; Bidirectional application power management)가 지지될 수 있다. 그것이 구현되지 않는 경우, BAPM은 온도 기반이고 그에 따라 보다 덜 정확할 수 있고, 에너지 축적이 디세이블될 수 있다. 그러나, 어느 하나의 경우, SMU는 기존 SmuPstateLimit을 통해 BAPM P-상태 제한을 전달할 수 있다.
- [0159] 인텔리전트 P-상태(iPstate)의 하나의 예는 FP 조절이다. 이는 x86 코어들 상에서 HDT2 인터페이스로 구현된다. 클러스터는 CoreSight라 칭해지는 유사한 인터페이스를 갖는다. iPstate가 HDT2를 CoreSight로 매핑하기 위해 CPL 변경들을 요청한다고 가정하자. iPstate는 AVFS 피쳐와 공유된 유니파이드 노스브리지(280)에서의 SMU P-상태 하드웨어 메커니즘을 사용할 수 있다. SMU P-상태에 대한 유니파이드 노스브리지(280) 및 SMU 간 인터페이스가 동일하게 유지될 것으로 예상된다.
- [0160] C-상태 부스트는 나머지 코어들이 CC6에 있을 때 코어들의 서브셋이 가능한 보다 높은 전압에서 Fmax로 부스팅되게 한다. C-상태 부스트는 SMU 상호작용들 없이 유니파이드 노스브리지(280) 하드웨어에서 전적으로 핸들링된다. 그것이 CC6 및 P-상태 변경들의 백엔드 상에서 작동하기 때문에, P-상태들 및 C-상태들을 지지하도록 요청되는 어떤 증가하는 변경도 존재하지 않는다.
- [0161] 전용 라이트백 채널이 교착 상태들을 회피하기 위해 사용될 수 있다. 하나의 하드 토큰이 CPU(102)로부터의 VicBlk 요청들을 위해 XCS/SPBC/SPBD/MCQ/MPBD 추가될 수 있다. 코어는 미처리 빅타임과 충돌 시 프로브들에 응답하지 않는다. 빅타임 채널은 드레인을 위해 사용될 수 있고 빅타임 명령이 수행된 후, 코어는 손실로서 프로브에 응답할 수 있다. CPU(102) 빅타임들은 그것을 보장된 방식으로 MCT에 이르게 할 수 있다.
- [0162] 이제 교착 상태의 흐름(900)을 도시하는 도 9를 또한 참조하자. 단계 910에서 관독(Rd)이 MCT에 이른다. 단계 920에서 프로브가 론칭되고 단계 930에서 프로브는 CPU(102)에서의 빅타임을 히팅한다. 단계 940에서 CPU(102)는 WrVicBlk을 발행하며, 이는 MCT에 이르나 그 후 MCT의 어드레스-기반 순서화 로직은 효과가 나타나기 시작하여 단계 950에서 원래 Rd가 완료할 때까지 WrBlk이 DCT에 발행되게 하지 않을 것이다. 단계 960에서 프로브 응답이 수신될 때까지 Rd는 완료되지 않을 것이고 단계 970에서 코어는 이러한 WrVicBlk에 대해 완료된 타겟이 코어로 다시 송신될 때까지 Rd에 대해 프로브 손실로 응답하지 않을 것이다. 이것이 교착 상태 상황이다.

- [0163] CPU(102)는 기록 데이터 채널 상에 데이터를 개스킷(210)으로 푸시할 수 있다. CPU(102)는 CPU(102)가 충돌 프로브에 응답하기 전 라이트백이 완료됨을 나타내는 기록 응답을 기다린다. 이는 노스브리지(280)가 데이터를 개스킷(210)으로부터 폴링하고 개스킷(210)이 기록 버퍼 릴리즈에 의해 전달 완료된 타겟을 수신한 후 제공될 수 있다. CPU(102)가 라이트백을 발행하는 경우, 개스킷(210) 및 SRI는 메모리(104)에 라이트백을 전달할 수 있다. 이전 판독으로부터 프로브를 통해 CPU(102)로의 교착 상태(900)는 라이트백이 완료하기를 기다리고 있다. 라이트백은 이를테면 즉각적으로, 예를 들어, 개스킷(210)에서 CPU(102)로부터 확인 응답되고 개스킷(210)이 프로브들에 응답하는 것 및 빅탐 상태를 저하하는 것을 포함하여 일관성을 유지하는 것에 책임이 있게 할 수 있다. 개스킷(210)은 가득 찬 상태일 수 있고 블로킹 라이트백은 CPU(102) 내부에 있을 수 있으며 여전히 프로브를 블로킹한다. 교착 상태(900)는 발행된 라이트백들에 의해 야기될 수 있고 내부 CPU(102)에 있는 것들은 프로브들을 블로킹하기를 큐에 요청한다. CPU(102)로부터 MCT로의 가상 채널이 생성될 수 있으며, MCT는 라이트백들이 보다 오래된 요청들을 전달하게 한다.
- [0164] 개스킷(210)은 비-라이트백 기록들로 가득차게 될 수 있다. 그러나, 이들은 포워드 진행하도록 이미 보장되고 가득 찬 개스킷(210)이 프로브 교착 상태를 야기하는 내부적으로 큐잉된 CPU(102) 라이트백을 블로킹하는 것을 방지하는 라이트백들 및 비-라이트백들을 인터리빙하는 것에 대한 CPU(102) 제약이 존재한다.
- [0165] 캐시에서의 더티 라인을 히팅하는 프로브는 빅탐이 DRAM으로 실제로 드레인될 때까지 응답하지 않을 수 있다. 이런 이유로, 교착 상태 없는 시스템을 생성하기 위해, 새로운 라이트백 채널이 추가되어야 한다. 이러한 새로운 라이트백 채널 전용 버퍼들이 XCS/SPBC/SPBD/MCQ/MCD에 할당될 수 있고, 따라서 보장된 포워드 진행이 빅탐들을 위해 제공될 수 있다. SRQ는 코어로부터의 빅탐/라이트백을 위해 전용 하드 할당 토큰을 필요로 할 수 있다. 이러한 라이트백 채널은 원래의 개스킷(210)으로 확장할 수 있고 개스킷(210)에서의 토큰 할당 기법은 빅탐들이 코어로부터 SRQ로 포워드 진행한다는 것 및 SRQ가 빅탐을 위해 이용가능한 토큰을 항상 가질 것이라는 것을 보장할 수 있다.
- [0166] 교착 상태를 방지하기 위해 4-홉 프로토콜은 PrbTgt 플레이버(flavor)를 발행할 수 있다. CPU(102) 클러스터가 단지 존재할 수 있기 때문에, 단지 CPU(102)로부터의 요청들은 프로브들을 GPU로 송신해야 하고 그것들의 응답들은 MCT로 간다. 유사하게, GPU로부터의 RdBlkS 요청들은 단지 CZ 상의 PrbTgt일 수 있고 응답들은 다시 MCT로 간다. I/O 요청들은 원래의 MCT로의 프로브 응답을 수신하도록 강제될 수 있고 그 후 MCT는 더티 또는 클린 데이터를 갖는 최종 응답을 송신할 수 있다. 이는 다시 DRAM을 판독하기 위한 기법의 구현을 가능하게 할 수 있다. 프로브/빅탐 충돌들이 프로브 응답들을 정지하고 빅탐이 포워드 진행하기를 예상함으로써 핸들링된다.
- [0167] 빅탐이 완료하면 그것은 프로브 응답 손실을 발행할 수 있다. 이는 전용 라이트백 채널, 빅탐들이 MCT에서의 보다 오래된 요청들을 전달하게 하는 것, 및 MCT가 프로브 소스 대신 호스트 브리지 판독을 위한 프로브 타겟을 발생하게 하는 것을 필요로 한다. 프로브 타겟을 발행하는 것은 MCT가 프로브 응답들, DRAM 판독 데이터, 및 가능한 빅탐 패싱 케이스들을 통합하게 그리고 그 후 단일 판독 응답(또는 변환 케이스에서 TgtDn)을 다시 SRI로 발행하게 할 수 있다.
- [0168] XBAR->MCT 요청들은 이제 MCD 엔트리를 더티 데이터 또는 보다 신규한 패싱 빅탐으로부터의 데이터에 할당한다. 다른 XBAR->MCT 요청들을 가질 때 McdLo 및 McdLo 토큰들 양자가 예비 할당된다. 데이터를 캐리하지 않는(즉 CPU(102)판독들) 비-호스트 브리지 요청들은 그것들의 토큰들을 mp1에 즉시 릴리즈한다. 사용되지 않는 채널에 대한 호스트 브리지(HB) 판독 토큰들은 mp1에 릴리즈된다(새로운 항들 Hb4HopMcdLoBufAvail/Hb4HopMcdHiBufAvail). 이전에 단지 데이터를 갖는 명령들은 MCD 토큰들을 할당했다. 데이터 명령들을 위한 MCD 토큰 관리 로직이 변경되지 않는다.
- [0169] 라이트 백들 및 프로브들 간 교착 상태를 해결하기 위해, 가상 채널이 라이트백들을 위해 추가될 수 있다. 이러한 피처의 부분은 그것이 MCT에 이르면 라이트백들이 보다 오래된 판독들에 앞서 재순서화하게 하는 것을 수반한다. 이는 이 우회가 발생할 때 보다 이전 DRAM 판독들을 재발행하는 것과 같은, 상당한 수의 변경을 수반한다. CPU(102)는 프로브된 블록이 빅탐화된 프로세스에 있는 경우 프로브 응답을 리턴하지 않을 수 있다. 대신, CPU(102)는 빅탐을 단지 송신한다. 취해지는 접근법은 빅탐이 실질적으로 프로브 응답이라는 것을 검출하고 이와 같이 취급될 수 있다는 것을 검출하는 것이다.
- [0170] 빅탐이 도착할 때, 그것은 보다 오래된 op들과의 어드레스 매치를 위해 검사될 수 있다. 매치가 발견되는 경우, 빅탐은 그것이 다음 기준들을 충족하는지 보기 위해 검사될 수 있다:
- [0171] 프로브 응답을 기다리고 있는 판독(프리페치가 아닌)에의 매치인지, 또는

- [0172] 프로브 응답을 기다리고 있는 기록에의 매치인지.
- [0173] 다수의 어드레스 매치가 검출되는 경우, 가장 오래된 것이 선택된다. 이는 가장 오래된 op가 임의의 이전 op들은 퇴거하기를 기다리지 않는 일자라는 원리를 사용하여 이제 MCA에 저장되는 MCQWaitOpRet 플래그의 카피에 의해 확인된다.
- [0174] 따라서 빅팀이 도착할 때, 그것은 다음 세 흐름 중 하나를 따를 수 있다:
- [0175] 1. 빅팀이 응답을 기다리고 있는 이전 op에 매치하지 않는 경우, 빅팀은 그것이 지금까지 있었던 것처럼 취급된다.
- [0176] 2. 빅팀이 응답을 기다리고 있는 기록에 매치하는 경우, 빅팀은 프로브 응답으로서 취급되고, 이전 기록의 MCD 엔트리로 병합된다. 빅팀에 할당되었던 새로운 MCD 엔트리가 릴리즈된다. 이러한 유형의 빅팀은 포워딩된 데이터의 소스인 것으로 간주될 수 없는데 이는 그것이 이 자체의 MCD 엔트리를 가지지 않기 때문이다. 빅팀은 DRAM(빅팀이 합병되는 기록에 의해 핸들링되는)에 기록을 수행하지 않는다. 그것은 이 자체의 MCQ 엔트리에 할당되고, 그것이 수행된 때 TgtDn을 송신할 수 있다.
- [0177] 3. 빅팀이 응답을 기다리고 있는 관독에 매치하는 경우, 빅팀은 관독의 MCD 엔트리로 병합되고 이 자체의 MCD 엔트리에 기록되는데, 이는 그것이 DRAM으로 송신될 필요가 있기 때문이다. 새로운 MCD 엔트리에서의 빅팀 데이터는 정규 빅팀과 같이 취급된다; 그것은 DRAM으로부터 포워딩되고, 이에 기록될 수 있으며, 완료될 때 TgtDn을 송신한다. 제2 MCD 기록 포트가 새로운 MCD 엔트리를 기록하기 위해 흐름 #3(빅팀-매치-관독)에 대해 요구될 수 있다.
- [0178] 실시예들
- [0179] 1. 입력/출력 메모리 맵 유닛(IOMMU)을 포함하는 프로세서로서, IOMMU는 페이지 테이블 엔트리에 액세스할 때 액세스 비트를 셋팅하도록 구성되는, 액세스 비트 메커니즘을 포함한다.
- [0180] 2. 실시예 1의 프로세서에 있어서, 액세스 비트 메커니즘이 레지스터들에 의해 이네이블된다.
- [0181] 3. 실시예 1 또는 2의 프로세서에 있어서, IOMMU는 더티 비트 메커니즘을 더 포함한다.
- [0182] 4. 실시예 3의 프로세서에 있어서, 더티 비트 메커니즘은 단지 액세스 비트 메커니즘이 이네이블되는 경우 이네이블된다.
- [0183] 5. 실시예 3 또는 4의 프로세서에 있어서, AP[2] 비트는 어떤 면에서 페이지가 더티인지 또는 클린한지를 나타낸다.
- [0184] 6. 실시예 3 내지 5 중 어느 하나의 프로세서에 있어서, 액세스 비트 및 AP[2] 비트가 동일한 페이지 변환 테이블 엔트리로부터 업데이트된다.
- [0185] 7. 실시예 6의 프로세서에 있어서, 액세스 비트 및 AP[2] 비트가 단일 원자 동작으로 업데이트된다.
- [0186] 8. 실시예 7의 프로세서에 있어서, 단일 원자 동작은 테스트-0-논리 OR 동작을 포함한다.
- [0187] 9. 실시예 7의 프로세서에 있어서, 단일 원자 동작은 테스트-셋-클리어 동작을 포함한다.
- [0188] 10. 실시예 1 내지 9 중 어느 하나의 프로세서에 있어서, 액세스 플래그가 하드웨어 메커니즘에 의해 관리된다.
- [0189] 11. 실시예 10의 프로세서에 있어서, 0으로 셋팅된 액세스 플래그를 갖는 변환 테이블 엔트리가 변환 색인 버퍼(TLB)로 관독되는 경우, 하드웨어는 변환 테이블 엔트리의 액세스 플래그 비트에 1을 기록한다.
- [0190] 12. 실시예 10 또는 11의 프로세서에 있어서, 하드웨어 액세스 플래그 필드는 이러한 구현 선택을 나타내기 위해 사용된다.
- [0191] 13. 실시예 10 내지 12 중 어느 하나의 프로세서에 있어서, SCTLR.HA 비트는 액세스 플래그의 하드웨어 관리를 이네이블하기 위해 사용된다.
- [0192] 14. 실시예 10 내지 13 중 어느 하나의 프로세서에 있어서, 짧은 기술자 변환 테이블 포맷을 사용할 때, 액세스 플래그의 하드웨어 관리는: SCTLR.AFE 비트가 셋팅되는 경우, 액세스 플래그의 사용을 이네이블하도록; 그리고 SCTLR.HA 비트가 셋팅되는 경우, 액세스 플래그의 하드웨어 관리를 이네이블하도록 수행된다.
- [0193] 15. 실시예 14의 프로세서에 있어서, SCTLR의 बैंकिंग은 SCTLR.AFE 및 SCTLR.HA 비트들이 안전 및 비-안전 어드레

스 변환들에 독립하게 정의된다는 것을 나타낸다.

- [0194] 16. 실시예 10 내지 15 중 어느 하나의 프로세서에 있어서, 액세스 플래그의 하드웨어 관리는 어드레스 변환의 스테이지를 위해 이네이블되고, 어떤 액세스 플래그 결과도 대응하는 변환들을 위해 발생되지 않는다.
- [0195] 17. 실시예 1 내지 16 중 어느 하나의 프로세서에 있어서, IOMMU는: 분산적 가상 메모리(DVM) 실효들을 수행하기 위한 메커니즘을 더 포함한다.
- [0196] 18. 실시예 17의 프로세서에 있어서, DVM 실효 메커니즘은 호스트 ISOC 포스트된 인터페이스를 사용하는 것을 포함한다.
- [0197] 19. 실시예 18의 프로세서에 있어서, ISOC 포스트된 기록 메시지가 디코딩되고 명령 프로세서 요청들과 유사한 실효 포맷으로 변경된다.
- [0198] 20. 실시예 17 내지 19 중 어느 하나의 프로세서에 있어서, 실효 메시지는 호스트 ISOC 포스트된 기록 채널 상의 중앙 처리 장치(CPU)로부터 IOMMU 레벨 2로의 실효 메시지를 포함한다.
- [0199] 21. 실시예 20의 프로세서에 있어서, 실효 메시지는 입력/출력 TLB들을 무효화하기 위해 사용된다.
- [0200] 22. 실시예 17 내지 19 중 어느 하나의 프로세서에 있어서, 실효 메시지는 호스트 ISOC 포스트된 기록 채널 상의 중앙 처리 장치(CPU)로부터 IOMMU 레벨 2로의 동기화 메시지를 포함한다.
- [0201] 23. 실시예 22의 프로세서에 있어서, 동기화 메시지는 완료 대기 명령과 유사하다.
- [0202] 24. 실시예 22 또는 23의 프로세서에 있어서, IOMMU 레벨 2는 완료 메시지를 리턴한다.
- [0203] 25. 실시예 17 내지 19 중 어느 하나의 프로세서에 있어서, 실효 메시지는 IOMMU 레벨 2로부터 ORB 또는 유니파이드 노스브리지로의 직접 메모리 액세스(DMA) ISOC 포스트된 기록인 완료 메시지를 포함한다.
- [0204] 26. 실시예 25의 프로세서에 있어서, 완료 메시지는 명령 파이프라인이 플러싱(flushing)되고 모든 미처리 입력/출력 TLB 실효가 완료되었을 때 송신된다.
- [0205] 27. 실시예 17 내지 26 중 어느 하나의 프로세서에 있어서, DVM 동작들은 CPU, IOMMU, 및 ATC에서의 엔트리들을 무효화하기 위해 사용된다.
- [0206] 28. 실시예 17 내지 27 중 어느 하나의 프로세서에 있어서, IOMMU 레벨 2는 DVM 무효화 명령을 수신 시 그래픽 처리 장치(GPU) ATC로 무효화 명령들을 발생시킨다.
- [0207] 29. 실시예 28의 프로세서에 있어서, IOMMU 레벨 2는 메시지 유형에 기초하여 DVM 무효화 명령을 프로세싱한다.
- [0208] 30. 실시예 28 또는 29의 프로세서에 있어서, IOMMU 레벨 2는 GPU ATC로 관련 동작들을 포워딩한다.
- [0209] 31. 실시예 30의 프로세서에 있어서, IOMMU 레벨 2는 GPU에 적용가능하지 않은 동작을 제거하기 위해 동작들을 필터링한다.
- [0210] 32. 실시예 28 내지 31 중 어느 하나의 프로세서에 있어서, IOMMU는 DomainID 및 PASID의 GPU 사용량을 추적하도록 더 구성된다.
- [0211] 33. 실시예 17 내지 32 중 어느 하나의 프로세서에 있어서, IOMMU는 DVM 실효들 동안 사용하기 위한 하나 이상의 레지스터를 더 포함한다.
- [0212] 34. 실시예 33의 프로세서에 있어서, 하나 이상의 레지스터는 내부 그래픽에 대해 유효한 ASID들을 확인하기 위해 사용되는 컨텍스트 셰도잉 레지스터(context shadowing register)를 포함한다.
- [0213] 35. 실시예 33 또는 34의 프로세서에 있어서, 하나 이상의 레지스터는 내부 그래픽 디바이스의 REQID를 포함하는 레지스터를 포함한다.
- [0214] 36. 실시예 33 내지 35 중 어느 하나의 프로세서에 있어서, 하나 이상의 레지스터는 queueID마다 최대 다수의 미처리 ATS 실효를 포함하는 레지스터를 포함한다.
- [0215] 37. 실시예 33 내지 36 중 어느 하나의 프로세서에 있어서, 하나 이상의 레지스터는 ATS TLB 실효들의 queueID를 포함하는 레지스터를 포함한다.
- [0216] 38. 실시예 33 내지 37 중 어느 하나의 프로세서에 있어서, 하나 이상의 레지스터는 실효가 내부의 그래픽 디바

이스 도메인을 타겟팅한다는 것을 확인하기 위해 사용되는, 내부 그래픽 디바이스의 도메인 ID를 포함하는 레지스터를 포함한다.

- [0217] 39. 실시예 33 내지 38 중 어느 하나의 프로세서에 있어서, 레지스터들로부터의 값들이 ATS 명령으로 삽입된다.
- [0218] 40. 실시예 17 내지 39 중 어느 하나의 프로세서에 있어서, IOMMU는 16개의 PASIDMAP 레지스터를 더 포함한다.
- [0219] 41. 실시예 40의 프로세서에 있어서, PASIDMAP 레지스터들은 SRBM 인터페이스를 통해 액세스가능하다.
- [0220] 42. 실시예 40 또는 41의 프로세서에 있어서, PASIDMAP 레지스터에 기록될 때, 동일한 데이터가 또한 IOMMU에 기록된다.
- [0221] 43. 실시예 42의 프로세서에 있어서, IOMMU에의 액세스가 관독을 수행함으로써 플러싱된다.
- [0222] 44. 실시예 40 내지 43 중 어느 하나의 프로세서에 있어서, IOMMU의 PASID 유효 레지스터가 PASID가 더 이상 요구되지 않을 때 클리어된다.
- [0223] 45. 실시예 44의 프로세서에 있어서, ATC가 VMID가 재할당되기 바로 전에 VMID에 의해 무효화된다.
- [0224] 46. 노스브리지를 포함하는 프로세서로서, 노스브리지는 분산적 가상 메모리(DVM) 메시지를 핸들링하도록 구성되는 회로를 포함한다.
- [0225] 47. 실시예 46의 프로세서에 있어서, 노스브리지는 변환 색인 버퍼(TLB) 엔트리 실패를 수행하도록 구성되는 회로를 더 포함한다.
- [0226] 48. 실시예 46 또는 47의 프로세서에 있어서, DVM 메시지는 입력/출력 제어(IOC) 블록으로 송신된다.
- [0227] 49. 실시예 46 내지 48 중 어느 하나의 프로세서에 있어서, DVM 동작들이 개스킷으로부터 WrDword로서 송신되며, 여기서 개스킷은 유니파이드 노스브리지(UNB) 및 프로세서 간 통신하는 디바이스를 포함한다.
- [0228] 50. 실시예 49의 프로세서에 있어서, WrDword는 128 비트 데이터 페이로드를 포함한다.
- [0229] 51. 실시예 46 내지 50 중 어느 하나의 프로세서에 있어서, 상이한 어드레스들이 상이한 유형들의 DVM 동작들을 위해 사용된다.
- [0230] 52. 실시예 46 내지 51 중 어느 하나의 프로세서에 있어서, WaitSpuSecureBootInit가 제로와 동등할 때까지 프로세서에의 토큰 릴리즈 기능을 정지하도록 구성되는 회로를 더 포함한다.
- [0231] 53. 실시예 52의 프로세서에 있어서, 프로세서는 DramValid 또는 HtInitStatusValid 비트들에 대해 폴링하지 않는다.
- [0232] 54. 실시예 52 또는 53의 프로세서에 있어서, 프로세서가 안전 부팅 모드에 있는 경우, 프로세스 스택 포인터(PSP)는 모든 노스브리지 레지스터를 셋 업하고 동적 랜덤 액세스 메모리(DRAM)를 트레이닝한다.
- [0233] 55. 실시예 46 내지 54 중 어느 하나의 프로세서에 있어서, DVM 메시지는 실패 또는 SyncOp 메시지들을 포함한다.
- [0234] 56. 실시예 55의 프로세서에 있어서, SMMU 엔트리에서의 DVM 메시지 결과가 무효화된다.
- [0235] 57. 실시예 55 또는 56의 프로세서에 있어서, DVM 메시지들은 완료 메시지가 송신되기 전 페이지에 대한 미처리 요청들을 완료해야 한다 .
- [0236] 58. 실시예 55 내지 57 중 어느 하나의 프로세서에 있어서, 디바이스로부터 UNB로 등시성 포스트된 크레딧들을 릴리즈하도록 구성되는 회로를 더 포함한다.
- [0237] 59. 실시예 55 내지 58 중 어느 하나의 프로세서에 있어서, 디바이스로부터 UNB로 등시성 포스트된 기록들을 수용하도록 구성되는 회로를 더 포함한다.
- [0238] 60. 실시예 55 내지 59 중 어느 하나의 프로세서에 있어서, 토큰들을 새로운 레지스터에서의 새로운 채널에 할당함으로써 이네이블되는 새로운 채널을 더 포함한다.
- [0239] 61. 실시예 60의 프로세서에 있어서, 새로운 채널은 DVM Op 및 DVM 동기화 요청들에 의해 사용된다.
- [0240] 62. 실시예 46 내지 61 중 어느 하나의 프로세서에 있어서, 전역적 타임 스탬프 카운터를 더 포함한다.

- [0241] 63. 실시예 62의 프로세서에 있어서, 전역적 타임 스탬프 카운터는 큐를 통해 프로세서 인터페이스에 제공된다.
- [0242] 64. 실시예 63의 프로세서에 있어서, 큐는 선입선출 큐이다.
- [0243] 65. 실시예 46 내지 64 중 어느 하나의 프로세서에 있어서, 세마포어 동작들에의 원자 액세스를 지지하기 위해, 어드레스에의 배타적 액세스를 제공하도록 구성되는 회로를 포함한다.
- [0244] 66. 실시예 65의 프로세서에 있어서, 회로는 배타적 로드가 MCT에 보여질 때 활성화된다.
- [0245] 67. 실시예 46 내지 66 중 어느 하나의 프로세서에 있어서, 프로세서로부터의 VicBlk 요청들에 대한 하드 토큰(hard token)을 더 포함한다.
- [0246] 68. 실시예 67의 프로세서에 있어서, 프로세서는 기록 데이터 채널 상의 데이터를 개스킷으로 푸시하도록 더 구성된다.
- [0247] 69. 실시예 68의 프로세서에 있어서, 데이터는 노스브릿지가 개스킷으로부터 데이터를 풀링하고 개스킷이 타겟 완료 신호를 수신한 후 기록 데이터 채널 상에 푸시된다.
- [0248] 70. 실시예 69의 프로세서에 있어서, 타겟 완료 신호가 기록 버퍼 릴리즈 신호에 의해 전달된다.
- [0249] 71. 실시예 67 내지 70 중 어느 하나의 프로세서에 있어서, 프로세서가 라이트백을 발행하는 경우, 개스킷 및 SRI은 라이트백을 메모리로 전달한다.
- [0250] 72. 실시예 71의 프로세서에 있어서, 프로세서가 라이트백이 완료되기 기다리고 있는 동안 교착 상태가 존재한다.
- [0251] 73. 실시예 67 내지 72 중 어느 하나의 프로세서에 있어서, 프로세서로부터 MCT로의 가상 채널을 더 포함하고, MCT는 라이트백들이 보다 오래된 요청들을 전달하게 한다.
- [0252] 74. 실시예 67 내지 73 중 어느 하나의 프로세서에 있어서, 추가적인 라이트백 채널 및 추가적인 라이트백 채널을 위한 전용 버퍼들을 더 포함한다.
- [0253] 75. 실시예 74의 프로세서에 있어서, 프로세서로부터 빅탐/라이트백에 대한 SRQ에 대한 전용 하드 할당 토큰을 더 포함한다.
- [0254] 76. 실시예 75의 프로세서에 있어서, SRQ는 항상 빅탐에 대해 이용가능한 토큰을 가질 것이다.
- [0255] 77. 실시예 46 내지 76 중 어느 하나의 프로세서에 있어서, MCT가 호스트 브리지 관독에 대한 프로브 타겟을 발생하기 위한 회로를 더 포함한다.
- [0256] 78. 실시예 77의 프로세서에 있어서, 프로브 타겟을 발행하는 것은 MCT가 프로브 응답들, DRAM 관독 데이터, 및 가능한 빅탐 패싱 케이스들을 통합하게, 그리고 SRI로 다시 단일 관독 응답을 발행하게 한다.
- [0257] 79. 실시예 77 또는 78의 프로세서에 있어서, MCT 요청들에 대한 모든 XBAR은 프로브 더티 데이터 또는 보다 신중한 패싱 빅탐으로부터의 데이터를 수용하기 위해 MCD 엔트리를 할당한다.
- [0258] 80. 실시예 79의 프로세서에 있어서, 예비 할당된 McdLo 및 McdHi 토큰들을 더 포함한다.
- [0259] 81. 실시예 46 내지 80 중 어느 하나의 프로세서에 있어서, 호스트 브리지 관독들을 수행하도록 구성되는 회로를 더 포함한다.
- [0260] 82. 실시예 81의 프로세서에 있어서, 호스트 브리지 관독은 MCD 엔트리를 할당하는 것 및 McaDatPtr을 유효하게 셋팅하는 것을 포함한다.
- [0261] 83. 실시예 82의 프로세서에 있어서, 호스트 브리지 관독은 MCD 바이트 마스크들을 클리어하는 것을 더 포함한다.
- [0262] 84. 실시예 83의 프로세서에 있어서, 호스트 브리지 관독은 호스트 브리지 요청들을 위해 포워딩하는 데이터를 디세이블하는 것을 더 포함한다.
- [0263] 85. 실시예 84의 프로세서에 있어서, 호스트 브리지 관독은 프로브 타겟을 발생하는 것을 더 포함한다.
- [0264] 86. 실시예 85의 프로세서에 있어서, 호스트 브리지 관독은 호스트 브리지 RdSz 요청을 통해 MCQRdRspWaitSys 비트를 셋팅하는 것을 더 포함한다.

- [0265] 87. 실시예 86의 프로세서에 있어서, 호스트 브리지 판독은: 프로세서 프로브가 계류 중이거나 프로브가 더티 데이터를 갖고 리턴하면서 DCT 판독 응답이 도착할 때 대기 비트로서 MCQWaitG5CanDet를 오버로딩하는 것을 더 포함한다.
- [0266] 88. 실시예 87의 프로세서에 있어서, 호스트 브리지 판독은 MPDB로의 DCT 데이터 기록이 완료되면 XBAR rd 응답을 송신하는 것을 더 포함한다.
- [0267] 89. 실시예 87 또는 88의 프로세서에 있어서, 호스트 브리지 판독은 MCQRdRspWaitSys를 클리어하는 것 및 후속 XBAR RdRsp 픽(pick)이 프로브 손실 시 발생하게 하는 것을 더 포함한다.
- [0268] 90. 실시예 87 내지 89 중 어느 하나의 프로세서에 있어서, 호스트 브리지 판독은 프로세서 프로브 응답이 손실된 후 DCT 응답이 도착한 경우 XBAR 판독 응답을 발생하는 것을 더 포함한다.
- [0269] 91. 실시예 46 내지 90 중 어느 하나의 프로세서에 있어서, SMM 유형 기능이 안전 메모리 영역을 사용하게 하기 위한 두 안전 메모리 레인지 레지스터를 더 포함한다.
- [0270] 92. 실시예 91의 프로세서에 있어서, 안전 메모리 영역에의 액세스는 PSP 외 모든 디바이스로부터 디제이블된다.
- [0271] 93. 실시예 91 또는 92의 프로세서에 있어서, 프로세서는 단지 안전 모드에 있을 때 안전 메모리 영역에 액세스할 수 있다.
- [0272] 94. 실시예 91 내지 93 중 어느 하나의 프로세서에 있어서, 안전 메모리 영역에의 판독/기록 액세스는 레지스터에서의 판독 이네이블(RE)/기록 이네이블(WE) 비트들을 통해 한정될 수 있다.
- [0273] 95. 실시예 91 내지 94 중 어느 하나의 프로세서에 있어서, 프로세서가 안전 메모리 영역에 액세스할 수 있는지를 결정하기 위한 요청을 갖는 NS 비트를 송신하는 것을 더 포함한다.
- [0274] 96. 실시예 46 내지 96 중 어느 하나의 프로세서에 있어서, coreSight 액세스들이 이들 액세스가 DRAM에 통상적 기록과 같이 보이게 함으로써 개스킷에 의해 핸들링된다.
- [0275] 97. 실시예 46 내지 96 중 어느 하나의 프로세서에 있어서, GNB 및 NB 간 인터럽트 와이어들을 더 포함한다.
- [0276] 98. 실시예 97의 프로세서에 있어서, 인터럽트 와이어들은 레벨 인터럽트들이다.
- [0277] 99. 실시예 46 내지 98 중 어느 하나의 프로세서에 있어서, 무손실의 CCI 전송을 보장하기 위해 PLL 재 로크를 개시하기 전 프로세서-노스브리지 인터페이스를 작업 거부하도록 구성되는 회로를 더 포함한다.
- [0278] 100. 실시예 99의 프로세서에 있어서, 작업 거부는 StpClk/StartClk 핸드셰이크 메커니즘을 사용하는 것을 포함한다.
- [0279] 101. 실시예 99 또는 100의 프로세서에 있어서, 노스브리지 및 개스킷 간 사이드밴드들을 생성하는 것을 더 포함한다.
- [0280] 102. 실시예 100 또는 101의 프로세서에 있어서, 작업 거부는 StpClk 핸들러에서 WFE를 사용하는 것을 포함한다.
- [0281] 103. 실시예 46 내지 102 중 어느 하나의 프로세서에 있어서, XC6에 대해 인터럽트 레이트 모니터링 지지를 추가하는 것을 더 포함한다.
- [0282] 104. 실시예 103의 프로세서에 있어서, XC6 엔트리 결정을 대면하면서 감소 레이트를 늦추는 것을 더 포함한다.
- [0283] 105. 실시예 46 내지 104 중 어느 하나의 프로세서에 있어서, L2 손실들 및 Bus_Accesses를 추적하기 위해, 추가적인 카운터들을 더 포함한다.
- [0284] 106. 실시예 46 내지 105 중 어느 하나의 프로세서에 있어서, 라이트백들 및 프로브들 간 교착 상태를 해결하기 위해, 라이트백을 위한 가상 채널을 더 포함한다.
- [0285] 107. 실시예 106의 프로세서에 있어서, 라이트백들은 이들이 MCT에 이르면 보다 오래된 판독들에 앞서 재순서화될 수 있다.
- [0286] 108. 실시예 107의 프로세서에 있어서, 재순서화는 이전 DRAM 판독들을 재발행하는 것을 포함한다.

- [0287] 109. 실시예 106 내지 108 중 어느 하나의 프로세서에 있어서, 빅탐이 도착할 때, 보다 오래된 동작들과의 어드레스 매치를 위해 빅탐을 검사하는 것을 더 포함한다.
- [0288] 110. 실시예 109의 프로세서에 있어서, 어드레스 매치가 발생하는 경우, 매치가 프로브 응답을 기다리고 있는 관독에 대한 것인지 또는 프로브 응답을 기다리고 있는 기록에 대한 것인지를 검사한다.
- [0289] 111. 실시예 109 또는 110의 프로세서에 있어서, 다수의 어드레스 매치가 검출되는 경우, 가장 오래된 어드레스 매치가 선택된다.
- [0290] 112. 실시예 109 내지 111 중 어느 하나의 프로세서에 있어서, 빅탐이 응답을 기다리고 있는 기록에 매치하는 경우, 빅탐은 프로브 응답으로서 처리되고 이전 기록의 MCD 엔트리로 병합된다.
- [0291] 113. 실시예 112의 프로세서에 있어서, 빅탐에 할당되었던 MCD 엔트리를 릴리즈하는 것을 더 포함한다.
- [0292] 114. 실시예 109 내지 111 중 어느 하나의 프로세서에 있어서, 빅탐이 응답을 기다리고 있는 기록에 매치하는 경우, 빅탐은 관독의 MCD 엔트리로 병합되고 이 자체의 MCD 엔트리에 기록된다.
- [0293] 115. 실시예 114의 프로세서에 있어서, 빅탐은 제2 MCD 기록 포트를 통해 이 자체의 MCD 엔트리로 기록된다.
- [0294] 116. 실시예 46 내지 115 중 어느 하나의 프로세서에 있어서, 비가시 프레임 버퍼(IVFB; invisible frame buffer)를 더 포함한다.
- [0295] 117. 실시예 116의 프로세서에 있어서, IVFB는 PSP에 의해 셋 업되고 임의의 크기로 셋팅될 수 있다.
- [0296] 118. 실시예 116 또는 117의 프로세서에 있어서, IVFB에의 관독 액세스가 한정된다.
- [0297] 119. 실시예 116 내지 118 중 어느 하나의 프로세서에 있어서, 임의의 호스트가 IVFB에 기록할 수 있다.
- [0298] 120. 실시예 46 내지 119 중 어느 하나의 프로세서에 있어서, 송신될 fdf7 안전 예외 백터를 언제 강제할지를 결정하기 위해 SecureEnable에 대한 CCI에 걸친 신호를 더 포함한다.
- [0299] 121. 실시예 46 내지 120 중 어느 하나의 프로세서에 있어서, 개스킷은 SMN 인터페이스 상에 구성 레지스터들을 구현하도록 구성된다.
- [0300] 122. 실시예 121의 프로세서에 있어서, 상태는 이의 XC6 캐시/상태 플러시 루틴 동안 프로세서에 의해 저장된다.
- [0301] 123. 실시예 122의 프로세서에 있어서, 상태는 C6 리셋 핸들러에 의해 복원된다.
- [0302] 124. 실시예 46 내지 123 중 어느 하나의 프로세서에 있어서, 프로세서는 랩 순서(wrap order)로서 공지된 버스트 순서를 지지하고, 랩 순서는 데이터가 송신되거나 수신될 것으로 예상되는 순서이다.
- [0303] 125. 실시예 124의 프로세서에 있어서, MPBD 또는 SRD로부터 리턴되는 데이터가 랩 순서로 리턴된다.
- [0304] 126. 실시예 124 또는 125의 프로세서에 있어서, MPBD로부터 랩 순서로 관독된 데이터는 SRD로 예상 버스트 순서로 기록될 수 있다.
- [0305] 127. 실시예 126의 프로세서에 있어서, SRD는 데이터를 프로세서로 리턴하기 위해 랩 순서로 관독될 것이다.
- [0306] 128. 실시예 46 내지 127 중 어느 하나의 프로세서에 있어서, CC6 영역이 구성가능하다.
- [0307] 129. 실시예 128의 프로세서에 있어서, 비트는 CC6 영역의 재구성가능성을 나타내기 위해 안전 부팅 레지스터에 셋팅된다.
- [0308] 130. 실시예 129의 프로세서에 있어서, 비트는 안전 부팅 프로세스 동안 PSP에 의해 기록된다.
- [0309] 131. 실시예 46 내지 130 중 어느 하나의 프로세서에 있어서, 유니파이드 노스브리지(UNB) 및 프로세서 간 통신하는, 개스킷을 더 포함한다.
- [0310] 132. 실시예 131의 프로세서에 있어서, 개스킷은 프로세서 클록 도메인에 상주한다.
- [0311] 133. 실시예 131 또는 132의 프로세서에 있어서, 개스킷은 트랜잭션들을 캐시 코히런트 인터커넥트(CCI) 트랜잭션들로 변환하도록 구성된다.
- [0312] 134. 실시예 131 내지 133 중 어느 하나의 프로세서에 있어서, 개스킷dms 시스템 프로브들을 스눕들로 변환하도

록 구성된다.

- [0313] 135. 실시예 131 내지 134 중 어느 하나의 프로세서에 있어서, 개스킷은 메모리 맵을 유니파이드 노스브리지 (UNB)/코히런트 하이퍼트랜스포트(cHT) 환경과 상호-작동하도록 변환하도록 구성된다.
- [0314] 136. 실시예 131 내지 135 중 어느 하나의 프로세서에 있어서, 개스킷은 전송 크기들을 UNB/cHT 환경과 상호-작동하도록 변환하도록 구성된다.
- [0315] 137. 실시예 131 내지 136 중 어느 하나의 프로세서에 있어서, 개스킷은 프로세서로부터의 요청들을 어드밴스드 익스텐시블 인터페이스(AXI) 코히런시 익스텐션(ACE) 명령들로부터 동등한 CCI 명령들로 변환하도록 구성된다.
- [0316] 138. 실시예 137의 프로세서에 있어서, 프로세서로부터의 각 요청은 특정한 CCI 요청 유형과 매핑한다.
- [0317] 139. 실시예 137 또는 138의 프로세서에 있어서, 프로세서로부터의 요청의 유형은 판독 및 기록 어드레스 채널들의 AxDomain 및 AxSnoop 필드들에 인코딩된다.
- [0318] 140. 실시예 137 내지 139 중 어느 하나의 프로세서에 있어서, 버스 상의 비트들이 ACE 속성들을 캐리하기 위해 오버로딩될 수 있다.
- [0319] 141. 실시예 137 내지 140 중 어느 하나의 프로세서에 있어서, 개스킷은 ACE 태그들을 CCI 태그들로 변환하도록 구성된다.
- [0320] 142. 실시예 141의 프로세서에 있어서, 동일한 태그를 공유하는 ACE 요청들은 개스킷 내에 직렬화된다.
- [0321] 143. 실시예 137 내지 142 중 어느 하나의 프로세서에 있어서, 개스킷은 SRI로부터의 CCI 인코딩된 프로브들을 동등한 ACE 스냅 트랜잭션들로 변환하도록 구성된다.
- [0322] 144. 실시예 137 내지 143 중 어느 하나의 프로세서에 있어서, 개스킷이 CCI 인터페이스로부터 판독 응답을 수신할 때, 그것은 클러스터로의 ACE 버스 상에 동등한 응답을 발생시킨다.
- [0323] 145. 실시예 144의 프로세서에 있어서, 개스킷은 데이터의 모든 비트(beat)가 CCI 인터페이스로부터 수신될 때까지 ACE 응답을 시작하는 것을 지연시키도록 구성된다.
- [0324] 146. 실시예 137 내지 145 중 어느 하나의 프로세서에 있어서, 개스킷은 각 요청을 갖고 SRI로 송신될 예상 보조 어드레스 유형 신호들을 생성하기 위해 메모리 맵을 사용하도록 구성된다.
- [0325] 147. 실시예 146의 프로세서에 있어서, 개스킷은 트랜잭션들을 cHT로 변환하기 위해 메모리 맵을 사용하도록 구성된다.
- [0326] 148. 실시예 46 내지 147 중 어느 하나의 프로세서에 있어서, 라이트백에 의해 액세스되는 독립적인 가상 채널을 더 포함하고, 라이트백은 동적 랜덤 액세스 메모리(DRAM)로 드레인할 수 있다.
- [0327] 149. 실시예 148의 프로세서에 있어서, 라이트백이 개스킷을 횡단할 비-블로킹 경로를 더 포함한다.
- [0328] 150. 실시예 149의 프로세서에 있어서, 라이트백이 SRQ를 횡단할 비-블로킹 경로를 더 포함한다.
- [0329] 151. 실시예 150의 프로세서에 있어서, 라이트백이 크로스-바를 거칠 비-블로킹 경로를 더 포함한다.
- [0330] 152. 실시예 148 내지 151 중 어느 하나의 프로세서에 있어서, MCT에 도착할 때, 라이트백은 다른 요청들을 전달한다.
- [0331] 153. 실시예 152의 프로세서에 있어서, 가장 오래된 판독이 라이트백에 종속된 것으로 표시된다.
- [0332] 154. 실시예 148 내지 153 중 어느 하나의 프로세서에 있어서, 라이트백은 비-라이트백 기록과 인터리빙되지 않는다.
- [0333] 155. 실시예 131 내지 154 중 어느 하나의 프로세서에 있어서, 개스킷은 동기화 장치로부터 신호들을 유도 및 수신하기 위해 스킵 클록 로직을 관찰하도록 구성된다.
- [0334] 156. 실시예 155의 프로세서에 있어서, 개스킷은 기록 명령이 다섯 개의 클록 분리를 따른다는 것을 보장하기 위해 예측된 선입선출(FIFO) 큐를 사용하도록 구성된다.
- [0335] 157. 실시예 155 또는 156의 프로세서에 있어서, 개스킷은 ACE 타이밍으로 응답들을 프로세서에 포워딩하도록 구성된다.

- [0336] 158. 실시예 155 내지 157 중 어느 하나의 프로세서에 있어서, 동기화 장치는 동기화 장치 FIFO가 이후에 판독/기록하는 10개의 클록에 대해 이네이블될지를 나타내기 위한 신호를 개스킷에 제공한다.
- [0337] 159. 실시예 137 내지 158 중 어느 하나의 프로세서에 있어서, 개스킷은 ACE 기록을 다수의 CCI 기록으로 분할하도록 구성된다.
- [0338] 160. 실시예 159의 프로세서에 있어서, 요청 큐 및/또는 기록 버퍼는 전송될 필요가 있는 특정한 바이트들을 추적하고; 적절한 명령들을 발생시키도록 구성된다.
- [0339] 161. 실시예 159 또는 160의 프로세서에 있어서, 엔트리는 모든 다수의 기록이 완료되고 릴리즈되었을 때 할당 해제된다.
- [0340] 162. 실시예 137 내지 161 중 어느 하나의 프로세서에 있어서, 기록들은 캐시 라인을 채우기 위해 조합된다.
- [0341] 163. 실시예 162의 프로세서에 있어서, 개스킷은 다수의 기록 요청 엔트리보다 적은 기록 버퍼를 포함한다.
- [0342] 164. 실시예 162 또는 163의 프로세서에 있어서, 기록 데이터는 그것이 SRI에 의해 버퍼로부터 풀링될 때까지 유지된다.
- [0343] 165. 실시예 162 내지 164 중 어느 하나의 프로세서에 있어서, 기록 요청은 응답이 프로세서로 송신될 수 있을 때까지 개방된 채로 유지된다.
- [0344] 166. 실시예 165의 프로세서에 있어서, 응답은 타겟 완료 신호에 기반하지 않는, SRI 버퍼 릴리즈 이후 프로세서로 송신될 수 있다.
- [0345] 167. 실시예 162 내지 166 중 어느 하나의 프로세서에 있어서, 개스킷은 버퍼를 플러싱하도록 더 구성된다.
- [0346] 168. 실시예 167의 프로세서에 있어서, 버퍼는 버퍼 플러시 트리거에 기초하여 플러싱된다.
- [0347] 169. 실시예 168의 프로세서에 있어서, 버퍼 플러시 트리거는 버퍼가 완전히 유효한 임의의 시간이다.
- [0348] 170. 실시예 168의 프로세서에 있어서, 버퍼 플러시 트리거는 송신되지 않은 버퍼들이 고갈되는 임의의 시간이다.
- [0349] 171. 실시예 168의 프로세서에 있어서, 버퍼 플러시 트리거는 프로세서 기록이 프로세서로 이미 개방된 버퍼를 히트하지 않는 임의의 시간이다.
- [0350] 172. 실시예 168의 프로세서에 있어서, 버퍼 플러시 트리거는 다수의 개방된 버퍼의 프로그래밍된 제한이 도달되는 임의의 시간이다.
- [0351] 173. 실시예 168의 프로세서에 있어서, 버퍼 플러시 트리거는 버퍼가 더 이상 기록되지 않고 있다는 표시이다.
- [0352] 174. 실시예 167 내지 173 중 어느 하나의 프로세서에 있어서, 단지 정확한 캐싱가능 속성들을 갖는 기록들이 조합될 수 있다.
- [0353] 175. 실시예 167 내지 174 중 어느 하나의 프로세서에 있어서, 배리어들을 위한 모든 버퍼가 플러싱된다.
- [0354] 176. 실시예 167 내지 175 중 어느 하나의 프로세서에 있어서, 모든 버퍼가 DvmSync를 위해 플러싱된다.
- [0355] 177. 실시예 137 내지 176 중 어느 하나의 프로세서에 있어서, 개스킷은 트랜잭션들의 순서가 유지되는, 배리어 트랜잭션을 수행하도록 더 구성된다.
- [0356] 178. 실시예 177의 프로세서에 있어서, 배리어 트랜잭션은 메모리 배리어를 포함한다.
- [0357] 179. 실시예 178의 프로세서에 있어서, 메모리 배리어는 마스터 구성요소가 배리어 이후 임의의 트랜잭션 발행을 관찰할 수 있는 경우 구성요소가 배리어 이전 발행된 모든 트랜잭션을 관찰할 수 있다는 것을 보장한다.
- [0358] 180. 실시예 177의 프로세서에 있어서, 배리어 트랜잭션은 동기화 배리어를 포함한다.
- [0359] 181. 실시예 180의 프로세서에 있어서, 동기화 배리어는 배리어가 완료될 때 적절한 도메인에서 모든 마스터 구성요소에 의해 배리어가 관찰가능하기 전 모든 트랜잭션이 발행했다는 것을 보장하기 위해 마스터 구성요소에 의해 발행된다.
- [0360] 182. 실시예 180 또는 181의 프로세서에 있어서, 동기화 배리어는 측-파대 시그널링 통신을 갖고 사용된다.

- [0361] 183. 실시예 177 내지 182 중 어느 하나의 프로세서에 있어서, 배리어 트랜잭션은 프로세서에 완료를 표시하기 전 모든 이전 요청이 개스킷에 의해 SRI로 발행되었다는 것을 보장함으로써 개스킷에 의해 핸들링된다.
- [0362] 184. 실시예 183의 프로세서에 있어서, 개스킷은 모든 보다 오래된 요청이 발행되었을 때 배리어 트랜잭션에 대한 응답을 프로세서로 송신한다.
- [0363] 185. 실시예 131 내지 184 중 어느 하나의 프로세서에 있어서, 개스킷은 제네릭 인터럽트 컨트롤러(GIC)를 포함한다.
- [0364] 186. 실시예 185의 프로세서에 있어서, GIC는 프로세서 인터페이스, 분배기 및 인터럽트 변환 서비스(ITS; interrupt translation service)를 포함한다.
- [0365] 187. 실시예 186의 프로세서에 있어서, 분배기는 로컬 SRI에 의해 수신될 MSI 패킷들을 통해 소스들로부터 인터럽트들을 수신하도록 구성된다.
- [0366] 188. 실시예 1 내지 187 중 어느 하나의 프로세서에 있어서, 프로세서는 페이지 테이블 업데이트들을 IOMMU로 전달하기 위해 분산적 가상 메모리(DVM) 트랜잭션들을 발행하도록 구성된다.
- [0367] 189. 실시예 188의 프로세서에 있어서, DVM 트랜잭션들은 포스트된 기록들로 변환된다.
- [0368] 190. 실시예 188 또는 189의 프로세서에 있어서, DVM 명령은 DVM 베이스 어드레스로부터의 오프셋인, 어드레스의 부분에 의해 확인된다.
- [0369] 191. 실시예 188 내지 190 중 어느 하나의 프로세서에 있어서, 프로세서는 하나 이상의 DVM 동작(DvmOp)의 시퀀스를 관독 채널 상에 발행한다.
- [0370] 192. 실시예 191의 프로세서에 있어서, 개스킷은 각 DvmOp를 DWORD 기록 동작으로 변환하도록 구성된다.
- [0371] 193. 실시예 192의 프로세서에 있어서, 기록 데이터는 DvmOp로부터의 필요한 정보를 포함한다.
- [0372] 194. 실시예 188 내지 193 중 어느 하나의 프로세서에 있어서, 프로세서는 임의의 이전 DvmOp들을 푸시하고 이들이 완료되었다는 것을 보장하는, DvmSync 동작을 관독 채널 상에 발행하도록 구성된다.
- [0373] 195. 실시예 194의 프로세서에 있어서, DvmSync 동작을 수신하는 각 에이전트는 그것이 모든 계류 중인 DvmOp 및 DvmSync를 완료했을 때 이 자체의 DvmCompletion 동작을 발행한다.
- [0374] 196. 실시예 195의 프로세서에 있어서, IOMMU는 임의의 이전 동작들을 푸시하는, 호스트 브리지로 FENCE/FLUSH 명령을 발행한다.
- [0375] 197. 실시예 196의 프로세서에 있어서, IOMMU는 등시성 포스트된 BYTE 기록을 통해 모든 DvmOp and the DvmSync의 완료를 시그널링한다 .
- [0376] 198. 실시예 194 내지 197 중 어느 하나의 프로세서에 있어서, DvmSync op들은 순차적으로 프로세싱될 수 있다.
- [0377] 199. 실시예 198의 프로세서에 있어서, 개스킷은 DvmSync op들을 순차적으로 프로세싱하도록 구성된다.
- [0378] 200. 실시예 199의 프로세서에 있어서, 개스킷은 다편의 DvmOp의 부분들을 조합할 수 있기 위해, 프로세서마다 하나의 DVM 버퍼를 포함한다.
- [0379] 201. 실시예 137 내지 200 중 어느 하나의 프로세서에 있어서, 개스킷은 프로세서 관독 및 기록 어드레스 채널들을 조합하도록 더 구성된다.
- [0380] 202. 실시예 201의 프로세서에 있어서, 관독 및 기록 어드레스 채널들은 입력에서 조합되고 단일 공유된 큐가 구현된다.
- [0381] 203. 실시예 202의 프로세서에 있어서, 프로세서는 단일 공유된 큐와 연관되고, 두 입력 중 하나를 선택하기 위해 사용된다.
- [0382] 204. 실시예 201의 프로세서에 있어서, 별개 큐들이 관독 및 기록 어드레스 채널들을 위해 구현되고, 큐들이 출력에서 조합된다.
- [0383] 205. 실시예 137 내지 204 중 어느 하나의 프로세서에 있어서, DSM 후크들이 개스킷의 ACE 측을 모니터링하기 위해 제공된다.

- [0384] 206. 실시예 131 내지 205 중 어느 하나의 프로세서에 있어서, 개스킷은 구성 레지스터들을 포함한다.
- [0385] 207. 실시예 206의 프로세서에 있어서, 개스킷은 레지스터 베이스 어드레스를 포함한다.
- [0386] 208. 실시예 206 또는 207의 프로세서에 있어서, 개스킷은 실행 상태 리셋 벡터 어드레스 레지스터를 포함한다.
- [0387] 209. 실시예 206 내지 208 중 어느 하나의 프로세서에 있어서, 개스킷은 GIC 어드레스 공간 레지스터를 포함한다.
- [0388] 210. 실시예 206 내지 209 중 어느 하나의 프로세서에 있어서, 개스킷은 주변 구성요소 인터커넥트(PCI; peripheral component interconnect) 구성 베이스 어드레스 레지스터를 포함한다.
- [0389] 211. 실시예 206 내지 210 중 어느 하나의 프로세서에 있어서, 개스킷은 구성 및 제어 비트 레지스터를 포함한다.
- [0390] 212. 실시예 211의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 셋팅된 요청 큐 크기의 표시를 포함한다.
- [0391] 213. 실시예 211 또는 212의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 어떤 특별한 트랜잭션들이 직렬화되는지에 대한 표시를 포함한다.
- [0392] 214. 실시예 211 내지 213 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 어떤 대안적인 ACE 대 CCI 요청 매핑들이 이용되는지에 대한 표시를 포함한다.
- [0393] 215. 실시예 211 내지 214 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 어떤 대안적인 CCI 프로브 대 ACE 스냅 매핑들이 이용되는지에 대한 표시를 포함한다.
- [0394] 216. 실시예 211 내지 215 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 개스킷 작업 거부 디сей블 표시를 포함한다 .
- [0395] 217. 실시예 211 내지 216 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 라이트백 직렬화 이네이블 표시를 포함한다.
- [0396] 218. 실시예 211 내지 217 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 모든 다중-트랜잭션 부분적 기록에 대한 강제 바이트 기록들에 대한 표시를 포함한다.
- [0397] 219. 실시예 211 내지 218 중 어느 하나의 프로세서에 있어서, 구성 및 제어 비트 레지스터는 DvmSync 동작들의 강제 직렬화에 대한 표시를 포함한다.
- [0398] 220. 실시예 206 내지 219 중 어느 하나의 프로세서에 있어서, 개스킷은 CAC 관련 레지스터들을 포함한다.
- [0399] 221. 실시예 206 내지 220 중 어느 하나의 프로세서에 있어서, SMN 슬레이브 인터페이스는 개스킷 레지스터들에 액세스하기 위해 제공된다.
- [0400] 222. 실시예 221의 프로세서에 있어서, SMN 슬레이브 인터페이스는 리셋 동안 임의의 필요한 레지스터들에 로딩 하도록 구성된다.
- [0401] 223. 실시예 221 또는 222의 프로세서에 있어서, SMN 슬레이브 인터페이스는 CAC 관련 레지스터들에의 전력 관리 액세스를 제공하도록 구성된다.
- [0402] 224. 실시예 221 내지 213 중 어느 하나의 프로세서에 있어서, SMN 슬레이브 인터페이스는 치킨 비트들(chicken bits)에의 소프트웨어 액세스를 제공하도록 구성된다.
- [0403] 225. 실시예 221 내지 224 중 어느 하나의 프로세서에 있어서, SMN 슬레이브 인터페이스는 패키지 CC6 상태에 진입 및 이탈할 때 레지스터들의 콘텐츠를 저장 및 복원하기 위한 수단을 제공하도록 구성된다.
- [0404] 226. 실시예 131 내지 225 중 어느 하나의 프로세서에 있어서, 개스킷은 단일 토큰 풀(single token pool)을 포함하며, 여기서 토큰은 요청들을 SRI로 송신하기 위해 사용된다.
- [0405] 227. 실시예 131 내지 226 중 어느 하나의 프로세서에 있어서, 개스킷은 전력 관리에 참여하도록 구성된다.
- [0406] 228. 실시예 227의 프로세서에 있어서, 개스킷은 프로세서가 위치되는 클러스터에 의해 제공되는 다양한 이벤트를 카운팅하는 요청된 CAC 레지스터들을 구현하도록 구성된다.
- [0407] 229. 실시예 227 또는 228의 프로세서에 있어서, 개스킷은 전력 관리 상태들을 변경하기 위해 작업 거부 기능을

제공하도록 구성된다.

- [0408] 230. 실시예 229의 프로세서에 있어서, 작업 거부 기능은 추가 CCI 요청들을 블로킹하는 것을 포함한다.
- [0409] 231. 실시예 227 내지 231 중 어느 하나의 프로세서에 있어서, 개스킷은 주파수에 기초하여 가중 사이클을 저장하도록 구성된다.
- [0410] 232. 실시예 185 내지 231 중 어느 하나의 프로세서에 있어서, GIC는 전력 관리 인터페이스와 상호작용하도록 구성된다.
- [0411] 233. 실시예 232의 프로세서에 있어서, 모든 인터럽트는 마이크로아키텍처 인터럽트들을 제공하고 임의의 아키텍처상 가시적인 인터럽트들을 운영 체제(OS)로 포워딩하는, 마이크로코드에 의해 인터셉트된다.
- [0412] 234. 실시예 233의 프로세서에 있어서, 마이크로아키텍처 인터럽트들은 중단 클록(StpClk) 및 시작 클록(StartClk) 신호들을 포함한다.
- [0413] 235. 실시예 232 내지 234 중 어느 하나의 프로세서에 있어서, 인터럽트들은 원격 기록에 의해 또는 로컬 인터럽트 핀들을 통해 전달된다.
- [0414] 236. 실시예 232 내지 235 중 어느 하나의 프로세서에 있어서, GIC 인터럽트들이 공유된 주변 인터럽트(SPI; shared peripheral interrupt) 또는 개인 주변 인터럽트(PPI; private peripheral interrupt)를 포함하여 로컬 인터럽트 핀들을 통해 그리고 AXI 인터페이스를 통해 데이터 패브릭으로 전달된다.
- [0415] 237. 실시예 236의 프로세서에 있어서, 모든 노스브리지 다중-타겟 인터럽트 유형들이 PPI들로서 이용가능해진다.
- [0416] 238. 실시예 237의 프로세서에 있어서, 인터럽트 핸들러를 위한 추가적인 상태 레지스터를 더 포함한다.
- [0417] 239. 실시예 236의 프로세서에 있어서, GIC로의 로컬 SPI 입력들이 모두 자주 클록들(free running clocks) 상에 있으면, SPI 입력들을 유향 상태 종료로 재맵핑한다.
- [0418] 240. 실시예 1 내지 239 중 어느 하나의 프로세서에 있어서, 클로킹 요청이 ConnectMask 신호가 정지 상태 또는 HTC SBC 상태에 있으면서 어썬트 해제될 때 발생된다.
- [0419] 241. 실시예 240의 프로세서에 있어서, 모든 코어에 대한 프로브 발행이 블로킹된다.
- [0420] 242. 실시예 241의 프로세서에 있어서, 모든 계류 중인 프로브는 이들의 프로브 응답들 및/또는 데이터 이동들이 완료가기를 기다림으로써 임의의 영향을 받은 코어들로부터 드레인된다.
- [0421] 243. 실시예 242의 프로세서에 있어서, 프로세서는 모든 영향을 받은 코어가 그것들의 NbBusy 신호들을 드롭하기를 기다린다.
- [0422] 244. 실시예 243의 프로세서에 있어서, 프로세서는 FIFO 큐들에서의 인플라이트 트래픽에 의해 트리거되는 새로운 이벤트들이 객체를 발생시키기에 충분한 시간을 갖도록 NBSYN 선입선출(FIFO) 큐들 왕복 지연을 커버하기에 충분한 시간을 무조건적으로 기다린다.
- [0423] 245. 실시예 244의 프로세서에 있어서, 프로세서는 블로킹하고 CAC 인터페이스 상에 계류 중인 APM 요청들이 응답 또는 확인 응답되기를 기다린다.
- [0424] 246. 실시예 245의 프로세서에 있어서, 프로세서는 전력 감소에 대한 이력 현상을 충족한다.
- [0425] 247. 실시예 240 내지 246 중 어느 하나의 프로세서에 있어서, 타겟팅된 코어에 대한 모든 클록 게이트가 어썬트 해제되고, 그 후 프로브들과 연관된 클록 게이트들이 재-이네이블된다.
- [0426] 248. 실시예 240 내지 247 중 어느 하나의 프로세서에 있어서, 코어 전압은 전력 게이팅(PG) 헤더들을 조절하고 피드백 회로를 통해 해당 전압을 유지함으로써 감소된다.
- [0427] 249. 실시예 248의 프로세서에 있어서, 조절 및 유지는 플러싱과 또는 플러싱 없이 적용된다.
- [0428] 250. 실시예 248 또는 249의 프로세서에 있어서, Q-채널 핸드셰이크는 조절 및 유지를 가능하게 하기 위해 전력 컨트롤러와 사용된다.
- [0429] 251. 실시예 250의 프로세서에 있어서, Q-채널 핸드셰이크 완료 시, 저 전력 상태가 종료될 수 있다.

- [0430] 252. 실시예 250 또는 251의 프로세서에 있어서, Q-채널 핸드셰이크가 완료된 후, 코어 C-상태 커밋 지점이 있을 수 있다.
- [0431] 253. 실시예 252의 프로세서에 있어서, Q-채널 핸드셰이크는 CPU LowPwrReq 어썬션 시 개시된다.
- [0432] 254. 실시예 253의 프로세서에 있어서, VceLowPwrRdy는 코어가 커밋되었다는 것을 나타내는 CPU QACCEPT 어썬션 까지 블로킹된다.
- [0433] 255. 실시예 240 내지 254 중 어느 하나의 프로세서에 있어서, 빠른 인터럽트 요청(FIQ; fast interrupt request)/인터럽트 요청(IRQ; interrupt request) 에지가 기상 이벤트를 생성하기 위해 사용된다.
- [0434] 256. 실시예 255의 프로세서에 있어서, 기상 이벤트는 RCC3 유지 모드로부터의 종료를 트리거한다.
- [0435] 257. 실시예 256의 프로세서에 있어서, RCC3 유지 모드로부터의 종료 시, 모든 인터럽트가 전달된다.
- [0436] 258. 실시예 1 내지 257 중 어느 하나의 프로세서에 있어서, 프로세서는 DvmOp들 수신 시 CPU QACTIVE를 어썬트 해제한다.
- [0437] 259. 실시예 258의 프로세서에 있어서, CC6 흐름에서, 전력 감소는 모든 DVM 타겟에 대해 실패로서의 역할을 한다.
- [0438] 260. 실시예 258 또는 259의 프로세서에 있어서, 프로세서는 DVM 검출 시 저 전력 모드를 종료하도록 구성된다.
- [0439] 261. 실시예 1 내지 260 중 어느 하나의 프로세서에 있어서, 프로세서는 전력 게이팅 모드(CC6 모드)를 사용하도록 구성된다.
- [0440] 262. 실시예 261의 프로세서에 있어서, CC6 모드에 진입하기 위해, 노스브릿지가 클록에 대한 클로킹 요청에 대한 전력 토글을 요청한다.
- [0441] 263. 실시예 262의 프로세서에 있어서, 프로세서는 캐시 및 코어 상태들이 플러싱될 때 CC6 모드에 진입할 수 있다.
- [0442] 264. 실시예 261 내지 263 중 어느 하나의 프로세서에 있어서, CC6 모드 종료 시, 상태 복원이 요구된다.
- [0443] 265. 실시예 261 내지 264 중 어느 하나의 프로세서에 있어서, GIC는 프로그램 몇몇 모드 중 하나로 프로세서 인터페이스에 FIQ/IRQ 거동을 프로그래밍한다.
- [0444] 266. 실시예 265의 프로세서에 있어서, 몇몇 모드 중 하나는 GIC로부터의 nIRQCPU[n]/nFIQCPU[n] 핀들이 모든 상황에서 구동되는 정규 모드를 포함한다.
- [0445] 267. 실시예 265의 프로세서에 있어서, 몇몇 모드 중 하나는 GIC 입력을 레거시 FIQ/IRQ 핀들로 오버라이드하는 우회 모드를 포함한다.
- [0446] 268. 실시예 265의 프로세서에 있어서, 몇몇 모드 중 하나는 코어들에 대해 IRQ/FIQ 핀들을 마스킹하고 저 전력 종료를 위한 유형들의 인터럽트들의 존재 시 nIRQOUT[n]/nFIQOUT[n] 핀들을 전력 컨트롤러로 유도하는 우회 디세이블 모드를 포함한다.
- [0447] 269. 실시예 261 내지 268 중 어느 하나의 프로세서에 있어서, CC6 모드 종료 시, 타겟팅된 코어는 리셋하고 예외 벡터로부터 코드를 페치하기 시작한다.
- [0448] 270. 실시예 269의 프로세서에 있어서, 예외 벡터는 건너뛴 테이블에 리셋 핸들러를 포함한다.
- [0449] 271. 실시예 1 내지 270 중 어느 하나의 프로세서에 있어서, 프로세서는 C-상태에 진입하는 마지막 코어에 대해 L2 유지를 요청함으로써 복합 유지 모드(RXC3)에 진입하도록 구성된다.
- [0450] 272. 실시예 271의 프로세서에 있어서, RXC3 모드는 L2 스탠바이 핸드셰이크를 구현하는 것을 포함한다.
- [0451] 273. 실시예 272의 프로세서에 있어서, L2 스탠바이 핸드셰이크는 RCC3 모드 CC6 모드 중 어느 하나에 진입하는 마지막 코어 상에서 수행된다.
- [0452] 274. 실시예 1 내지 273 중 어느 하나의 프로세서에 있어서, 프로세서는 복합 전력 게이팅 모드(XC6)에 진입하도록 구성된다.
- [0453] 275. 실시예 274의 프로세서에 있어서, XC6 모드 동안, 개스킷은 또한 전력 차단된다.

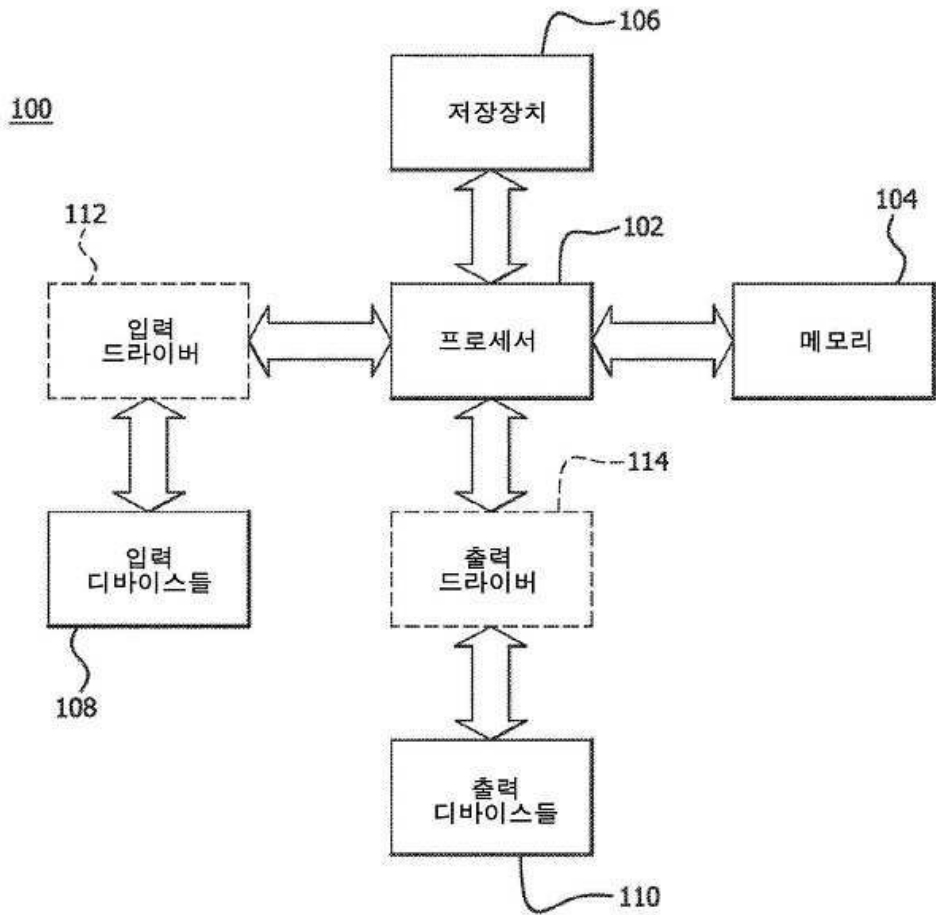
- [0454] 276. 실시예 275의 프로세서에 있어서, 개스킷은 모든 크레딧이 노스브리지에 의해 릴리즈될 때까지 노스브리지로 전달할 수 없다.
- [0455] 277. 실시예 275 또는 276의 프로세서에 있어서, 개스킷은 그것이 크레딧들을 수용할 준비가 되었을 때 ResetUcodeGo를 구동한다.
- [0456] 278. 실시예 1 내지 277 중 어느 하나의 프로세서에 있어서, 모니터 또는 대기 메커니즘 중 어느 하나에 의해 발생하는 플러시 요청 시, StpClk 신호가 타겟팅된 코어로 송신된다.
- [0457] 279. 실시예 278의 프로세서에 있어서, 타겟팅된 코어는 캐시를 플러싱하고 플러싱된 상태를 노스브리지에 시그널링하기 위해 적절한 비트들을 표시한다.
- [0458] 280. 실시예 279의 프로세서에 있어서, 노스브리지는 클록에 대한 클로킹/전력 요청을 발생할 때 더 깊은 C-상태로 진행한다.
- [0459] 281. 실시예 1 내지 280 중 어느 하나의 프로세서에 있어서, CPU 주파수 루틴은 성능 레벨 퍼센테이지를 이산적 P-상태 레벨에 매핑한다.
- [0460] 282. 실시예 281의 프로세서에 있어서, 이산적 P- 상태 레벨은 노스브리지에서의 CofVidCtl 레지스터를 통해 요청된다.
- [0461] 283. 실시예 281 또는 282의 프로세서에 있어서, 인터럽트 기반 작업 거부 및 비 작업 거부 루틴이 구현된다.
- [0462] 284. 실시예 283의 프로세서에 있어서, 작업 거부 루틴은 StpClk 기반 작업 거부 루틴을 포함한다.
- [0463] 285. 실시예 284의 프로세서에 있어서, StpClk이 안전 인터럽트로서 사용되는, FIQ로 후킹된다.
- [0464] 286. 실시예 284 또는 285의 프로세서에 있어서, 신뢰 존이 구현되면, P-상태 변경 및 StpClk 소스들이 안전 서비스들로서 등록된다.
- [0465] 287. 실시예 286의 프로세서에 있어서, 감독 모드가 이의 बैं킹된 링크 레지스터를 사용하기 위해 사용되고, 그렇게 함으로써 FIQ로의 재진입의 두 인스턴스를 가능하게 한다.
- [0466] 288. 실시예 287의 프로세서에 있어서, FIQ 예외 핸들러는 다른 신뢰 서비스들의 StpClk 선점을 지지하기 위해 감독 모드를 스위칭한다.
- [0467] 289. 실시예 284 내지 288 중 어느 하나의 프로세서에 있어서, P-상태 StpClk 제공 시, HTC SBC 시스템 관리 브로드캐스트가 노스브리지로 송신된다.
- [0468] 290. 실시예 289의 프로세서에 있어서, 모든 미처리 트래픽이 데이터 동기화 배리어(DSB; data synchronization barrier) 명령을 통해 작업 거부된다.
- [0469] 291. 실시예 290의 프로세서에 있어서, 인터럽트 대기(WFI; wait for interrupt)가 시그널링된다.
- [0470] 292. 실시예 291의 프로세서에 있어서, WFI 수신 시, 노스브리지는 프로브들이 선택되는 것을 블로킹함으로써 클러스터에 대한 프로브들을 작업 거부한다.
- [0471] 293. 실시예 292의 프로세서에 있어서, 노스브리지는 이미 선택된 프로브들을 떠나 이의 파이프라인을 드레인한다.
- [0472] 294. 실시예 293의 프로세서에 있어서, 노스브리지는 미처리 프로브들이 클러스터가 이들을 완료함에 따라 이들의 버퍼들을 할당 해제하기를 기다린다.
- [0473] 295. 실시예 293 또는 294의 프로세서에 있어서, 프로브들이 드레인되면, 노스브리지는 P-상태 변경을 스케줄링하기 전 클러스터와 ACINACTM/STANDBYWFI2 핸드셰이크를 수행한다.
- [0474] 296. 실시예 281 내지 295 중 어느 하나의 프로세서에 있어서, 클러스터가 P-상태 변경에 있는 동안, 클러스터는 인터럽트 취하거나 시스템 요청을 형성하지 않을 수 있다.
- [0475] 297. 실시예 296의 프로세서에 있어서, 인터럽트 예지들이 노스브리지에 의해 마스킹된다.
- [0476] 298. 실시예 296 또는 297의 프로세서에 있어서, 노스브리지는 P-상태 변경이 완료되면 작업 거부로부터 코어를 기상시킨다.

- [0477] 299. 실시예 298의 프로세서에 있어서, 노스브리지는 StartClk을 통해 코어를 기상시킨다.
- [0478] 300. 실시예 299의 프로세서에 있어서, StartClk은 가장 높은 FIQ 우선순위를 갖는다.
- [0479] 301. 실시예 298의 프로세서에 있어서, 노스브리지는 이벤트 대기(WFE; wait for event)를 통해 코어를 기상시킨다.
- [0480] 302. 실시예 301의 프로세서에 있어서, WFE는 EVENTI 핀을 통한 이벤트들 또는 인터럽트들로부터의 트리거들이다.
- [0481] 303. 실시예 301 또는 302의 프로세서에 있어서, 코어가 WFE를 종료할 때, 그것은 StpClk 핸들러에 있을 수 있으며, 이는 인터럽트들을 마스킹하지 않고 종료될 것이다.
- [0482] 304. 실시예 281 내지 303 중 어느 하나의 프로세서에 있어서, 개스킷은 작업 거부를 구현한다.
- [0483] 305. 실시예 304의 프로세서에 있어서, 클러스터 및 개스킷 간 ACE 인터페이스는 계속해서 작동하고; NBSYN의 단부 중 어느 하나 상의 CCI 인터페이스가 작업 거부된다.
- [0484] 306. 실시예 304 또는 305의 프로세서에 있어서, 개스킷은 새로운 요청들을 블로킹하고 미처리 응답들을 기다린다.
- [0485] 307. 실시예 304 내지 306 중 어느 하나의 프로세서에 있어서, P-상태 변경 요청 시, 노스브리지는 작업 거부하기 위해 사이드밴드를 통해 개스킷에 시그널링한다.
- [0486] 308. 실시예 307의 프로세서에 있어서, 개스킷은 노스브리지로 송신된 것으로부터 새로운 비-라이트백 요청들을 블로킹하고 모든 미처리 라이트백을 드레인한다.
- [0487] 309. 실시예 308의 프로세서에 있어서, 개스킷은 모든 미처리 라이트백이 드레인될 때 작업 거부 응답을 노스브리지로 시그널링한다.
- [0488] 310. 실시예 309의 프로세서에 있어서, 노스브리지는 선택된 것으로부터 새로운 프로브들을 블로킹하고 미처리 프로브들이 완료하기를 기다린다.
- [0489] 311. 실시예 310의 프로세서에 있어서, 개스킷은 단지 어떤 프로브도 노스브리지로부터 미처리되지 않고 있을 때 라이트백을 블로킹한다.
- [0490] 312. 실시예 311의 프로세서에 있어서, 개스킷은 노스브리지에 계류 중인 라이트백들이 존재하는 한 NbBusy를 구동한다.
- [0491] 313. 실시예 312의 프로세서에 있어서, 노스브리지는 단지 모든 프로브 버퍼가 리턴되고 개스킷이 NbBusy를 어써트하고 있지 않을 때 P-상태 변경을 개시하도록 허용된다.
- [0492] 314. 실시예 304 내지 313 중 어느 하나의 프로세서에 있어서, 비 작업 거부가 작업 거부 요청을 비활성화하는 노스브리지 및 작업 거부 응답을 제거함으로써 응답하는 개스킷에 의해 수행된다.
- [0493] 315. 실시예 314의 프로세서에 있어서, 노스브리지는 응답이 새로운 요청을 개시하기 전 이전 핸드셰이크로부터 어써트 해제되었다는 것을 보장한다
- [0494] 본 출원에서의 개시 내용에 기초하여 많은 변경이 가능하다는 것이 이해되어야 한다. 피처들 및 요소들이 특정한 조합들로 위에서 설명되지만, 각 피처 또는 요소가 다른 피처들 또는 요소들 없이 단독으로 또는 다른 피처들 및 요소들을 갖거나 갖지 않는 다양한 조합으로 사용될 수 있다.
- [0495] 제공된 방법들은 범용 컴퓨터, 프로세서, 또는 프로세서 코어로 구현될 수 있다. 적합한 프로세서들은 예로서, 범용 프로세서, 전용 프로세서, 종래 프로세서, 디지털 신호 프로세서(DSP), 복수의 마이크로프로세서, DSP 코어와 관련하여 하나 이상의 마이크로프로세서, 컨트롤러, 마이크로컨트롤러, 어플리케이션 특정 집적 회로(ASIC), 필드 프로그램가능 게이트 어레이들(FPGA들) 회로들, 임의의 다른 유형의 집적 회로(IC), 및/또는 상태 기계를 포함한다. 그러한 프로세서들은 프로세싱된 하드웨어 기술 언어(HDL) 명령들 및 넷 리스트를 포함하는 다른 중개자 데이터(컴퓨터 판독가능한 매체 상에 저장될 수 있는 명령들)의 결과들을 사용하여 제조 프로세스를 구성함으로써 제조될 수 있다. 그러한 프로세싱의 결과는 그 후 실시예들의 측면들을 구현하는 프로세서를 제조하기 위한 반도체 제조 프로세스에서 사용되는 마스크워크들일 수 있다.
- [0496] 본 출원에서 제공된 방법들 또는 흐름도들은 범용 컴퓨터 또는 프로세서에 의해 실행하기 위해 비-일시적 컴퓨

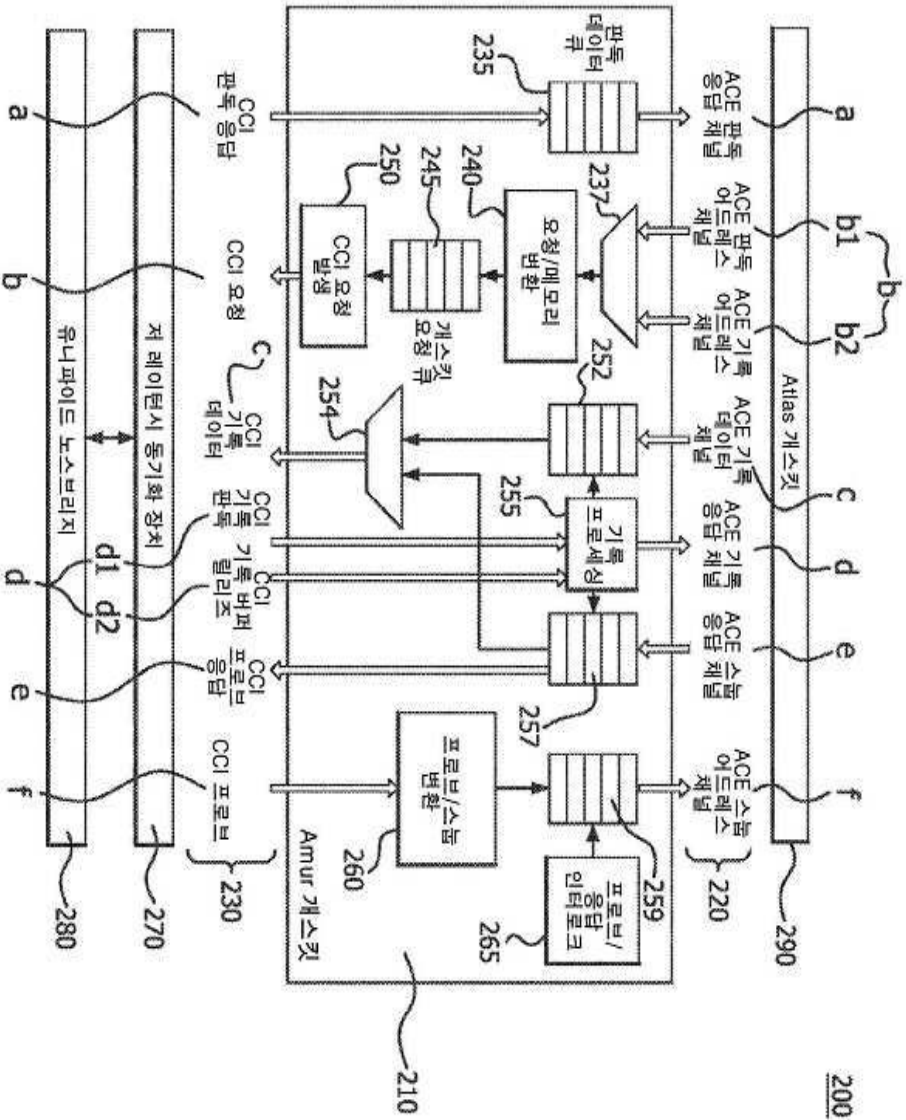
터-판독가능한 저장 매체로 통합되는 컴퓨터 프로그램, 소프트웨어, 또는 펌웨어로 구현될 수 있다. 과 같은과 같은비-일시적 컴퓨터-판독가능한 저장 매체들의 예들은 판독 전용 메모리(ROM), 랜덤 액세스 메모리(RAM), 레지스터, 캐시 메모리, 반도체 메모리 디바이스, 자기 미디어 이블테면 내부의 하드 디스크들 및 착탈가능한 디스크들, 광 자기 매체, 및 광 매체 이블테면 CD-ROM 디스크들, 및 디지털 다목적 디스크들(DVD들)을 포함한다.

도면

도면1

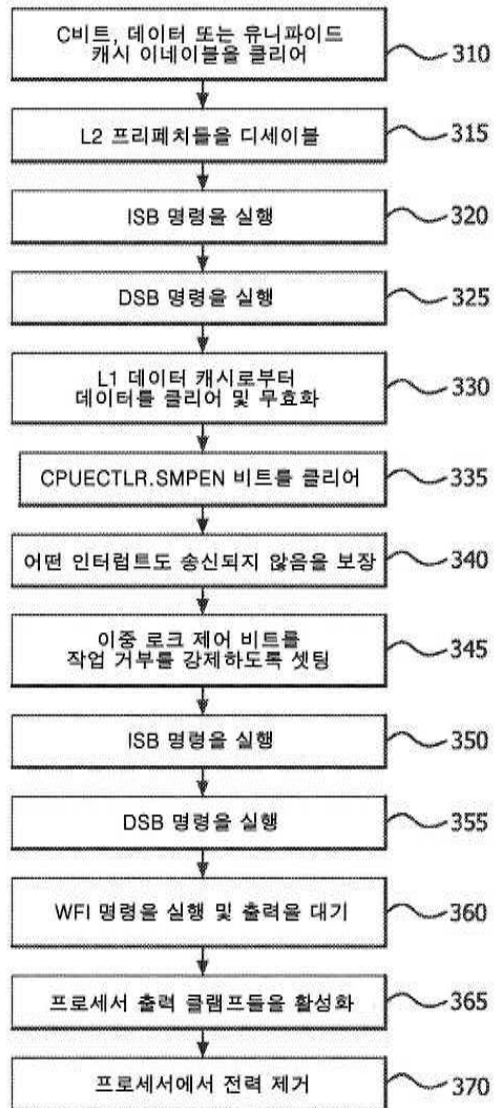


도면2

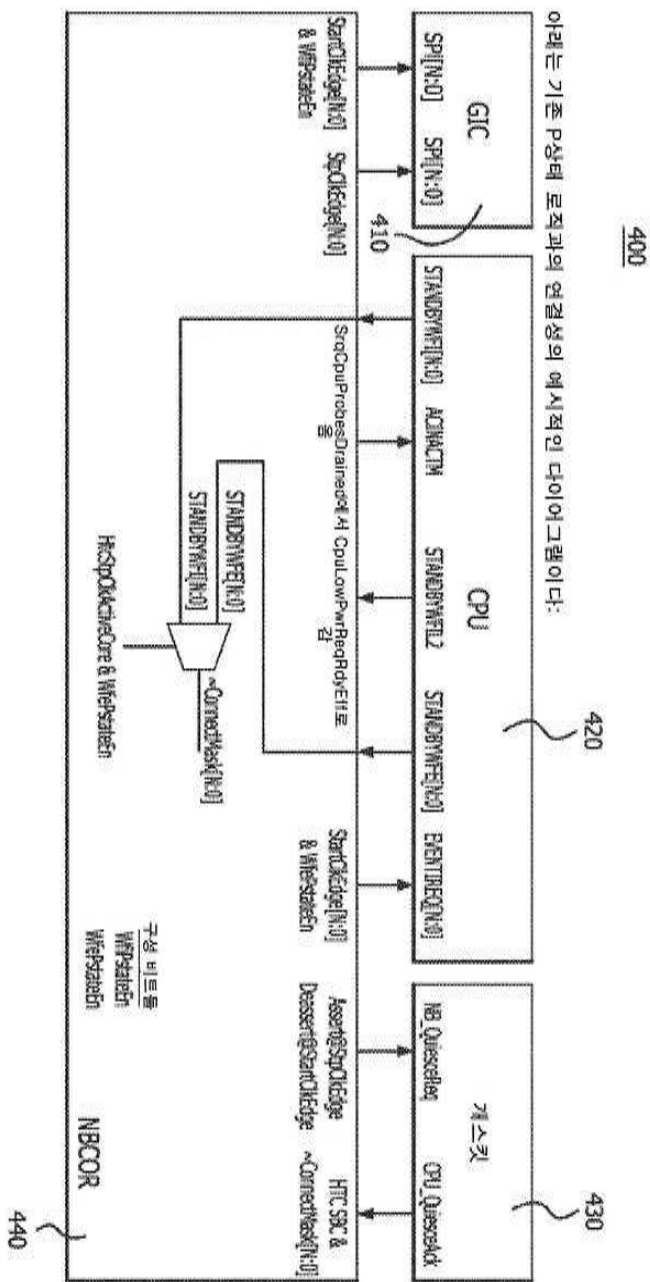


도면3

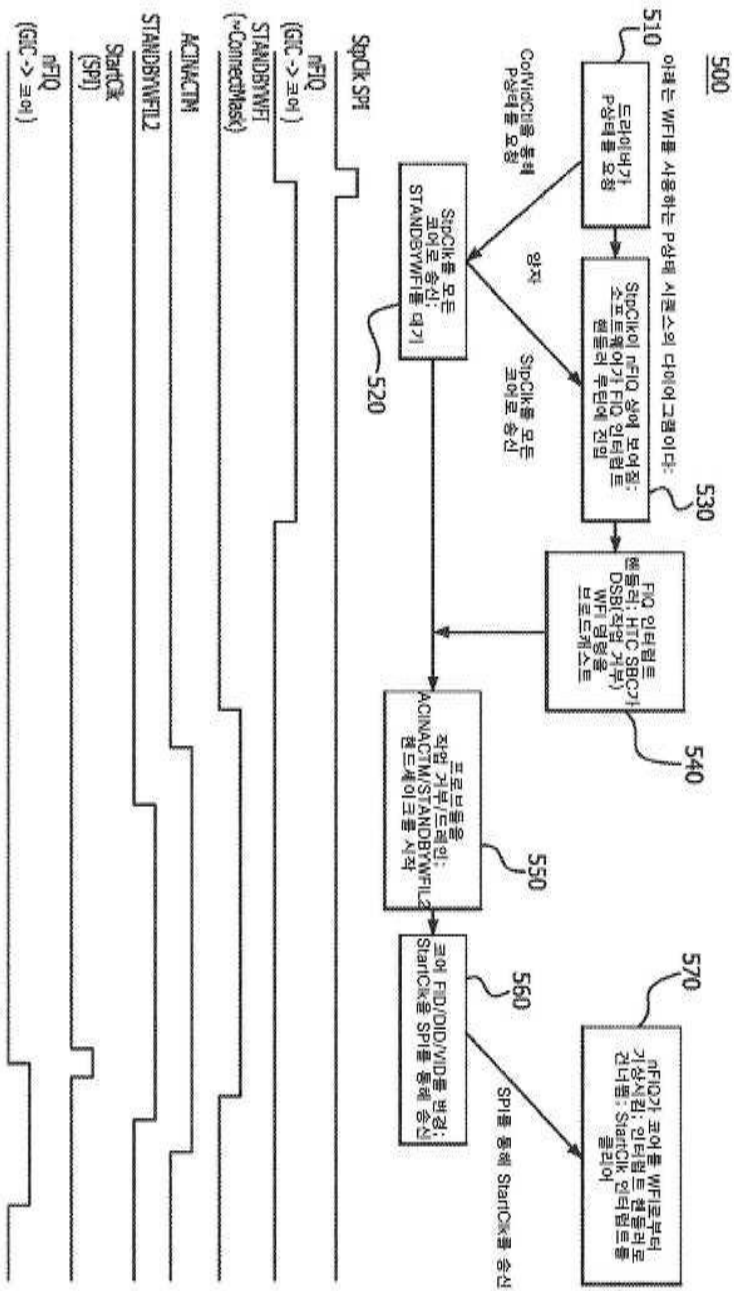
300



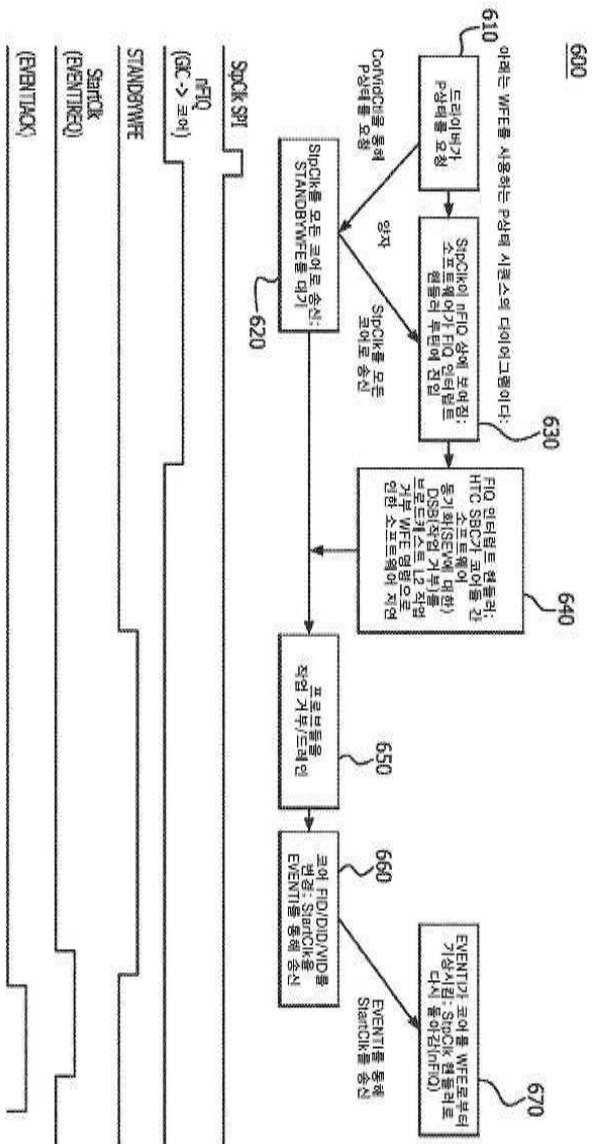
도면4



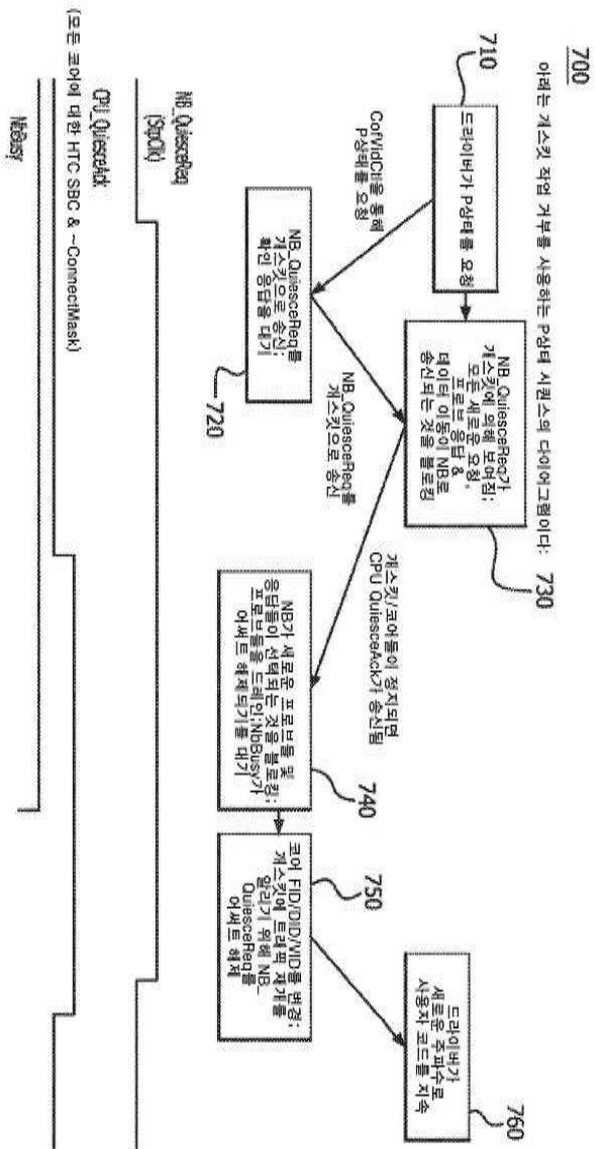
도면5



도면6

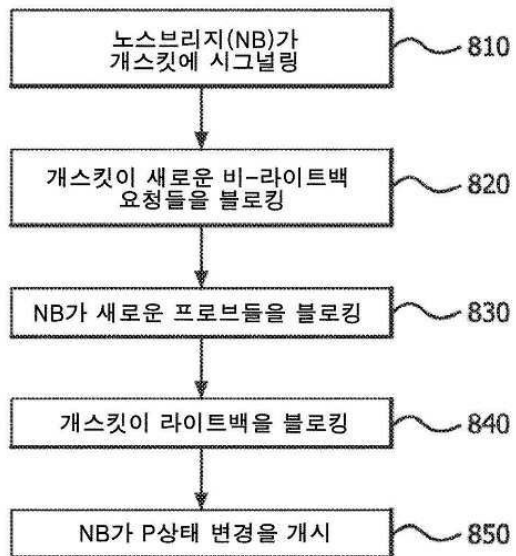


도면7



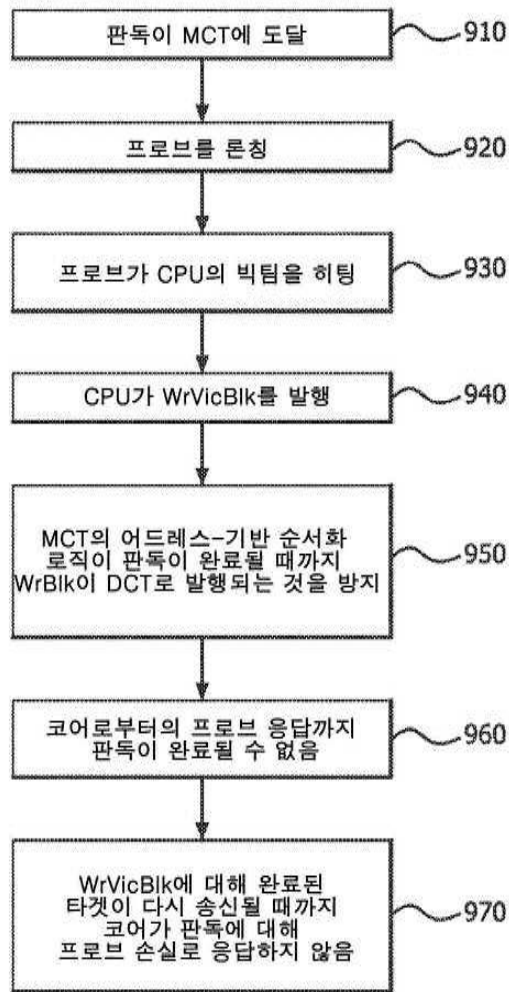
도면8

800



도면9

900



【심사관 직권보정사항】

【직권보정 1】

【보정항목】 청구범위

【보정세부항목】 청구항 제6항의 2번째 줄

【변경전】

상기 액세스 플래그 필드(access flag field)는

【변경후】

액세스 플래그 필드(access flag field)는

【직권보정 2】

【보정항목】 청구범위

【보정세부항목】 청구항 제16항의 2번째 줄

【변경전】

상기 액세스 플래그 필드는

【변경후】

액세스 플래그 필드는

【직권보정 3】

【보정항목】 청구범위

【보정세부항목】 청구항 제11항의 8번째 줄

【변경전】

상기 하드웨어는

【변경후】

하드웨어는